

Estudio de P4 y su papel en la telemetría de red en banda para las redes 5G y beyond

Edinson Montero Beltre

January 18, 2026

Contents

Dedicatoria	6
Acrónimos	7
1 Introducción	11
1.1 Contexto general de las redes 5G y beyond	11
1.2 Motivación del estudio	11
1.3 Problemática en la visibilidad y monitoreo de redes 5G y beyond	12
1.4 Hipótesis y preguntas de investigación	12
1.5 Objetivos generales y específicos	13
1.6 Metodología de trabajo	13
1.7 Estructura del documento	14
1.8 Resumen de capítulos	14
2 Fundamentos de las Redes 5G y Beyond	16
2.1 Evolución de tecnologías móviles	16
2.1.1 De 1G a 6G: Hitos clave	16
2.2 Arquitectura NR en 5G	17
2.3 Arquitectura 5G Non Standalone (NSA)	18
2.4 Arquitectura 5G Standalone (SA)	20
2.4.1 Visión general de la arquitectura 5G SA	20
2.4.2 Arquitectura basada en servicios (SBA)	21
2.4.3 Interfaces HTTP REST	22
2.4.4 Componentes de 5G Core (5GC)	23
2.4.5 Componentes principales de 5GC	23
2.4.6 Pila de protocolos del plano de control y plano de usuario	25
2.4.6.1 Plano de Control	25
2.4.6.2 Plano de Usuario	26
2.4.7 Interfaces clave en 5GC	26
2.4.8 Tendencias emergentes en redes beyond 5G	27

3	Telemetría y QoS en 5G	29
3.1	Desafíos de Telemetría en Redes 5G	30
3.1.1	Limitaciones de mecanismos tradicionales (SNMP, Net- Flow, sFlow)	30
3.1.2	Requisitos de visibilidad en URLLC, eMBB y mMTC .	30
3.2	In-Band Network Telemetry (INT)	30
3.2.1	Concepto y motivación	30
3.2.2	Arquitectura INT: source, transit y sink	30
3.2.3	Metadatos INT-MD y su estructura	30
3.2.4	INT vs telemetría out-of-band	30
3.3	Calidad de Servicio en 5G	30
3.4	QoS basado en 5G Flujo	30
3.4.1	Definición de flujo en 5G	30
3.4.2	Identificación y clasificación de flujos	30
3.4.3	Mecanismos de gestión de QoS por flujo	30
3.5	Señalización de QoS en 5G	30
3.6	Características de QoS en 5G	30
4	Fundamentos de P4 y Programación del Plano de Datos	31
4.1	Introducción a P4	31
4.1.1	Historia y evolución	31
4.1.2	Modelo de arquitectura PISA	31
4.1.3	P4_14 vs P4_16	31
4.2	Herramientas y ecosistema P4	31
4.2.1	BMv2 como switch software	31
4.2.2	P4C: compilador	31
4.2.3	P4Runtime: control del data plane	31
4.3	Implementación de INT en P4	31
4.3.1	Definición de cabeceras INT	31
4.3.2	Parsing y deparsing en BMv2	31
4.3.3	Inyección hop-by-hop de metadatos	31
4.3.4	Limitaciones del pipeline P4	31
5	Entorno de Desarrollo Implementado	32
5.1	Arquitectura general del sistema	32
5.1.1	Diseño de la red 5G SA	33
5.1.1.1	Diseño de alto nivel (HLD)	34
5.1.1.2	Diseño de bajo nivel (LLD)	34
5.2	Despliegue del Core 5G SA	35
5.2.1	Configuración de Core-Host	35
5.2.1.1	Obtención del código fuente 5GC	35

5.2.1.2	Configuración de Docker Compose	37
5.2.1.2.1	DB	37
5.2.1.2.2	NRF	38
5.2.1.2.3	AUSF	39
5.2.1.2.4	NSSF	39
5.2.1.2.5	PCF	40
5.2.1.2.6	UDM	40
5.2.1.2.7	UDR	41
5.2.1.2.8	Despliegue del CHF	42
5.2.1.2.9	Despliegue del NEF	43
5.2.1.2.10	Despliegue del WebUI	43
5.2.1.2.11	Configuración de Red y Volúmenes	44
5.2.1.3	Construcción y despliegue	44
5.2.2	Configuración de AMF	46
5.2.2.1	Construcción de la imagen Docker del AMF	47
5.2.2.2	Despliegue del contenedor AMF	47
5.2.2.3	Registro del AMF en el NRF	48
5.2.3	Configuración de SMF	50
5.2.3.1	Instalación del módulo GTP	50
5.2.3.2	Obtención e instalación del código fuente SMF	56
5.2.3.3	Configuración de Docker Compose	56
5.2.3.4	Registro del SMF en el NRF y conexión N4 con UPF	57
5.2.4	Configuración de UPF	59
5.2.4.1	Configuración de Docker Compose	59
5.2.4.2	Registro del UPF en el NRF	60
5.2.4.3	Conexión N4 con SMF	61
5.3	Implementación Red de Transporte con INT	62
5.4	Despliegue de switches BMv2 con INT-MD	63
5.4.1	Switch INT Source	64
5.4.1.1	Definición de Constantes	64
5.4.1.2	Definición de Cabeceras	64
5.4.1.3	Parser	66
5.4.1.4	Ingress	68
5.4.1.5	Egress	71
5.4.1.6	Deparser	72
5.4.2	Switch INT Transit	73
5.4.2.1	Parser	73
5.4.2.2	Ingress	75
5.4.2.3	Egress	76
5.4.2.4	Deparser	77

5.4.3	Switch INT Sink	78
5.4.3.1	Parser	78
5.4.3.2	Ingress	79
5.4.3.3	Egress	81
5.4.3.4	Deparser	84
5.4.4	Configuración de los Switches INT	85
5.5	Servidor de recolección y análisis	86
5.5.1	Instalación Grafana y InfluxDB	86
5.5.2	Creacion de la base de datos InfluxDB	87
5.5.3	Configuration de Grafana	88
5.5.4	Captura y procesamiento de paquetes INT	89
5.5.4.1	Configuración y dependencias	90
5.5.4.2	Detección automática de endianness	91
5.5.4.3	Parsing de cabeceras INT	92
5.5.4.4	Extracción y validación de metadatos	94
5.5.4.5	Almacenamiento en InfluxDB	96
5.5.4.6	Ejecución y monitoreo	98
5.6	Despliegue del NGRAN con UERANSIM	99
5.6.1	Despliegue de UERANSIM	100
5.6.2	Configuración del gNB	100
6	Resultados	101
6.1	Validación funcional	101
6.1.1	INT en tráfico N2	101
6.1.2	INT en tráfico N3	101
6.1.3	Recepción de metadatos en el servidor	101
6.2	Presentación de resultados	101
7	Conclusiones y Trabajo Futuro	102
7.1	Conclusiones principales	102
7.2	Contribuciones del trabajo	102
7.3	Limitaciones del estudio	102
7.4	Líneas futuras de investigación	102
7.4.1	INT en 6G	102
7.4.2	UPF programable con P4	102
7.4.3	IA para análisis de telemetría	102
A	Apéndices	103
A.1	Código	103
A.2	Topologías de red	103
A.3	Configuraciones del core 5G	103

A.4	Capturas de tráfico	103
A.5	Dashboards de Grafana	103
A.6	Scripts y herramientas auxiliares	103

Dedictory

A quienes me apoyaron en este proyecto.

Acrónimos

Acrónimo	Significado
5G	Fifth Generation (Quinta Generación)
5GC	5G Core (Núcleo de Red 5G)
API	Application Programming Interface (Interfaz de Programación de Aplicaciones)
DNS	Domain Name System (Sistema de Nombres de Dominio)
INT	In-band Network Telemetry (Telemetría en Banda)
IP	Internet Protocol (Protocolo de Internet)
MIMO	Multiple Input Multiple Output (Entrada Múltiple Salida Múltiple)
NFV	Network Functions Virtualization (Virtualización de Funciones de Red)
P4	Programming Protocol-Independent Packet Processors
QoS	Quality of Service (Calidad de Servicio)
SDN	Software-Defined Networking (Redes Definidas por Software)
TCP	Transmission Control Protocol (Protocolo de Control de Transmisión)
TLS	Transport Layer Security (Seguridad de la Capa de Transporte)
UDP	User Datagram Protocol (Protocolo de Datagrama de Usuario)
UE	User Equipment (Equipo de Usuario)
UL	Uplink (Enlace Ascendente)
UMTS	Universal Mobile Telecommunications System (Sistema Universal de Telecomunicaciones Móviles)
UPF	User Plane Function (Función del Plano de Usuario)
URLLC	Ultra-Reliable Low-Latency Communications (Comunicaciones de Ultra Fiabilidad y Baja Latencia)

Contents

List of Figures

2.1	Opciones de implementación de NR en 5G: NSA y SA [1]. . .	18
2.2	Arquitectura 5GC basada en interfaces basadas en servicios [4].	21
2.3	Arquitectura 5GC con interfaces punto a punto. [4].	22
2.4	Pila de protocolos del plano de control en 5GC.	25
2.5	Pila de protocolos del plano de usuario en 5GC.	26
5.1	Arquitectura general	33
5.2	Topología de red implementada en GNS3	34
5.3	Clonación del repositorio de free5gc	35
5.4	Clonación de base de NFs adicionales	36
5.5	Compilación de NFs adicionales	37
5.6	Construcción de imágenes en Core-Host	45
5.7	Despliegue de contenedores en Core-Host	46
5.8	Construcción de la imagen Docker del AMF	47
5.9	Registro del AMF en el NRF	49
5.10	Verificación del registro del AMF en el NRF	50
5.11	Instalación de kernel compatible con GTP	51
5.12	Edición del archivo y actualización de /etc/default/grub . . .	52
5.13	Clonación del repositorio del módulo GTP	53
5.14	Compilación e instalación del módulo GTP	54
5.15	Carga y verificación del módulo GTP	55
5.16	Verificación del módulo GTP en los logs del sistema	56
5.17	Registro del SMF en el NRF	58
5.18	UPF registro y preparacion de interfaz N3	61
5.19	UPF registro y preparacion de interfaz N3	62
5.20	Topologia INT-MD	63
5.21	Pantalla de inicio de Grafana	87
5.22	Creación de la base de datos InfluxDB	87
5.23	Configuración de la fuente de datos InfluxDB en Grafana . . .	88
5.24	Configuración de la fuente de datos InfluxDB en Grafana - Parte 2	89

List of Tables

5.1	Diseño de bajo nivel (LLD)	34
-----	----------------------------	----

Chapter 1

Introducción

1.1 Contexto general de las redes 5G y beyond

Las redes de quinta generación (5G) representan un avance significativo en la evolución de las telecomunicaciones móviles, ofreciendo mayores velocidades, menor latencia y una capacidad mejorada para conectar dispositivos masivos. Con la creciente demanda de servicios en tiempo real y aplicaciones críticas, como la realidad aumentada, los vehículos autónomos y la telemedicina, en donde la latencia y la fiabilidad son esenciales debido a que cualquier retraso puede tener consecuencias graves, la necesidad de una visibilidad y monitoreo efectivos de la red se ha vuelto crucial. En esta evolución, las bajas latencias son cruciales para los nuevos servicios en la actualidad y los futuros, incluyendo la cuarta revolución industrial.

1.2 Motivación del estudio

La complejidad y dinamismo de las redes 5G y superiores, plantean desafíos significativos para la gestión y el monitoreo de la red. Los métodos tradicionales de telemetría, como SNMP y NetFlow, a menudo no proporcionan la granularidad y la rapidez necesarias para detectar y resolver problemas en tiempo real otorgando una visibilidad de extremo a extremo sobre el estado de la red de forma precisa. La telemetría en banda (In-Band Network Telemetry, INT) emerge como una solución prometedora para abordar estas limitaciones, permitiendo la recopilación de datos detallados directamente desde los paquetes que atraviesan la red. Este estudio se centra en explorar el papel de P4, un lenguaje de programación para definir el comportamiento

de los dispositivos de red, en la implementación y optimización de soluciones de telemetría en banda para redes 5G y beyond.

1.3 Problemática en la visibilidad y monitoreo de redes 5G y beyond

A medida que las redes 5G se vuelven más complejas, la visibilidad y el monitoreo efectivos se convierten en desafíos críticos. La diversidad de servicios y la naturaleza dinámica del tráfico generan dificultades para identificar cuellos de botella, latencias elevadas y otros problemas de rendimiento. Además, la necesidad de cumplir con estrictos requisitos de calidad de servicio (QoS) y experiencia del usuario (QoE) exige soluciones de monitoreo que puedan adaptarse rápidamente a las condiciones cambiantes de la red. La problemática radica en cómo implementar mecanismos de telemetría que sean capaces de proporcionar datos precisos y en tiempo real sin introducir una sobrecarga significativa en la red.

1.4 Hipótesis y preguntas de investigación

La hipótesis central de este estudio es que la implementación de telemetría en banda utilizando P4 puede mejorar significativamente la visibilidad y el monitoreo de las redes 5G y beyond, permitiendo una gestión más eficiente y una mejor calidad de servicio. Las preguntas de investigación que guían este estudio incluyen:

- ¿Cómo puede P4 facilitar la implementación de soluciones de telemetría en banda en redes 5G y beyond?
- ¿Qué beneficios específicos ofrece la telemetría en banda en comparación con los métodos tradicionales de monitoreo?
- ¿Cuáles son los desafíos y limitaciones asociados con el uso de P4 para telemetría en banda en entornos 5G y beyond?
- ¿Cómo afecta la telemetría en banda al rendimiento general de la red y a la experiencia del usuario?

1.5 Objetivos generales y específicos

El objetivo general de este estudio es evaluar el papel de P4 en la implementación de telemetría en banda para mejorar la visibilidad y el monitoreo de las redes 5G y beyond. Los objetivos específicos incluyen:

- Analizar las capacidades de P4 para definir y programar el comportamiento de los dispositivos de red en el contexto de la telemetría en banda, específicamente INT-MD.
- Diseñar e implementar un prototipo de telemetría en banda (INT-MD) utilizando P4 en un entorno de red 5G SA simulado.
- Evaluar el rendimiento y la efectividad de la solución propuesta en términos de visibilidad, latencia y sobrecarga de la red.
- Identificar los desafíos y limitaciones encontrados durante la implementación y proponer posibles soluciones o mejoras.

1.6 Metodología de trabajo

Este estudio adoptará un enfoque experimental y analítico para investigar el papel de P4 en la telemetría en banda para redes 5G y beyond. La metodología incluirá las siguientes etapas:

- Revisión bibliográfica: Se realizará una revisión exhaustiva de la literatura existente sobre telemetría en banda, P4 y redes 5G y beyond para establecer un marco teórico sólido.
- Diseño del prototipo: Se diseñará un prototipo de telemetría en banda utilizando P4, definiendo las tablas y acciones necesarias para insertar metadatos INT-MD en el tráfico N2 (SCTP) y N3 (GTP-U).
- Implementación: El prototipo se implementará en un entorno emulado usando herramientas como GNS3 y VMware workstation que contará con Routers Cisco y switches P4 (BMv2), los cuales serán programados para soportar INT-MD y por ultimo, un servidor sink para la recolección y análisis de los datos de telemetría usando influxDB y Grafana para representar los datos. Este prototipo integrará un core 5G SA básico y una red de distribución programable con P4.

- Evaluación: Se llevarán a cabo pruebas para evaluar el rendimiento del prototipo, midiendo métricas como la latencia, la sobrecarga de la red y la precisión de los datos recopilados, así como el impacto de la telemetría en banda en la calidad del servicio.
- Análisis de resultados: Los resultados obtenidos se analizarán críticamente para identificar beneficios, desafíos y áreas de mejora.

1.7 Estructura del documento

El documento se estructura en varios capítulos que abordan diferentes aspectos del estudio:

- Capítulo 1: **Introducción** - Presenta el contexto, la motivación, los objetivos y la metodología del estudio.
- Capítulo 2: **Redes 5G** - Proporciona una visión general de las redes 5G, sus características y desafíos.
- Capítulo 3: **Telemetría en Redes 5G** - Explora la necesidad de telemetría, los métodos tradicionales y la telemetría en banda (INT).
- Capítulo 4: **Fundamentos de P4** - Describe el lenguaje P4, su arquitectura y capacidades relevantes para la telemetría.
- Capítulo 5: **Entorno de Desarrollo** - Detalla el entorno utilizado para implementar y probar el prototipo de telemetría en banda.
- Capítulo 6: **Resultados** - Presenta los resultados obtenidos durante las pruebas del prototipo.
- Capítulo 7: **Discusión** - Analiza críticamente los resultados, comparándolos con el estado del arte y discutiendo beneficios y desafíos.
- Capítulo 8: **Conclusiones y Trabajo Futuro** - Resume las conclusiones principales, contribuciones, limitaciones y propone líneas futuras de investigación.

1.8 Resumen de capítulos

Cada capítulo del documento se enfoca en aspectos específicos del estudio, proporcionando una comprensión integral del papel de P4 en la telemetría

en banda para redes 5G. A lo largo del documento, se integran conceptos teóricos con aplicaciones prácticas, culminando en un análisis crítico de los resultados obtenidos y su relevancia para el campo de las telecomunicaciones móviles.

Chapter 2

Fundamentos de las Redes 5G y Beyond

En este capítulo se presentan los conceptos fundamentales de las redes 5G y beyond, incluyendo su evolución desde generaciones anteriores, las características clave de la arquitectura 5G, y las tendencias emergentes hacia futuras generaciones de redes móviles. Se abordan aspectos técnicos como la arquitectura del núcleo de red 5GC, las funciones principales del plano de control y del plano de usuario, así como las tecnologías habilitadoras que sustentan el despliegue y operación de estas redes avanzadas.

2.1 Evolución de tecnologías móviles

La evolución de las tecnologías móviles ha sido un proceso continuo que ha transformado la forma en que las personas se comunican y acceden a la información. Desde la primera generación (1G) hasta la actual quinta generación (5G) y más allá, cada etapa ha introducido avances significativos en términos de velocidad, capacidad, latencia y funcionalidad. A continuación, se presenta un resumen de la evolución de las tecnologías móviles:

2.1.1 De 1G a 6G: Hitos clave

- **1G:** Introducción de la comunicación analógica.
- **2G:** Digitalización de la voz y servicios básicos de datos (SMS).
- **3G:** Introducción de datos móviles y servicios multimedia.
- **4G:** Redes IP y mayor velocidad de datos.

- **LTE y LTE-Advanced:** Mejoras en eficiencia espectral y latencia.
- **5G Non-Standalone (NSA):** Uso combinado de 4G y 5G para una transición suave. Aumentando la capacidad y velocidad de la red.
- **5G Standalone (SA):** Arquitectura completamente nueva basada en servicios y virtualización, permitiendo nuevas funcionalidades y mejoras en latencia y confiabilidad.
- **5G Advanced:** Mejoras en IA, eficiencia energética y soporte para nuevas aplicaciones, como XR y comunicaciones vehiculares.
- **Beyond-5G:** Investigación en tecnologías emergentes como comunicaciones cuánticas y redes holográficas, con miras a la futura generación 6G.
- **Visión de 6G:** Redes ultra confiables, latencia casi nula y capacidades de inteligencia artificial integradas de manera nativa.
- **Tecnologías emergentes:** Comunicaciones terahertz, redes auto-organizadas y computación en el borde (Edge Computing).
- **Aplicaciones futuras:** Realidad extendida (XR), ciudades inteligentes (Smart Cities) y redes de sensores masivos (IoT masivo).
- **Desafíos:** Seguridad, privacidad y sostenibilidad ambiental.

2.2 Arquitectura NR en 5G

3GPP introdujo 6 opciones de implementación para la tecnología New Radio (NR) como se muestra en la figura 2.1. Estas opciones se dividen en dos categorías principales: **Non-Standalone (NSA)** y **Standalone (SA)**. SA representa una arquitectura con NR como única tecnología de acceso, mientras que NSA combina NR con la infraestructura existente de 4G LTE. Las opciones 1, 2, 5 son implementaciones SA, mientras que las opciones 3, 4 y 7 son implementaciones NSA. Cabe destacar que la opción 1 es puramente LTE sin NR.

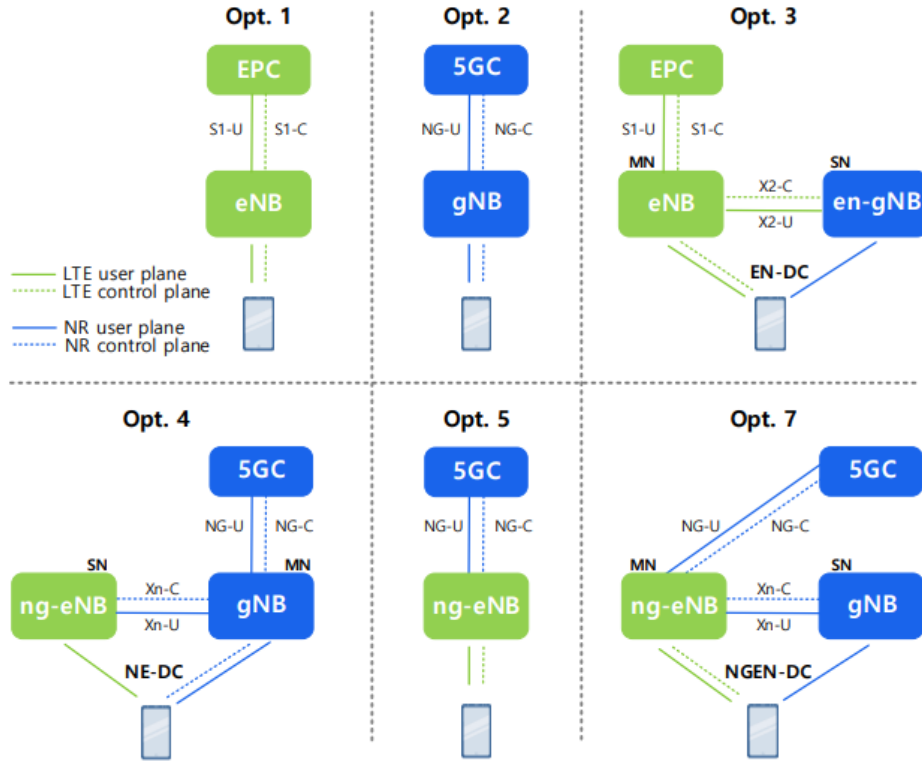


Figure 2.1: Opciones de implementación de NR en 5G: NSA y SA [1].

2.3 Arquitectura 5G Non Standalone (NSA)

La arquitectura 5G Non-Standalone (NSA) es una configuración de red que permite la coexistencia y colaboración entre las tecnologías 4G LTE y 5G NR. En esta arquitectura, la red 4G LTE actúa como la red principal para la señalización y el control, mientras que la red 5G NR se utiliza principalmente para el transporte de datos de alta velocidad. Esta coexistencia se logra mediante **Multi-Radio Dual Connectivity (MR-DC)**, descrito mas adelante y que permite a los dispositivos de usuario (UE) conectarse simultáneamente a dos distintas generaciones de red de radio. Las opciones pueden ser:

- **EN-DC (E-UTRA-NR Dual Connectivity)**: *Opcion 3*, donde el UE se conecta a una estación base LTE (eNodeB) como nodo maestro y a una estación base NR (gNodeB) como nodo secundario. El eNodeB

gestiona la señalización y el control, mientras que el gNodeB se utiliza principalmente para la transferencia de datos de alta velocidad.

- **NGEN-DC (NG-RAN E-UTRA-NR Dual Connectivity):** *Opcion 7*, similar a la opcion 3, pero con una integración más estrecha entre las estaciones base LTE y NR. En este modo, el eNodeB y el gNodeB están conectados a través de la interfaz Xn, lo que permite una mejor coordinación y gestión de recursos entre ambas tecnologías.
- **NE-DC (NR-E-UTRA Dual Connectivity):** *Opcion 4*, donde el UE se conecta a ng-eNB como MN y gNB como SN y ambas estaciones base están conectadas a través de la interfaz Xn. En este modo, el ng-eNB gestiona la señalización y el control, mientras que el gNB se utiliza para la transferencia de datos.
- **NR-DC (NR-NR Dual Connectivity):** *Opcion 2*, En la que UE se conecta a dos estaciones base NR (gNodeB) como nodos maestro y secundario. Ambas estaciones base están conectadas a través de la interfaz Xn, lo que permite una coordinación y gestión de recursos más eficiente entre ellas.

La arquitectura NSA permite a los proveedores de servicios móviles activar capacidades 5G de forma gradual sin necesidad de reemplazar completamente la infraestructura de núcleo de red existente. El despliegue de NSA comienza típicamente con la instalación de nuevas estaciones base NR que coexisten con las estaciones base LTE existentes. Los dispositivos del usuario (UE) que soportan tanto LTE como NR pueden conectarse simultáneamente a ambas redes, aprovechando la mejor señal disponible para la señalización de control a través de LTE y utilizando NR para el tráfico de datos cuando está disponible. Este enfoque dual proporciona beneficios inmediatos de rendimiento sin requerir una arquitectura de núcleo completamente nueva, lo que reduce significativamente los costos de inversión durante la transición. Desde el punto de vista del usuario final, la arquitectura NSA ofrece una experiencia mejorada en comparación con las redes LTE puras. Los usuarios pueden experimentar velocidades de descarga más altas (potencialmente en el rango de gigabits por segundo), menor latencia en aplicaciones específicas de datos, y mejor eficiencia general de la red. Sin embargo, las ventajas están limitadas principalmente al tráfico de datos, ya que los servicios de señalización y control siguen estando limitados por las especificaciones de LTE. Para casos de uso que requieren baja latencia extrema o requisitos ultra confiables (como comunicaciones críticas), la arquitectura NSA puede no ser suficiente, lo que subraya la necesidad eventual de migrar a 5G SA para alcanzar todas las capacidades de 5G.

2.4 Arquitectura 5G Standalone (SA)

2.4.1 Visión general de la arquitectura 5G SA

La arquitectura 5G Standalone (SA) representa una evolución significativa en comparación con las generaciones anteriores de redes móviles. A diferencia de las implementaciones Non-Standalone (NSA), que dependen de la infraestructura existente de 4G LTE, la arquitectura SA está diseñada desde cero para aprovechar al máximo las capacidades de la tecnología 5G. Esta arquitectura se basa en una serie de principios clave que permiten una mayor flexibilidad, eficiencia y capacidad para soportar una amplia gama de servicios y aplicaciones. Entre los aspectos más destacados de la arquitectura 5G SA se encuentran:

- **Red basada en servicios:** La arquitectura 5G SA adopta un enfoque basado en servicios, donde las funciones de red se implementan como servicios independientes que pueden ser orquestados y gestionados de manera flexible.
- **Virtualización y desagregación:** La arquitectura permite la virtualización de funciones de red (NFV) y la desagregación de hardware y software, lo que facilita la implementación en entornos de nube y mejora la escalabilidad.
- **Separación del plano de control y plano de usuario:** Esta separación permite una gestión más eficiente del tráfico y una mejor calidad de servicio (QoS) para diferentes tipos de aplicaciones.
- **Soporte para nuevas tecnologías:** La arquitectura 5G SA está diseñada para integrar tecnologías emergentes como la inteligencia artificial (IA), el edge computing y la Internet de las cosas (IoT).
- **Categorías de servicio: eMBB, URLLC y mMTC:** 5G define tres clases de servicio principales —**eMBB** (enhanced Mobile Broadband) para altas tasas de datos y capacidad; **URLLC** (Ultra-Reliable Low-Latency Communications) para comunicaciones con latencia muy baja y alta fiabilidad; y **mMTC** (massive Machine Type Communications) para conectividad masiva de dispositivos IoT de baja potencia y baja tasa. La arquitectura SA y el 5GC están pensados para soportar simultáneamente estas demandas diferenciadas [3GPP TS.22.261].
- **Network Slicing:** Permite crear múltiples redes virtuales independientes (slices) sobre la misma infraestructura física, cada una con sus

propios requisitos de rendimiento, seguridad y gestión (por ejemplo, un slice para eMBB y otro para URLLC). Network Slicing se apoya en SBA, NFV y orquestación para instanciar, aislar y escalar slices según demanda.

2.4.2 Arquitectura basada en servicios (SBA)

La diferencia más destacada en 5G frente a las arquitecturas 3GPP anteriores es la adopción del concepto de interfaces basadas en servicios (**SBA**). Esto implica que las funciones de red que contienen la lógica y los mecanismos para procesar los flujos de señalización ya no se conectan mediante interfaces punto a punto, sino que **exponen sus capacidades como servicios** accesibles para otras funciones de red. En cada intercambio, una función actúa como **consumidora de servicios** y la otra como **proveedora de dichos servicios**. 2.2 muestra la arquitectura 5GC basada en SBA.

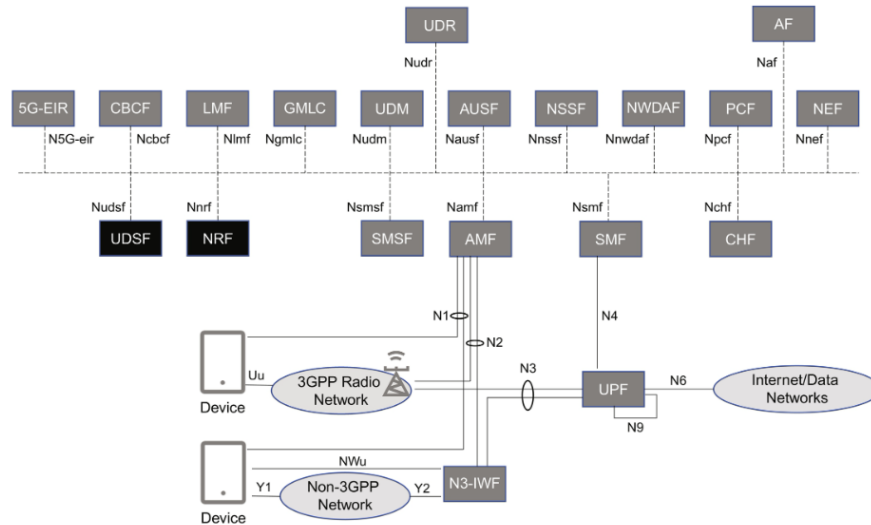


Figure 2.2: Arquitectura 5GC basada en interfaces basadas en servicios [4].

Las funciones de red en 5GC se comunican entre sí a través de una interfaz común llamada **Service-Based Interface (SBI)**. Esta interfaz utiliza protocolos estándar como HTTP/2 y RESTful APIs para facilitar la comunicación entre las funciones de red. La adopción de SBA permite una mayor flexibilidad y escalabilidad en la arquitectura de la red, ya que las funciones pueden ser desarrolladas, desplegadas y actualizadas de manera independiente. Además, esta arquitectura facilita la integración con tecnologías emergentes como la virtualización de funciones de red (NFV) y la computación

en la nube, permitiendo a los operadores de red adaptarse rápidamente a las demandas cambiantes del mercado y ofrecer nuevos servicios de manera más eficiente.

La arquitectura SBA también se puede representar como punto a punto, donde cada función de red se conecta directamente con las demás funciones que requieren sus servicios, utilizando la interfaz SBI para la comunicación. Como se muestra en la figura 2.3.

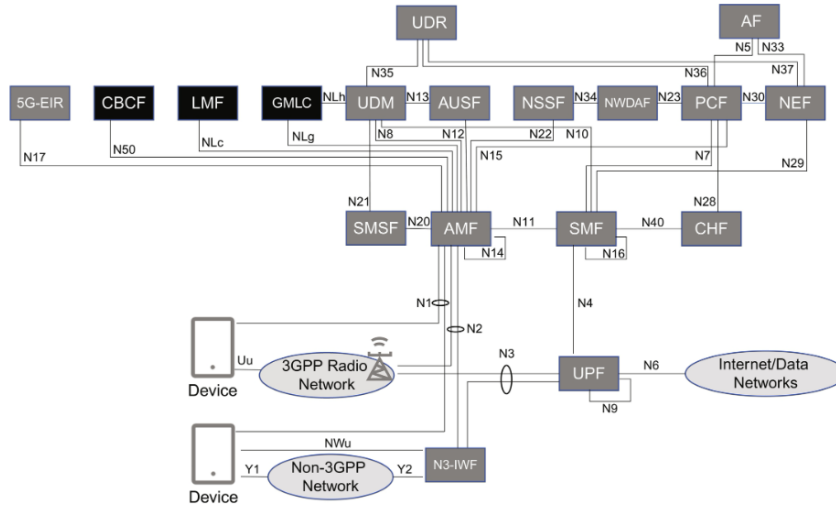


Figure 2.3: Arquitectura 5GC con interfaces punto a punto. [4].

2.4.3 Interfaces HTTP REST

En la arquitectura 5G Core (5GC), las funciones de red se comunican entre sí utilizando una interfaz basada en servicios conocida como Service-Based Interface (SBI). Esta interfaz utiliza el protocolo HTTP/2 junto con RESTful APIs para facilitar la comunicación y el intercambio de información entre las diferentes funciones de red. A continuación, se describen los aspectos clave de las interfaces HTTP REST en 5GC:

- **Protocolo HTTP/2:** 5GC utiliza HTTP/2 como el protocolo de transporte para las comunicaciones entre funciones de red. HTTP/2 ofrece varias ventajas sobre su predecesor, HTTP/1.1, incluyendo una mayor eficiencia en la multiplexación de solicitudes, compresión de encabezados y reducción de la latencia, lo que es crucial para las aplicaciones de baja latencia en 5G.
- **RESTful APIs:** Las funciones de red en 5GC exponen sus capacidades a través de RESTful APIs, que son interfaces basadas en prin-

cipios REST (Representational State Transfer). Estas APIs permiten a las funciones de red interactuar de manera sencilla y estandarizada, utilizando métodos HTTP como GET, POST, PUT y DELETE para realizar operaciones sobre los recursos.

- **Formato de datos JSON:** La comunicación entre funciones de red a través de las RESTful APIs generalmente utiliza JSON (JavaScript Object Notation) como formato de datos para el intercambio de información. JSON es ligero y fácil de leer, lo que facilita la integración entre diferentes sistemas y tecnologías.
- **Seguridad:** La seguridad es un aspecto crítico en las comunicaciones entre funciones de red. En 5GC, se implementan mecanismos de autenticación y autorización para garantizar que solo las funciones autorizadas puedan acceder a los servicios expuestos a través de las RESTful APIs. Además, se utilizan protocolos seguros como TLS (Transport Layer Security) para proteger la integridad y confidencialidad de los datos transmitidos.
- **Escalabilidad y flexibilidad:** La adopción de interfaces HTTP REST permite una mayor escalabilidad y flexibilidad en la arquitectura 5GC. Las funciones de red pueden ser desarrolladas, desplegadas y actualizadas de manera independiente, lo que facilita la adaptación a las demandas cambiantes del mercado y la incorporación de nuevas tecnologías.

2.4.4 Componentes de 5G Core (5GC)

En 5G Core (5GC), los componentes tienen funciones específicas a diferencia de las arquitecturas anteriores, las cuales combinaban algunas funciones como por ejemplo, el MME y el SGW en una sola entidad llamada AMF.

2.4.5 Componentes principales de 5GC

NRF El Network Repository Function es un componente clave en la arquitectura 5G Core (5GC) que actúa como un repositorio centralizado para la gestión y descubrimiento de servicios de red. Su función principal es mantener un registro actualizado de todas las funciones de red disponibles en la red 5G, permitiendo que otras funciones de red puedan descubrir y comunicarse con ellas de manera eficiente. El NRF mantiene un catálogo dinámico de instancias de funciones de red (NF) con sus perfiles, incluyendo identidad (NF Instance ID), tipo de NF, direcciones de servicio (Service URIs) y

versiones de API soportadas, con soporte para mecanismos de heartbeat y actualización periódica del estado operacional [1].

AMF El Access and Mobility Management Function es una de las funciones clave en la arquitectura 5G Core (5GC) y se encarga de gestionar el acceso y la movilidad de los dispositivos de usuario (UE) en la red 5G. El AMF desempeña un papel fundamental en la gestión de la conexión del UE, la autenticación, la autorización y el control de movilidad, asegurando que los dispositivos puedan acceder a los servicios de red de manera eficiente y segura [1].

SMF El Session Management Function se encarga de gestionar las sesiones de datos del usuario (UE) en la red 5G. El SMF desempeña un papel fundamental en la configuración, mantenimiento y liberación de las sesiones de datos, asegurando que los dispositivos puedan acceder a los servicios de red de manera eficiente y segura [1].

UPF El User Plane Function es responsable de la gestión del plano de usuario (User Plane) en la arquitectura 5GC, actuando como el punto de anclaje para el procesamiento y enrutamiento de tráfico de datos. El UPF desempeña un papel crítico en la cadena de procesamiento de paquetes, implementando funciones de forwarding, encapsulación y aplicación de políticas a nivel de flujo [1].

AUSF El Authentication Server Function Se encarga de gestionar la autenticación de los dispositivos de usuario (UE) en la red 5G. El AUSF desempeña un papel fundamental en la seguridad de la red, asegurando que solo los dispositivos autorizados puedan acceder a los servicios de red [1].

UDM El Unified Data Management Es responsable de gestionar los datos de suscripción y la información del usuario en la red 5G. El UDM desempeña un papel fundamental en la gestión de la identidad del usuario, las políticas de acceso y las configuraciones de servicio, asegurando que los dispositivos puedan acceder a los servicios de red de manera eficiente y segura [1].

PCF El Policy Control Function Gestiona las políticas de control y calidad de servicio (QoS) en la red 5G. El PCF desempeña un papel fundamental en la aplicación de políticas de acceso, control de tráfico y gestión de recursos, asegurando que los dispositivos puedan acceder a los servicios de red de manera eficiente y segura.

NSSF El Network Slice Selection Function Se encarga de gestionar la selección y asignación de redes de corte (slices) para los dispositivos de usuario (UE) en la red 5G. El NSSF desempeña un papel fundamental en la implementación de redes de corte, que permiten a los operadores ofrecer servicios personalizados y optimizados para diferentes tipos de aplicaciones y usuarios.

NEF El Network Exposure Function Expone las capacidades y servicios de la red 5G a aplicaciones externas y terceros. El NEF desempeña un

papel fundamental en la facilitación de la interoperabilidad entre la red 5G y aplicaciones de terceros, permitiendo a los desarrolladores crear aplicaciones innovadoras que aprovechen las capacidades avanzadas de la red 5G.

AF El Application Function Gestiona las aplicaciones que interactúan con la red 5G. El AF desempeña un papel fundamental en la facilitación de la comunicación entre las aplicaciones y las funciones de red, permitiendo a los desarrolladores crear aplicaciones innovadoras que aprovechen las capacidades avanzadas de la red 5G.

2.4.6 Pila de protocolos del plano de control y plano de usuario

Uno de los aspectos diferenciadores de la arquitectura 5G Core (5GC) es la separación clara entre el plano de control (Control Plane - CP) y el plano de usuario (User Plane - UP) mediante CUPS (Control and User Plane Separation). Esta separación permite una gestión más eficiente del tráfico y una mejor calidad de servicio (QoS) para diferentes tipos de aplicaciones. A continuación, se describen las pilas de protocolos utilizadas en ambos planos.

2.4.6.1 Plano de Control

En el plano de control (Control Plane - CP) se gestionan las señales y la lógica necesarias para establecer, mantener y liberar las conexiones entre los dispositivos de usuario (UE) y la red. incluyendo protocolos como SCTP, NGAP, NAS, HTTP/2 y RESTful APIs. A continuación se muestra la pila de protocolos del plano de control de forma detallada 2.4.

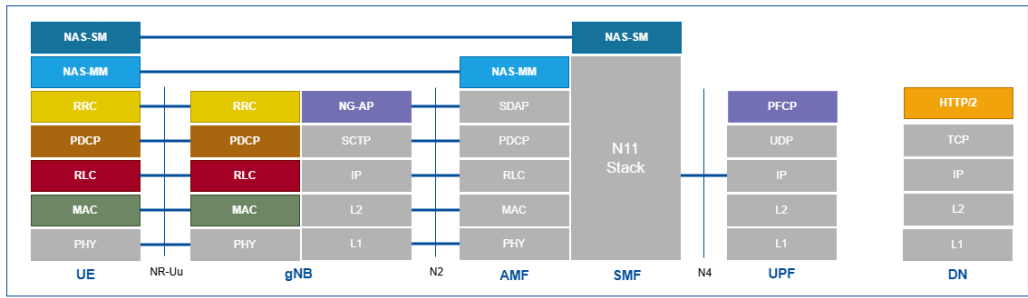


Figure 2.4: Pila de protocolos del plano de control en 5GC.

Para mas detalles de los acrónimos y protocolos ver la tabla ?? y ?? en el apéndice ??.

2.4.6.2 Plano de Usuario

En este plano se maneja el tráfico de datos real que fluye entre los dispositivos de usuario (UE) y la red. El plano de usuario se encarga de la transmisión eficiente y segura de los datos, asegurando que se cumplan los requisitos de calidad de servicio (QoS) y latencia para diferentes tipos de aplicaciones. A continuación se muestra la pila de protocolos del plano de usuario de forma detallada 2.5.

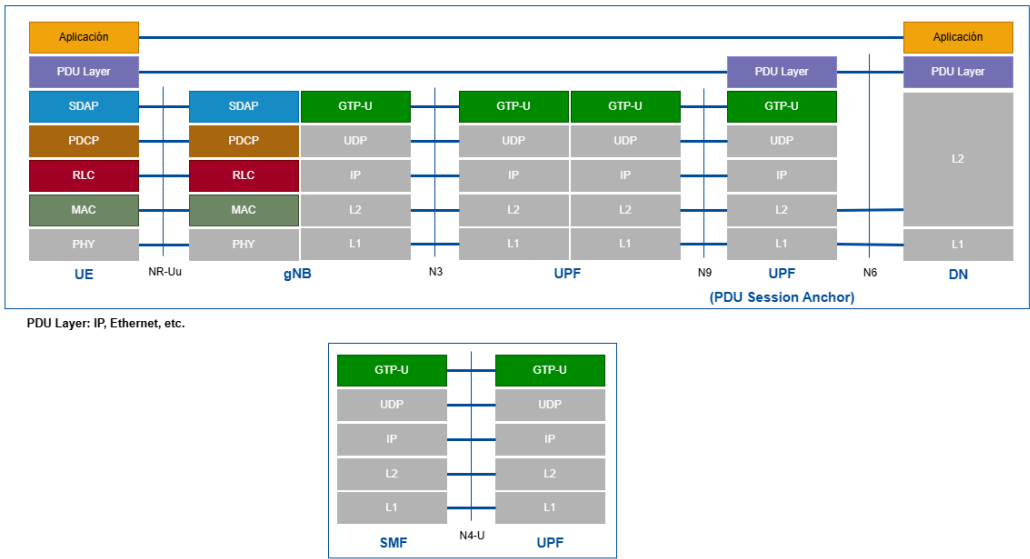


Figure 2.5: Pila de protocolos del plano de usuario en 5GC.

Al igual que en el plano de control, para mas detalles de los acrónimos y protocolos ver la tabla ?? y ?? en el apéndice ??.

2.4.7 Interfaces clave en 5GC

Las interfaces N1, N2, N3, N4, N6 y N9 son fundamentales en la arquitectura de las redes 5G, cada una desempeñando un papel crucial en la interconexión de dispositivos y la gestión de datos. A continuación, se describen cada una de estas interfaces, los protocolos asociados y los dispositivos que conectan.

- **N1:** Esta interfaz conecta el dispositivo de usuario (UE) con el Access and Mobility Management Function (AMF) en el plano de control. Utiliza el protocolo NAS (Non-Access Stratum) para la señalización y gestión de la conexión del UE.

- **N2:** Conecta la estación base 5G (gNodeB) con el AMF en el plano de control. Utiliza el protocolo NGAP (Next Generation Application Protocol) para la señalización y gestión de la conexión entre el gNodeB y el AMF.
- **N3:** Esta interfaz conecta la estación base 5G (gNodeB) con el User Plane Function (UPF) en el plano de usuario. Utiliza el protocolo GTP-U (GPRS Tunneling Protocol - User Plane) para la transmisión de datos entre el gNodeB y el UPF.
- **N4:** Conecta el SMF (Session Management Function) con el UPF en el plano de control. Utiliza el protocolo PFCP (Packet Forwarding Control Protocol) para la gestión y configuración del plano de usuario en el UPF.
- **N6:** Esta interfaz conecta el UPF con la red externa, como Internet o una red privada. Utiliza protocolos estándar como IP para la transmisión de datos entre el UPF y la red externa.
- **N9:** Conecta diferentes instancias de UPF entre sí en el plano de usuario. Utiliza el protocolo GTP-U para la transmisión de datos entre las instancias de UPF, permitiendo la movilidad y continuidad del servicio.
- **N11:** Conecta el AMF con el SMF en el plano de control. Utiliza interfaces basadas en servicios (SBI) y protocolos HTTP/2 y RESTful APIs para la comunicación entre el AMF y el SMF.

2.4.8 Tendencias emergentes en redes beyond 5G

Las tendencias emergentes en redes beyond 5G incluyen:

- **Comunicaciones Terahertz (THz):** Utilización de frecuencias en el rango de terahercios para ofrecer velocidades de datos ultra altas y baja latencia.
- **Redes auto-organizadas (Self-Organizing Networks - SON):** Implementación de algoritmos de inteligencia artificial para la gestión automática y optimización de la red.
- **Computación en el borde (Edge Computing):** Despliegue de capacidades de procesamiento cerca del usuario final para reducir la latencia y mejorar la eficiencia.

- **Integración de inteligencia artificial (IA):** Uso de IA para la gestión de red, optimización del rendimiento y personalización de servicios.
- **Redes holográficas y realidad extendida (XR):** Soporte para aplicaciones avanzadas que requieren alta capacidad y baja latencia, como hologramas y experiencias inmersivas.
- **Sostenibilidad y eficiencia energética:** Desarrollo de tecnologías y prácticas que reduzcan el consumo energético y el impacto ambiental de las redes móviles.
- **Seguridad y privacidad mejoradas:** Implementación de mecanismos avanzados para proteger los datos y la privacidad de los usuarios en un entorno de red cada vez más complejo.
- **Redes cuánticas:** Investigación en la integración de tecnologías de comunicación cuántica para mejorar la seguridad y la capacidad de las redes móviles.
- **Conectividad masiva y ubicua:** Desarrollo de infraestructuras que permitan una conectividad continua y confiable en cualquier lugar y momento, soportando una amplia gama de dispositivos y aplicaciones.

Estas tendencias reflejan el compromiso de la industria de las telecomunicaciones para avanzar hacia redes móviles más inteligentes, eficientes y capaces de satisfacer las necesidades futuras de los usuarios y las aplicaciones.

Chapter 3

Telemetría y QoS en 5G

En 5GC, la telemetría juega un papel crucial en la monitorización y gestión del rendimiento de la red, especialmente en el contexto de las demandas crecientes de servicios como URLLC (Ultra-Reliable Low-Latency Communications), eMBB (enhanced Mobile Broadband) y mMTC (massive Machine Type Communications). La telemetría en banda (In-Band Network Telemetry, INT) permite la recopilación de datos en tiempo real directamente desde los paquetes que atraviesan la red, proporcionando una visibilidad detallada del estado de la red y facilitando la detección y resolución de problemas.

3.1 Desafíos de Telemetría en Redes 5G

3.1.1 Limitaciones de mecanismos tradicionales (SNMP, NetFlow, sFlow)

3.1.2 Requisitos de visibilidad en URLLC, eMBB y mMTC

3.2 In-Band Network Telemetry (INT)

3.2.1 Concepto y motivación

3.2.2 Arquitectura INT: source, transit y sink

3.2.3 Metadatos INT-MD y su estructura

3.2.4 INT vs telemetría out-of-band

3.3 Calidad de Servicio en 5G

3.4 QoS basado en 5G Flujo

3.4.1 Definición de flujo en 5G

3.4.2 Identificación y clasificación de flujos

3.4.3 Mecanismos de gestión de QoS por flujo

3.5 Señalización de QoS en 5G

3.6 Características de QoS en 5G

Chapter 4

Fundamentos de P4 y Programación del Plano de Datos

4.1 Introducción a P4

4.1.1 Historia y evolución

4.1.2 Modelo de arquitectura PISA

4.1.3 P4_14 vs P4_16

4.2 Herramientas y ecosistema P4

4.2.1 BMv2 como switch software

4.2.2 P4C: compilador

4.2.3 P4Runtime: control del data plane

4.3 Implementación de INT en P4

4.3.1 Definición de cabeceras INT

4.3.2 Parsing y deparsing en BMv2

4.3.3 Inyección hop-by-hop de metadatos

4.3.4 Limitaciones del pipeline P4

Chapter 5

Entorno de Desarrollo Implementado

5.1 Arquitectura general del sistema

A diferencia de las tecnologías pasadas, 5G esta diseñada para ser facilmente desplegada en entornos virtualizados y basados en contenedores, lo que permite una mayor flexibilidad y escalabilidad. En este proyecto, se ha implementado un entorno de desarrollo que emula una red 5G Standalone (SA), virtualizando sus componentes principales mediante Docker y orquestando el Core con Docker Compose. El entorno de desarrollo se ha desplegado utilizando GNS3, GNS3 VM y VMware Workstation para crear una infraestructura base o underlay sobre la cual se han implementado otras máquinas virtuales que alojan los contenedores Docker. Esta infraestructura virtual proporciona la conectividad necesaria entre los componentes y permite emular una red de un MNO (Mobile Network Operator) real, con componentes tanto de red y VFs (Virtualized Functions) como de 5G. Tambien cabe destacar que para los componentes de 5G se empleó el proyecto open source free5gc [3], el cual permite desplegar un core 5G SA completo en un entorno virtualizado con Docker. La siguiente figura muestra la arquitectura general del entorno de desarrollo implementado, destacando la integración entre GNS3, Docker y la máquina virtual en vmware 5.1.

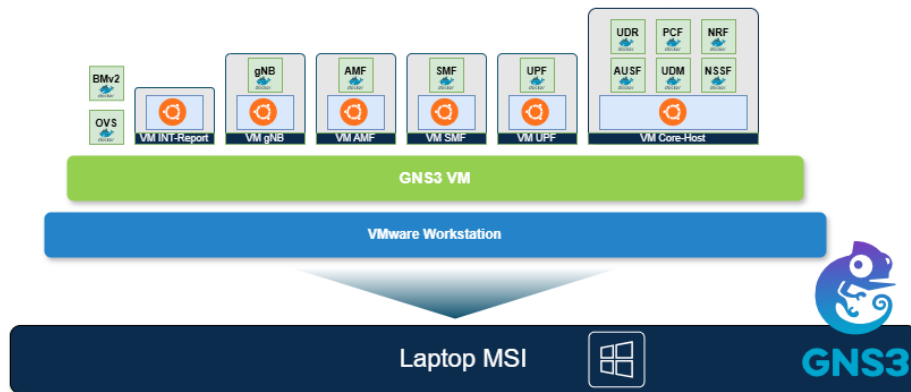


Figure 5.1: Arquitectura general

5.1.1 Diseño de la red 5G SA

Con respecto a la red de transporte, se ha implementado utilizando switches programables P4 para insertar metadatos de telemetría en banda (INT-MD) en el tráfico N2 (SCTP) y N3 (GTP-U), estos switches están contruidos con P4 para el reenvío de tráfico a nivel de L3. además de estos switches P4, tanto para el tráfico entre los Nodos en el Core como en la comunicación entre el NGRAN y R1, se ha empleado switches virtuales Open vSwitch (OVS) para gestionar los paquetes ARP. La siguiente figura es un diseño de alto nivel (HLD) que muestra la topología de red implementada en GNS3 5.2.

5.1.1.1 Diseño de alto nivel (HLD)

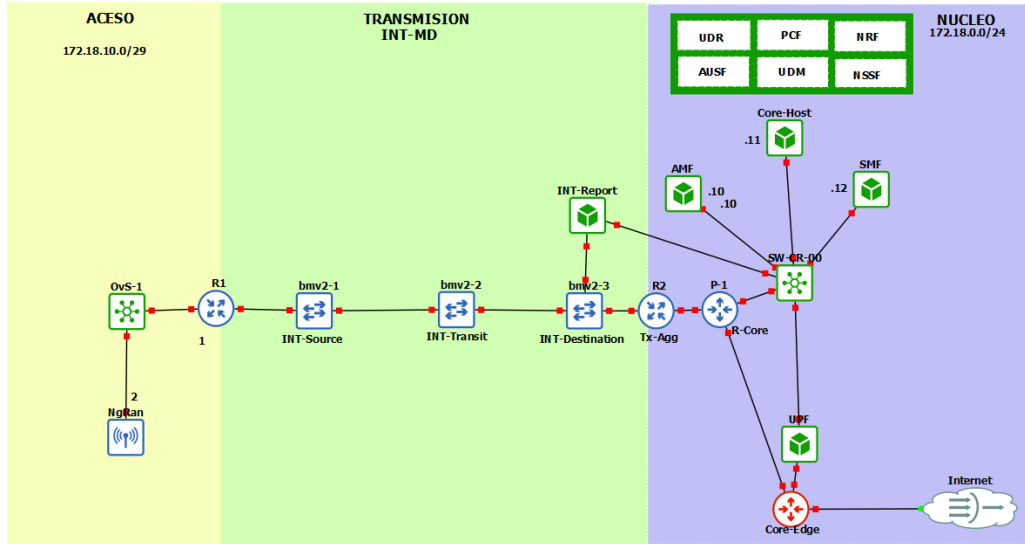


Figure 5.2: Topología de red implementada en GNS3

5.1.1.2 Diseño de bajo nivel (LLD)

La anterior tabla 5.1 detalla la configuración de red de cada contenedor Docker, direcciones IP y MAC asignadas, el tipo de red utilizado (macvlan), etc. Con esta configuración, se asegura una correcta interconexión y funcionamiento del Core 5G SA dentro del entorno virtualizado.

NF	Direccion IP	Mascara	Direccion MAC	Tipo de Red	Alias	Puertos	BD	Deps.
db	172.18.0.50	/24	22:e5:63:00:49:85	macvlan	db	27017	-	-
NRF	172.18.0.51	/24	3a:e5:07:18:07:fb	macvlan	nrf.free5gc.org	8000	db	db
AUSF	172.18.0.52	/24	aa:61:e3:b4:07:2e	macvlan	ausf.free5gc.org	8000	-	nrf
NSSF	172.18.0.53	/24	3e:74:95:c5:bc:4e	macvlan	nssf.free5gc.org	8000	-	nrf
PCF	172.18.0.54	/24	72:b2:53:59:fc:ce	macvlan	pcf.free5gc.org	8000	-	nrf
UDM	172.18.0.55	/24	9a:a1:ed:b3:60:8c	macvlan	udm.free5gc.org	8000	-	db, nrf
UDR	172.18.0.56	/24	ea:b1:9d:90:a4:d1	macvlan	udr.free5gc.org	8000	db	db, nrf
CHF	172.18.0.57	/24	ee:e6:15:22:6f:ad	macvlan	chf.free5gc.org	8000, 2122	db	db, nrf, webui
NEF	172.18.0.58	/24	e2:66:f3:0f:e0:e2	macvlan	nef.free5gc.org	8000	db	db, nrf
WebUI	172.18.0.59	/24	2a:5c:48:5c:aa:a0	macvlan	webui	2121 (ext: 5000)	-	db, nrf
AMF	172.18.0.20	/24	f2:ff:cb:34:07:b5	macvlan	amf.free5gc.org	38412 (SCTP), 8000	-	-
SMF	172.18.0.22	/24	ca:76:58:9c:8e:ec	macvlan	smf.free5gc.org	8000	-	-
UPF	172.18.0.24	/24	be:44:98:a4:e5:c4	macvlan	upf.free5gc.org	8000, 38412 (SCTP)	-	-
ueransim	172.18.10.3	/29	f6:b8:eb:4d:0f:3c	macvlan	ueransim.free5gc.org	38412 (SCTP)	-	-

Table 5.1: Diseño de bajo nivel (LLD)

5.2 Despliegue del Core 5G SA

Como se pudo apreciar en la figura 5.1, el Core 5G SA se ha desplegado utilizando contenedores Docker orquestados con Docker Compose dentro de maquinas virtuales alojadas en una maquina virtual principal (GNS3 VM). los componentes que interactuan mediante HTTP/2 se han desplegado en una VM con Ubuntu 20.04, mientras que el resto de componentes; AMF, SMF y UPF se han desplegado en maquinas virtuales separadas con la misma version de Ubuntu. Juntar los componentes que interactuan mediante HTTP/2 en una sola VM permite reducir la latencia y mejorar el rendimiento de las comunicaciones entre ellos.

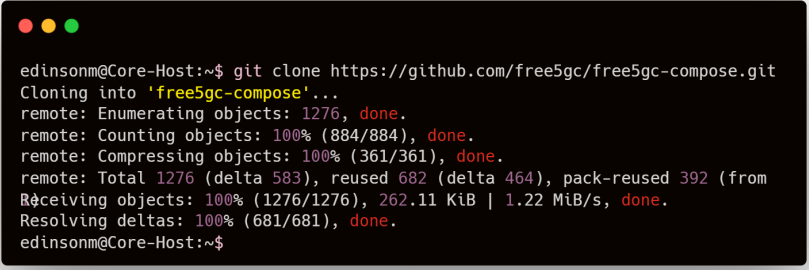
La configuracion de red de cada contenedor Docker se realizó con la modalidad de red **macvlan**, detallada en la tabla 5.1, que permite asignar una direccion IP en el mismo ranfo que la red fisica de la VM anfitriona. Esto facilita la comunicacion entre los contenedores y hace posible el establecimiento del enlace **N2** (SCTP) entre el NGRAN y el AMF.

5.2.1 Configuración de Core-Host

Los detalles sobre la instalacion de Docker y Docker Compose no estan incluidos en este documento, pero se asume que el lector tiene conocimientos basicos sobre estas tecnologias. En caso contrario, puede consultar la documentacion oficial [2].

5.2.1.1 Obtención del código fuente 5GC

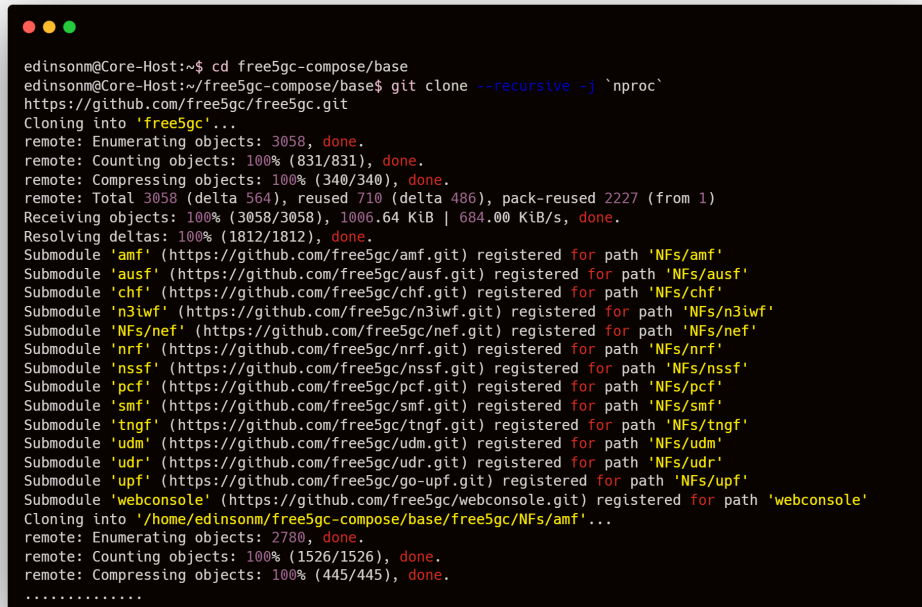
El primer paso para desplegar el Core 5G SA es clonar el repositorio oficial de free5gc desde GitHub como se muestra en la figura 5.3.



```
edinsonm@Core-Host:~$ git clone https://github.com/free5gc/free5gc-compose.git
Cloning into 'free5gc-compose'...
remote: Enumerating objects: 1276, done.
remote: Counting objects: 100% (884/884), done.
remote: Compressing objects: 100% (361/361), done.
remote: Total 1276 (delta 583), reused 682 (delta 464), pack-reused 392 (from
Receiving objects: 100% (1276/1276), 262.11 KiB | 1.22 MiB/s, done.
Resolving deltas: 100% (681/681), done.
edinsonm@Core-Host:~$
```

Figure 5.3: Clonación del repositorio de free5gc

Luego de clonar el repositorio, se procede a clonar las bases de las Network Functions (NFs) adicionales que no vienen incluidas en el repositorio principal de free5gc, como se muestra en la figura 5.4.



```

edinsonm@Core-Host:~$ cd free5gc-compose/base
edinsonm@Core-Host:~/free5gc-compose/base$ git clone --recursive -j `nproc`
https://github.com/free5gc/free5gc.git
Cloning into 'free5gc'...
remote: Enumerating objects: 3058, done.
remote: Counting objects: 100% (831/831), done.
remote: Compressing objects: 100% (340/340), done.
remote: Total 3058 (delta 564), reused 710 (delta 486), pack-reused 2227 (from 1)
Receiving objects: 100% (3058/3058), 1006.64 KiB | 684.00 KiB/s, done.
Resolving deltas: 100% (1812/1812), done.
Submodule 'amf' (https://github.com/free5gc/amf.git) registered for path 'NFs/amf'
Submodule 'ausf' (https://github.com/free5gc/ausf.git) registered for path 'NFs/ausf'
Submodule 'chf' (https://github.com/free5gc/chf.git) registered for path 'NFs/chf'
Submodule 'n3iwf' (https://github.com/free5gc/n3iwf.git) registered for path 'NFs/n3iwf'
Submodule 'nef' (https://github.com/free5gc/nef.git) registered for path 'NFs/nef'
Submodule 'nrf' (https://github.com/free5gc/nrf.git) registered for path 'NFs/nrf'
Submodule 'nssf' (https://github.com/free5gc/nssf.git) registered for path 'NFs/nssf'
Submodule 'pcf' (https://github.com/free5gc/pcf.git) registered for path 'NFs/pcf'
Submodule 'smf' (https://github.com/free5gc/smf.git) registered for path 'NFs/smf'
Submodule 'tngf' (https://github.com/free5gc/tngf.git) registered for path 'NFs/tngf'
Submodule 'udm' (https://github.com/free5gc/udm.git) registered for path 'NFs/udm'
Submodule 'udr' (https://github.com/free5gc/udr.git) registered for path 'NFs/udr'
Submodule 'upf' (https://github.com/free5gc/go-upf.git) registered for path 'NFs/upf'
Submodule 'webconsole' (https://github.com/free5gc/webconsole.git) registered for path 'webconsole'
Cloning into '/home/edinsonm/free5gc-compose/base/free5gc/NFs/amf'...
remote: Enumerating objects: 2780, done.
remote: Counting objects: 100% (1526/1526), done.
remote: Compressing objects: 100% (445/445), done.
.....

```

Figure 5.4: Clonación de base de NFs adicionales

Una vez clonado el repositorio principal y las bases de las NFs adicionales, se procede a compilar las NFs que en este caso, como es en el Core-Host, los compilamos todos como se puede ver en la figura 5.5.

```

edinsonm@Core-Host:~/free5gc-compose$ sudo make all
[sudo] password for edinsonm:
docker build -t 'free5gc/' 'base': 'latest' ./base
[+] Building 488.5s (7/7) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                2.0s
=> => transferring dockerfile: 479B                               0.4s
=> [internal] load metadata for docker.io/library/golang:1.21.8-bullseye 13.4s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 2B                                       0.0s
=> [1/3] FROM docker.io/library/golang:1.21.8-bullseye@sha256:81d98548f08e22a59c5c0be814acc099f 331.2s
=> => resolve docker.io/library/golang:1.21.8-bullseye@sha256:81d98548f08e22a59c5c0be814acc099f 0.2s
=> => sha256:81d98548f08e22a59c5c0be814acc099f3df3313487adcf4e1d3d962af3a43 1.64kB / 1.64kB 0.0s
=> => sha256:2b64e533430ee61e12544f2155ff8b93557bdeee898e068b91045f279c9fcf0c 1.79kB / 1.79kB 0.0s
=> => sha256:2fdff29b343a8e184d06dfaf378ff3393bf97d2b59e07f40ac0e73d3bed33de0 2.88kB / 2.88kB 0.0s
=> => sha256:ec335f17d0c74f7a270925cb1bbd29acc72ae904c6f4570f9ae369e3eebb64e 55.08MB / 55.08MB 192.9s
=> => sha256:d2b4675e1918dcb7f5c9bfedbb5a8634d2459306d1f3b91f08c7293380f10585 15.76MB / 15.76MB 39.9s
=> => sha256:7f67b1746a83d257a6398cf8eeca47bfa1f854670097ea4234f12857cfc7d593 54.59MB / 54.59MB 143.8s
=> => sha256:fbd73e3517bc8fa1e9863e448b6ad3b54c5482345b15ce6ad28439cf1f01752 86.11MB / 86.11MB 259.4s
=> => sha256:0635831cef590cc18d45a50a86211bcdbb7e2dc2aeac5a3d19671604e6b7e3d 67.01MB / 67.01MB 275.3s
=> => sha256:579c95fc9df09d4b635a801a036ce19e0793e293b61b4621bac859996c4bbd47 173B / 173B 197.8s
=> => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb577484a6d75e68dc38e8acc1 32B / 32B 199.8s
=> => extracting sha256:ec335f17d0c74f7a270925cb1bbd29acc72ae904c6f4570f9ae369e3eebb64e 25.2s
=> => extracting sha256:d2b4675e1918dcb7f5c9bfedbb5a8634d2459306d1f3b91f08c7293380f10585 3.7s
=> => extracting sha256:7f67b1746a83d257a6398cf8eeca47bfa1f854670097ea4234f12857cfc7d5932 28.4s
=> => extracting sha256:fbd73e3517bc8fa1e9863e448b6ad3b54c5482345b15ce6ad28439cf1f017523 23.0s
=> => extracting sha256:0635831cef590cc18d45a50a86211bcdbb7e2dc2aeac5a3d19671604e6b7e3d9 38.8s
=> => extracting sha256:579c95fc9df09d4b635a801a036ce19e0793e293b61b4621bac859996c4bbd47 0.0s
=> => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb577484a6d75e68dc38e8acc1 0.0s
=> [2/3] RUN apt-get update && apt-get -y install gcc cmake autoconf libtool pkg-config lib 94.4s
=> [3/3] RUN apt-get clean 6.3s
=> => exporting to image 33.6s
=> => exporting layers 32.5s
=> => writing image sha256:8e04e0e085a5583cc8ac3d2d3556b962ed01d02c2abe69d33a6579dala75fea4 0.3s
=> => naming to docker.io/free5gc/base:latest 0.4s
.....

```

Figure 5.5: Compilación de NFs adicionales

5.2.1.2 Configuración de Docker Compose

Luego de haberlos compilado, se procede a configurar cada NF de acuerdo con el diseño de bajo nivel (LLD) mostrado en la tabla 5.1. Como la VM Core-Host corre los contenedores UDR, UDM, PCF, NRF, AUSF, UDM y NSSF, DB, se despliegan creando un unico archivo Docker Compose llamado *docker-compose-build-core-host.yaml*, el cual se mostrará contenedor por contenedor a continuación:

5.2.1.2.1 DB

```

1 services:
2   db:
3     container_name: mongoddb
4     image: mongo:3.6.8
5     command: mongod --port 27017
6     expose:

```

```
7     - "27017"
8   volumes:
9     - dbdata:/data/db
10  networks:
11    macvlan_net:
12      ipv4_address: 172.18.0.50
13      mac_address: "22:e5:63:00:49:85"
14      aliases:
15        - db
```

Código 5.1: Configuración del MongoDB en Docker Compose

5.2.1.2.2 NRF

```
1  free5gc-nrf:
2    container_name: nrf
3    build:
4      context: ./nf_nrf
5      args:
6        DEBUG_TOOLS: "false"
7    command: ./nrf -c ./config/nrfcfg.yaml
8    expose:
9      - "8000"
10   volumes:
11     - ./config/nrfcfg.yaml:/free5gc/config/nrfcfg.yaml
12     - ./cert:/free5gc/cert
13   environment:
14     DB_URI: mongodb://db/free5gc
15     GIN_MODE: release
16   networks:
17     macvlan_net:
18       ipv4_address: 172.18.0.51
19       mac_address: "3a:e5:07:18:07:fb"
20       aliases:
21         - nrf.free5gc.org
22   extra_hosts:
23     - "amf.free5gc.org:172.18.0.20"
24     - "smf.free5gc.org:172.18.0.22"
25     - "upf.free5gc.org:172.18.0.23"
26   ports:
27     - "8000"
28
29   depends_on:
30     - db
```

Código 5.2: Configuración del NRF en Docker Compose

5.2.1.2.3 AUSF

```
1  free5gc-ausf:
2    container_name: ausf
3    build:
4      context: ./nf_ausf
5      args:
6        DEBUG_TOOLS: "false"
7    command: ./ausf -c ./config/ausfcfg.yaml
8    expose:
9      - "8000"
10   volumes:
11     - ./config/ausfcfg.yaml:/free5gc/config/ausfcfg.yaml
12     - ./cert:/free5gc/cert
13   environment:
14     GIN_MODE: release
15   networks:
16     macvlan_net:
17       ipv4_address: 172.18.0.52
18       mac_address: "aa:61:e3:b4:07:2e"
19       aliases:
20         - ausf.free5gc.org
21   extra_hosts:
22     - "amf.free5gc.org:172.18.0.20"
23     - "smf.free5gc.org:172.18.0.22"
24     - "upf.free5gc.org:172.18.0.23"
25
26   depends_on:
27     - free5gc-nrf
```

Código 5.3: Configuración del AUSF en Docker Compose

5.2.1.2.4 NSSF

```
1  free5gc-nssf:
2    container_name: nssf
3    build:
4      context: ./nf_nssf
5      args:
6        DEBUG_TOOLS: "false"
7    command: ./nssf -c ./config/nssfcfg.yaml
8    expose:
9      - "8000"
10   volumes:
11     - ./config/nssfcfg.yaml:/free5gc/config/nssfcfg.yaml
12     - ./cert:/free5gc/cert
13   environment:
14     GIN_MODE: release
```



```
15 networks:
16     macvlan_net:
17         ipv4_address: 172.18.0.53
18         mac_address: "3e:74:95:c5:bc:4e"
19         aliases:
20             - nssf.free5gc.org
21 extra_hosts:
22     - "amf.free5gc.org:172.18.0.20"
23     - "smf.free5gc.org:172.18.0.22"
24     - "upf.free5gc.org:172.18.0.23"
25 depends_on:
26     - free5gc-nrf
```

Código 5.4: Configuración del NSSF en Docker Compose

5.2.1.2.5 PCF

```
1 free5gc-pcf:
2     container_name: pcf
3     build:
4         context: ./nf_pcf
5         args:
6             DEBUG_TOOLS: "false"
7     command: ./pcf -c ./config/pcfcfg.yaml
8     expose:
9         - "8000"
10    volumes:
11        - ./config/pcfcfg.yaml:/free5gc/config/pcfcfg.yaml
12        - ./cert:/free5gc/cert
13    environment:
14        GIN_MODE: release
15    networks:
16        macvlan_net:
17            ipv4_address: 172.18.0.54
18            mac_address: "72:b2:53:59:fc:ce"
19            aliases:
20                - pcf.free5gc.org
21    extra_hosts:
22        - "amf.free5gc.org:172.18.0.20"
23        - "smf.free5gc.org:172.18.0.22"
24        - "upf.free5gc.org:172.18.0.23"
25    depends_on:
26        - free5gc-nrf
```

Código 5.5: Configuración del PCF en Docker Compose

5.2.1.2.6 UDM

```
1  free5gc-udm:
2    container_name: udm
3    build:
4      context: ./nf_udm
5      args:
6        DEBUG_TOOLS: "false"
7    command: ./udm -c ./config/udmcfg.yaml
8    expose:
9      - "8000"
10   volumes:
11     - ./config/udmcfg.yaml:/free5gc/config/udmcfg.yaml
12     - ./cert:/free5gc/cert
13   environment:
14     GIN_MODE: release
15   networks:
16     macvlan_net:
17       ipv4_address: 172.18.0.55
18       mac_address: "9a:a1:ed:b3:60:8c"
19       aliases:
20         - udm.free5gc.org
21   extra_hosts:
22     - "amf.free5gc.org:172.18.0.20"
23     - "smf.free5gc.org:172.18.0.22"
24     - "upf.free5gc.org:172.18.0.23"
25   depends_on:
26     - db
27     - free5gc-nrf
```

Código 5.6: Configuración del UDM en Docker Compose

5.2.1.2.7 UDR

```
1  free5gc-udr:
2    container_name: udr
3    build:
4      context: ./nf_udr
5      args:
6        DEBUG_TOOLS: "false"
7    command: ./udr -c ./config/udrcfg.yaml
8    expose:
9      - "8000"
10   volumes:
11     - ./config/udrcfg.yaml:/free5gc/config/udrcfg.yaml
12     - ./cert:/free5gc/cert
13   environment:
14     DB_URI: mongodb://db/free5gc
15     GIN_MODE: release
```

```
16 networks:
17     macvlan_net:
18         ipv4_address: 172.18.0.56
19         mac_address: "ea:b1:9d:90:a4:d1"
20         aliases:
21             - udr.free5gc.org
22     extra_hosts:
23         - "amf.free5gc.org:172.18.0.20"
24         - "smf.free5gc.org:172.18.0.22"
25         - "upf.free5gc.org:172.18.0.23"
26     depends_on:
27         - db
28         - free5gc-nrf
```

Código 5.7: Configuración del UDR en Docker Compose

5.2.1.2.8 Despliegue del CHF

```
1 free5gc-chf:
2     container_name: chf
3     build:
4         context: ./nf_chf
5         args:
6             DEBUG_TOOLS: "false"
7     command: ./chf -c ./config/chfcfg.yaml
8     expose:
9         - "8000"
10        - "2122"
11     volumes:
12         - ./config/chfcfg.yaml:/free5gc/config/chfcfg.yaml
13         - ./cert:/free5gc/cert
14     environment:
15         DB_URI: mongodb://db/free5gc
16         GIN_MODE: release
17     networks:
18         macvlan_net:
19             ipv4_address: 172.18.0.57
20             mac_address: "ee:e6:15:22:6f:ad"
21             aliases:
22                 - chf.free5gc.org
23     extra_hosts:
24         - "amf.free5gc.org:172.18.0.20"
25         - "smf.free5gc.org:172.18.0.22"
26         - "upf.free5gc.org:172.18.0.23"
27     depends_on:
28         - db
29         - free5gc-nrf
30         - free5gc-webui
```

Código 5.8: Configuración del CHF en Docker Compose

5.2.1.2.9 Despliegue del NEF

```
1 free5gc-nef:
2   container_name: nef
3   build:
4     context: ./nf_nef
5     args:
6       DEBUG_TOOLS: "false"
7   command: ./nef -c ./config/nefcfg.yaml
8   expose:
9     - "8000"
10  volumes:
11    - ./config/nefcfg.yaml:/free5gc/config/nefcfg.yaml
12    - ./cert:/free5gc/cert
13  environment:
14    GIN_MODE: release
15  networks:
16    macvlan_net:
17      ipv4_address: 172.18.0.58
18      mac_address: "e2:66:f3:0f:e0:e2"
19      aliases:
20        - nef.free5gc.org
21  extra_hosts:
22    - "amf.free5gc.org:172.18.0.20"
23    - "smf.free5gc.org:172.18.0.22"
24    - "upf.free5gc.org:172.18.0.23"
25  depends_on:
26    - db
27    - free5gc-nrf
```

Código 5.9: Configuración del NEF en Docker Compose

5.2.1.2.10 Despliegue del WebUI

A pesar de que el contenedor WebUI no es un Network Function (NF) del Core 5G SA, se ha incluido en el mismo archivo Docker Compose para facilitar su despliegue y gestión. El WebUI proporciona una interfaz gráfica para monitorizar y gestionar el Core 5G, facilitando la administración de la red.

```
1 free5gc-webui:
2   container_name: webui
3   build:
4     context: ./webui
```

```

5     args:
6         DEBUG_TOOLS: "false"
7     command: ./webui -c ./config/webuicfg.yaml
8     expose:
9         - "2121"
10    volumes:
11        - ./config/webuicfg.yaml:/free5gc/config/webuicfg.yaml
12    environment:
13        - GIN_MODE=release
14    ports:
15        - "5000:5000"
16    networks:
17        macvlan_net:
18            ipv4_address: 172.18.0.59
19            mac_address: "2a:5c:48:5c:aa:a0"
20            aliases:
21                - webui
22    extra_hosts:
23        - "amf.free5gc.org:172.18.0.20"
24        - "smf.free5gc.org:172.18.0.22"
25        - "upf.free5gc.org:172.18.0.23"
26    depends_on:
27        - db
28        - free5gc-nrf

```

Código 5.10: Configuración del WebUI en Docker Compose

5.2.1.2.11 Configuración de Red y Volúmenes

```

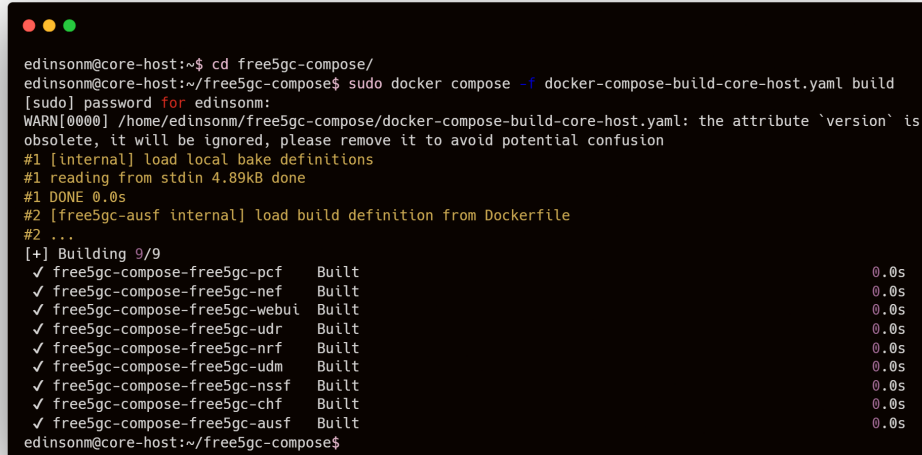
1    networks:
2        macvlan_net:
3            external: true
4
5    volumes:
6        dbdata:

```

Código 5.11: Configuración de red y volúmenes en Docker Compose

5.2.1.3 Construcción y despliegue

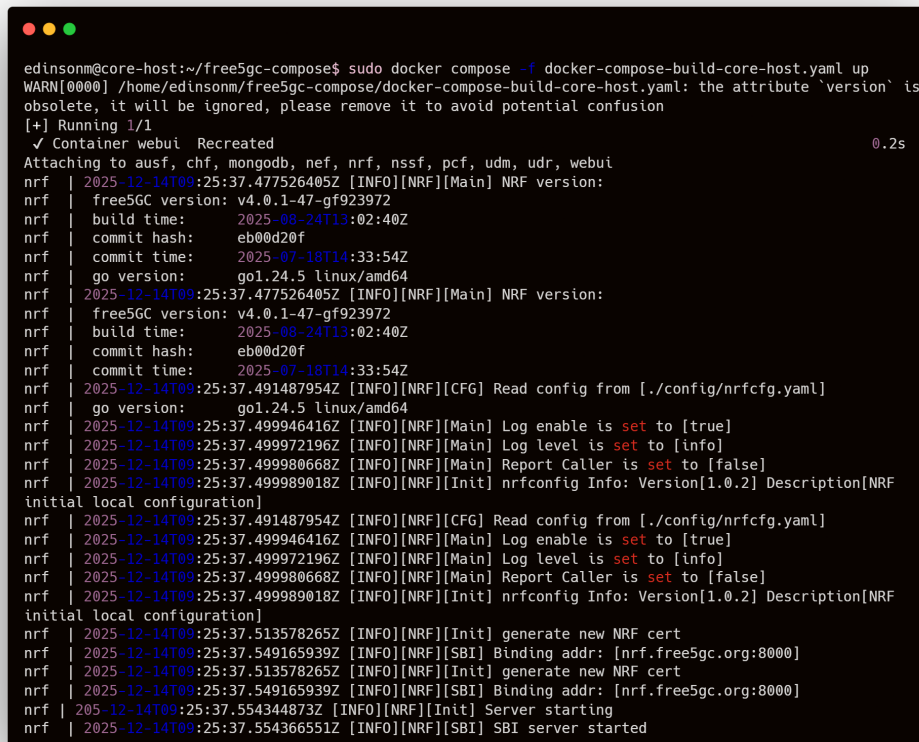
Una vez definido el archivo Docker Compose con la configuración de todas las NFs de Core-Host, se procede a construir las imágenes de acuerdo con el archivo Docker Compose como se puede apreciar en la figura 5.6.

A terminal window with a dark background and light text. The prompt is 'edinsonm@core-host:~\$'. The user enters 'cd free5gc-compose/'. The prompt changes to 'edinsonm@core-host:~/free5gc-compose\$'. The user enters 'sudo docker compose -f docker-compose-build-core-host.yaml build'. The prompt changes to '[sudo] password for edinsonm:'. The user enters a password. The terminal shows a warning: 'WARN[0000] /home/edinsonm/free5gc-compose/docker-compose-build-core-host.yaml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion'. Then it shows progress: '#1 [internal] load local bake definitions', '#1 reading from stdin 4.89kB done', '#1 DONE 0.0s', '#2 [free5gc-ausf internal] load build definition from Dockerfile', '#2 ...'. Then it shows a list of services being built: '[+] Building 9/9', followed by a list of services and their build status and time: '✓ free5gc-compose-free5gc-pcf Built 0.0s', '✓ free5gc-compose-free5gc-nef Built 0.0s', '✓ free5gc-compose-free5gc-webui Built 0.0s', '✓ free5gc-compose-free5gc-udr Built 0.0s', '✓ free5gc-compose-free5gc-nrf Built 0.0s', '✓ free5gc-compose-free5gc-udm Built 0.0s', '✓ free5gc-compose-free5gc-nssf Built 0.0s', '✓ free5gc-compose-free5gc-chf Built 0.0s', '✓ free5gc-compose-free5gc-ausf Built 0.0s'. The prompt returns to 'edinsonm@core-host:~/free5gc-compose\$'.

```
edinsonm@core-host:~$ cd free5gc-compose/
edinsonm@core-host:~/free5gc-compose$ sudo docker compose -f docker-compose-build-core-host.yaml build
[sudo] password for edinsonm:
WARN[0000] /home/edinsonm/free5gc-compose/docker-compose-build-core-host.yaml: the attribute `version` is
obsolete, it will be ignored, please remove it to avoid potential confusion
#1 [internal] load local bake definitions
#1 reading from stdin 4.89kB done
#1 DONE 0.0s
#2 [free5gc-ausf internal] load build definition from Dockerfile
#2 ...
[+] Building 9/9
✓ free5gc-compose-free5gc-pcf      Built      0.0s
✓ free5gc-compose-free5gc-nef      Built      0.0s
✓ free5gc-compose-free5gc-webui    Built      0.0s
✓ free5gc-compose-free5gc-udr      Built      0.0s
✓ free5gc-compose-free5gc-nrf      Built      0.0s
✓ free5gc-compose-free5gc-udm      Built      0.0s
✓ free5gc-compose-free5gc-nssf     Built      0.0s
✓ free5gc-compose-free5gc-chf      Built      0.0s
✓ free5gc-compose-free5gc-ausf     Built      0.0s
edinsonm@core-host:~/free5gc-compose$
```

Figure 5.6: Construcción de imágenes en Core-Host

Luego de construir las imagenes y como se puede apreciar en la figura 5.7, se procede a desplegar los contenedores de los cuales se destaca el log del NRF, quien orquesta todo el Core 5G basado en la arquitectura SBI descrita en la figura 2.3. El resto de logs de los demas contenedores no se muestran por cuestiones de espacio.



```

edinsonm@core-host:~/free5gc-compose$ sudo docker compose -f docker-compose-build-core-host.yaml up
WARN[0000] /home/edinsonm/free5gc-compose/docker-compose-build-core-host.yaml: the attribute `version` is
obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 1/1
  ✓ Container webui Recreated                                0.2s
Attaching to ausf, chf, mongodb, nef, nrf, nssf, pcf, udm, udr, webui
nrf | 2025-12-14T09:25:37.477526405Z [INFO][NRF][Main] NRF version:
nrf | free5GC version: v4.0.1-47-gf923972
nrf | build time: 2025-08-24T13:02:40Z
nrf | commit hash: eb00d20f
nrf | commit time: 2025-07-18T14:33:54Z
nrf | go version: go1.24.5 linux/amd64
nrf | 2025-12-14T09:25:37.477526405Z [INFO][NRF][Main] NRF version:
nrf | free5GC version: v4.0.1-47-gf923972
nrf | build time: 2025-08-24T13:02:40Z
nrf | commit hash: eb00d20f
nrf | commit time: 2025-07-18T14:33:54Z
nrf | 2025-12-14T09:25:37.491487954Z [INFO][NRF][CFG] Read config from [./config/nrfcfg.yaml]
nrf | go version: go1.24.5 linux/amd64
nrf | 2025-12-14T09:25:37.499946416Z [INFO][NRF][Main] Log enable is set to [true]
nrf | 2025-12-14T09:25:37.499972196Z [INFO][NRF][Main] Log level is set to [info]
nrf | 2025-12-14T09:25:37.499980668Z [INFO][NRF][Main] Report Caller is set to [false]
nrf | 2025-12-14T09:25:37.499989018Z [INFO][NRF][Init] nrfconfig Info: Version[1.0.2] Description[NRF
initial local configuration]
nrf | 2025-12-14T09:25:37.491487954Z [INFO][NRF][CFG] Read config from [./config/nrfcfg.yaml]
nrf | 2025-12-14T09:25:37.499946416Z [INFO][NRF][Main] Log enable is set to [true]
nrf | 2025-12-14T09:25:37.499972196Z [INFO][NRF][Main] Log level is set to [info]
nrf | 2025-12-14T09:25:37.499980668Z [INFO][NRF][Main] Report Caller is set to [false]
nrf | 2025-12-14T09:25:37.499989018Z [INFO][NRF][Init] nrfconfig Info: Version[1.0.2] Description[NRF
initial local configuration]
nrf | 2025-12-14T09:25:37.513578265Z [INFO][NRF][Init] generate new NRF cert
nrf | 2025-12-14T09:25:37.549165939Z [INFO][NRF][SBI] Binding addr: [nrf.free5gc.org:8000]
nrf | 2025-12-14T09:25:37.513578265Z [INFO][NRF][Init] generate new NRF cert
nrf | 2025-12-14T09:25:37.549165939Z [INFO][NRF][SBI] Binding addr: [nrf.free5gc.org:8000]
nrf | 2025-12-14T09:25:37.554344873Z [INFO][NRF][Init] Server starting
nrf | 2025-12-14T09:25:37.554366551Z [INFO][NRF][SBI] SBI server started

```

Figure 5.7: Despliegue de contenedores en Core-Host

5.2.2 Configuración de AMF

El proceso de obtención, compilación y despliegue del AMF es similar al realizado para el Core-Host. Para fines de ahorrar espacio, no se mostrará el proceso de clonación. En cambio se mostrará la construcción de la imagen Docker del AMF y su posterior despliegue.

5.2.2.1 Construcción de la imagen Docker del AMF

```

edinsonm@amf:~/free5gc-compose$ sudo make amf
docker build -t 'free5gc/' 'base': 'latest' ./base
[+] Building 2.4s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 479B
=> [internal] load metadata for docker.io/library/golang:1.21.8-bullseye
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/golang:1.21.8-bullseye@sha256:81d98548f08e22a59c5c0be814acc0997f
=> CACHED [2/3] RUN apt-get update && apt-get -y install gcc cmake autoconf libtool pkg-conf
=> CACHED [3/3] RUN apt-get clean
=> exporting to image
=> => exporting layers
=> => writing image sha256:8e04e0e085a5583cc8ac3d2d3556b962ed01d02c2abe69d33a6579dala75fea4
=> => naming to docker.io/free5gc/base:latest

1 warning found (use docker --debug to expand):
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line
0)
docker image ls 'free5gc/' 'base': 'latest'
REPOSITORY TAG IMAGE ID CREATED SIZE
free5gc/base latest 8e04e0e085a5 22 hours ago 849MB
docker build --build-arg F5GC_MODULE=amf -t 'free5gc/' 'amf-base': 'latest' -f ./base/Dockerfile.nf ./base
[+] Building 387.8s (15/15) FINISHED
=> [internal] load build definition from Dockerfile.nf
=> => transferring dockerfile: 767B
=> WARN: FromAsCasing: 'as' and 'FROM' keywords' casing do not match (line 5)
=> [internal] load metadata for docker.io/library/alpine:3.15
=> [internal] load metadata for docker.io/free5gc/base:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [stage-1 1/6] FROM docker.io/library/alpine:3.15@sha256:19b4bcc4f60e99dd5ebdca0cbce22c503bbcf
=> [my-base 1/3] FROM docker.io/free5gc/base:latest
=> [internal] load build context
=> => transferring context: 142.30kB
=> CACHED [stage-1 2/6] WORKDIR /free5gc
=> CACHED [stage-1 3/6] RUN mkdir -p cert/ public
=> CACHED [my-base 2/3] COPY free5gc/ /go/src/free5gc/
=> [my-base 3/3] RUN cd /go/src/free5gc && make amf
=> [stage-1 4/6] COPY --from=my-base /go/src/free5gc/bin/ ./
=> [stage-1 5/6] COPY --from=my-base /go/src/free5gc/config/* ./config/
=> [stage-1 6/6] COPY --from=my-base /go/src/free5gc/cert/* ./cert/
=> exporting to image
=> => exporting layers
=> => writing image sha256:acd76169aa75d716f3875b30dba2ac69789c10595cc38315181c9374d6a35dee
=> => naming to docker.io/free5gc/amf-base:latest

3 warnings found (use docker --debug to expand):
- UndefinedVar: Usage of undefined variable '$F5GC_MODULE' (line 23)
- FromAsCasing: 'as' and 'FROM' keywords' casing do not match (line 5)
- LegacyKeyValueFormat: "ENV key=value" should be used instead of legacy "ENV key value" format (line
0)
edinsonm@amf:~/free5gc-compose$

```

Figure 5.8: Construcción de la imagen Docker del AMF

5.2.2.2 Despliegue del contenedor AMF

Para el despliegue del contenedor AMF, se crea un archivo Docker Compose llamado *docker-compose-build-amf.yaml* con la configuración del AMF mostrado en el siguiente código 5.12.

```
1 services:
```



```
2 free5gc-amf:
3   container_name: amf
4   build:
5     context: ./nf_amf
6     args:
7       DEBUG_TOOLS: "false"
8   command: ./amf -c ./config/amfcfg.yaml
9   expose:
10    - "8000"
11    - "38412"
12   volumes:
13    - ./config/amfcfg.yaml:/free5gc/config/amfcfg.yaml
14    - ./cert:/free5gc/cert
15   environment:
16     GIN_MODE: release
17   ports:
18    - "38412:38412"
19    - "8000:8000"
20   networks:
21     macvlan_net:
22       ipv4_address: 172.18.0.20
23       mac_address: f2:ff:cb:34:07:b5
24       aliases:
25         - amf.free5gc.org
26   extra_hosts:
27     - "nrf.free5gc.org:172.18.0.51"
28     - "udr.free5gc.org:172.18.0.56"
29     - "pcf.free5gc.org:172.18.0.54"
30     - "ausf.free5gc.org:172.18.0.52"
31     - "udm.free5gc.org:172.18.0.55"
32     - "nssf.free5gc.org:172.18.0.53"
33     - "smf.free5gc.org:172.18.0.22"
34     - "upf.free5gc.org:172.18.0.24"
35     - "gnb.free5gc.org:172.18.10.3"
36 networks:
37   macvlan_net:
38     external: true
```

Código 5.12: Configuración del Docker Compose AMF

5.2.2.3 Registro del AMF en el NRF

Una vez desplegado el contenedor AMF, se verifica en los logs del NRF que el AMF se haya registrado correctamente en el NRF mediante la interfaz SBI de acuerdo con la arquitectura mostrada en la figura 2.3. Además, que empiece a escuchar en el puerto SCTP 38412 y que este listo para recibir conexiones desde el NGRAN, como se muestra en la figura 5.9.

```

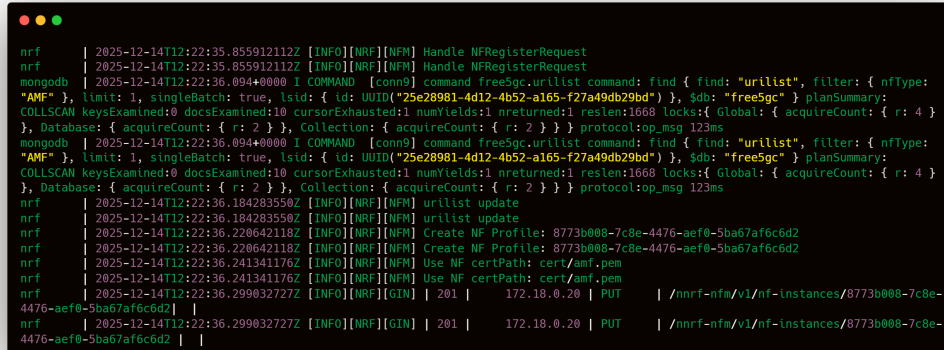
edinsonm@amf:~$ cd free5gc-compose/
edinsonm@amf:~/free5gc-compose$
edinsonm@amf:~/free5gc-compose$ sudo docker compose -f docker-compose-build-amf.yaml up
Attaching to amf
amf | 2025-12-14T12:22:35.650929989Z [INFO][AMF][Main] AMF version:
amf | free5GC version: v4.0.1-47-gf923972
amf | build time: 2025-08-24T11:38:21Z
amf | commit hash: cd9f6775
amf | commit time: 2025-07-18T14:24:15Z
amf | go version: go1.24.5 linux/amd64
amf | 2025-12-14T12:22:35.650929989Z [INFO][AMF][Main] AMF version:
amf | free5GC version: v4.0.1-47-gf923972
amf | build time: 2025-08-24T11:38:21Z
amf | commit hash: cd9f6775
amf | commit time: 2025-07-18T14:24:15Z
amf | go version: go1.24.5 linux/amd64
amf | 2025-12-14T12:22:35.663255384Z [INFO][AMF][CFG] Read config from [./config/amfcfg.yaml]
amf | 2025-12-14T12:22:35.663255384Z [INFO][AMF][CFG] Read config from [./config/amfcfg.yaml]
amf | 2025-12-14T12:22:35.686628116Z [INFO][AMF][Main] Log enable is set to [true]
amf | 2025-12-14T12:22:35.686771861Z [INFO][AMF][Main] Log level is set to [info]
amf | 2025-12-14T12:22:35.686781793Z [INFO][AMF][Main] Report Caller is set to [false]
amf | 2025-12-14T12:22:35.686628116Z [INFO][AMF][Main] Log enable is set to [true]
amf | 2025-12-14T12:22:35.686771861Z [INFO][AMF][Main] Log level is set to [info]
amf | 2025-12-14T12:22:35.686781793Z [INFO][AMF][Main] Report Caller is set to [false]
amf | 2025-12-14T12:22:35.693597345Z [INFO][AMF][SBI] Binding addr: [amf.free5gc.org:8000]
amf | 2025-12-14T12:22:35.693892050Z [INFO][AMF][Util] amfconfig Info: Version[1.0.9]

amf | 2025-12-14T12:22:35.693597345Z [INFO][AMF][SBI] Binding addr: [amf.free5gc.org:8000]
amf | 2025-12-14T12:22:35.723822566Z [INFO][AMF][Init] Server started
amf | 2025-12-14T12:22:35.723822566Z [INFO][AMF][Init] Server started
amf | 2025-12-14T12:22:35.792339468Z [INFO][AMF][Ngap] Listen on 172.18.0.20:38412
amf | 2025-12-14T12:22:35.792339468Z [INFO][AMF][Ngap] Listen on 172.18.0.20:38412
amf | 2025-12-14T12:22:36.320213828Z [INFO][AMF][Main] OAuth2 setting receive from NRF: true
amf | 2025-12-14T12:22:36.320213828Z [INFO][AMF][Main] OAuth2 setting receive from NRF: true
amf | 2025-12-14T12:22:36.321023128Z [INFO][AMF][SBI] Start SBI server (listen on
amf.free5gc.org:8000) 2025-12-14T12:22:36.321023128Z [INFO][AMF][SBI] Start SBI server (listen on

```

Figure 5.9: Registro del AMF en el NRF

En la figura 5.10 se muestran los logs del NRF que confirman que el AMF se ha registrado correctamente y que esta escuchando en el puerto SCTP 38412, listo para recibir conexiones del NGRAN.



```

nrf | 2025-12-14T12:22:35.855912112Z [INFO][NRF][NFM] Handle NFRegisterRequest
nrf | 2025-12-14T12:22:35.855912112Z [INFO][NRF][NFM] Handle NFRegisterRequest
mongod | 2025-12-14T12:22:36.094400000 I COMMAND [conn9] command free5gc.urllist command: find { find: "urllist", filter: { nfType:
"AMF" }, limit: 1, singleBatch: true, lsid: { id: UUID("25e28981-4d12-4b52-a165-f27a49db29bd") }, $db: "free5gc" } planSummary:
COLLSCAN keysExamined:0 docsExamined:10 cursorExhausted:1 numYields:1 nreturned:1 reslen:1668 locks:{ Global: { acquireCount: { r: 4 }
}, Database: { acquireCount: { r: 2 } }, Collection: { acquireCount: { r: 2 } }} protocol:op_msg 123ms
mongod | 2025-12-14T12:22:36.094400000 I COMMAND [conn9] command free5gc.urllist command: find { find: "urllist", filter: { nfType:
"AMF" }, limit: 1, singleBatch: true, lsid: { id: UUID("25e28981-4d12-4b52-a165-f27a49db29bd") }, $db: "free5gc" } planSummary:
COLLSCAN keysExamined:0 docsExamined:10 cursorExhausted:1 numYields:1 nreturned:1 reslen:1668 locks:{ Global: { acquireCount: { r: 4 }
}, Database: { acquireCount: { r: 2 } }, Collection: { acquireCount: { r: 2 } }} protocol:op_msg 123ms
nrf | 2025-12-14T12:22:36.184283550Z [INFO][NRF][NFM] urllist update
nrf | 2025-12-14T12:22:36.184283550Z [INFO][NRF][NFM] urllist update
nrf | 2025-12-14T12:22:36.220642118Z [INFO][NRF][NFM] Create NF Profile: 8773b008-7c8e-4476-aef0-5ba67af6c6d2
nrf | 2025-12-14T12:22:36.220642118Z [INFO][NRF][NFM] Create NF Profile: 8773b008-7c8e-4476-aef0-5ba67af6c6d2
nrf | 2025-12-14T12:22:36.241341176Z [INFO][NRF][NFM] Use NF certPath: cert/amf.pem
nrf | 2025-12-14T12:22:36.241341176Z [INFO][NRF][NFM] Use NF certPath: cert/amf.pem
nrf | 2025-12-14T12:22:36.299032727Z [INFO][NRF][GIN] | 201 | 172.18.0.20 | PUT | /nnrf-nfm/v1/nf-Instances/8773b008-7c8e-
4476-aef0-5ba67af6c6d2 |
nrf | 2025-12-14T12:22:36.299032727Z [INFO][NRF][GIN] | 201 | 172.18.0.20 | PUT | /nnrf-nfm/v1/nf-Instances/8773b008-7c8e-
4476-aef0-5ba67af6c6d2 |

```

Figure 5.10: Verificación del registro del AMF en el NRF

Las configuraciones del AMF, como las direcciones IP y puertos de escucha, se realizan en el archivo de configuración *amfcfg.yaml* ubicado en el directorio *./config/* del repositorio del AMF, las cuales no se muestran en este documento por cuestiones de espacio.

5.2.3 Configuración de SMF

A diferencia de los NFs desplegados en el Core-Host y SMF, tanto gNB (UER-ANSIM), SMF como UPF requieren de la version kernel 5.0.0-23-generic o superior a la 5.4 los cuales son compatibles con el modulo de kernel GTP para 5G.

5.2.3.1 Instalacion del modulo GTP

Como primer paso antes de instalar el modulo GTP, se instala la version de kernel requerida como se muestra en la figura 5.11.



```
# Descarga los paquetes del kernel 5.4.0-65.73
edinsonm@smf:~/Downloads$ wget http://security.ubuntu.com/ubuntu/pool/main/l/linux/linux-headers-5.4.0-65.73_5.4.0-65.73_all.deb
edinsonm@smf:~/Downloads$ wget http://security.ubuntu.com/ubuntu/pool/main/l/linux/linux-headers-5.4.0-65-generic_5.4.0-65.73_amd64.deb
edinsonm@smf:~/Downloads$ wget http://security.ubuntu.com/ubuntu/pool/main/l/linux/linux-image-unsigned-5.4.0-65-generic_5.4.0-65.73_amd64.deb
edinsonm@smf:~/Downloads$ wget http://security.ubuntu.com/ubuntu/pool/main/l/linux/linux-modules-5.4.0-65-generic_5.4.0-65.73_amd64.deb

# Instalacion del Kernel
edinsonm@smf:~/Downloads$ sudo dpkg -i linux-headers-5.4.0-65.73_5.4.0-65.73_all.deb \
> linux-headers-5.4.0-65-generic_5.4.0-65.73_amd64.deb \
> linux-image-unsigned-5.4.0-65-generic_5.4.0-65.73_amd64.deb \
> linux-modules-5.4.0-65-generic_5.4.0-65.73_amd64.deb
[sudo] password for edinsonm:
(Reading database ... 173781 files and directories currently installed.)
Preparing to unpack linux-headers-5.4.0-65.73_5.4.0-65.73_all.deb ...
Unpacking linux-headers-5.4.0-65 (5.4.0-65.73) over (5.4.0-65.73) ...
Preparing to unpack linux-headers-5.4.0-65-generic_5.4.0-65.73_amd64.deb ...
Unpacking linux-headers-5.4.0-65-generic (5.4.0-65.73) over (5.4.0-65.73) ...
Preparing to unpack linux-image-unsigned-5.4.0-65-generic_5.4.0-65.73_amd64.deb ...
Unpacking linux-image-unsigned-5.4.0-65-generic (5.4.0-65.73) over (5.4.0-65.73) ...
Preparing to unpack linux-modules-5.4.0-65-generic_5.4.0-65.73_amd64.deb ...
Unpacking linux-modules-5.4.0-65-generic (5.4.0-65.73) over (5.4.0-65.73) ...
Setting up linux-headers-5.4.0-65 (5.4.0-65.73) ...
Setting up linux-headers-5.4.0-65-generic (5.4.0-65.73) ...
Setting up linux-modules-5.4.0-65-generic (5.4.0-65.73) ...
Setting up linux-image-unsigned-5.4.0-65-generic (5.4.0-65.73) ...
Processing triggers for linux-image-unsigned-5.4.0-65-generic (5.4.0-65.73) ...
/etc/kernel/postinst.d/initramfs-tools:
update-initramfs: Generating /boot/initrd.img-5.4.0-65-generic
I: The initramfs will attempt to resume from /dev/vda3
I: (UUID=2812a9af-5b71-4795-84cd-e9cff3852e2a)
I: Set the RESUME variable to override this.
/etc/kernel/postinst.d/zz-update-grub:
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-5.13.0-30-generic
Found initrd image: /boot/initrd.img-5.13.0-30-generic
Found linux image: /boot/vmlinuz-5.4.0-65-generic
Found initrd image: /boot/initrd.img-5.4.0-65-generic
Found memtest86+ image: /memtest86+.elf
Found memtest86+ image: /memtest86+.bin
done
```

Figure 5.11: Instalación de kernel compatible con GTP

Luego de instalar el kernel compatible, se edita y actualiza el archivo `/etc/default/grub` para establecer la nueva version de kernel como predeterminada al iniciar el sistema, como se muestra en la figura 5.12.

```
edinsonm@smf:~/Downloads$ more /etc/default/grub
# If you change this file, run 'update-grub' afterwards to update
# /boot/grub/grub.cfg.
# For full documentation of the options in this file, see:
#   info -f grub -n 'Simple configuration'

GRUB_DEFAULT="1>2"
GRUB_TIMEOUT_STYLE=hidden
#GRUB_TIMEOUT=0
GRUB_TIMEOUT_STYLE=menu GRUB_TIMEOUT=10
GRUB_DISTRIBUTOR=`lsb_release -i -s 2> /dev/null || echo Debian`
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
GRUB_CMDLINE_LINUX=""

# Uncomment to enable BadRAM filtering, modify to suit your needs
# This works with Linux (no patch required) and with any kernel that obtains
# the memory map information from GRUB (GNU Mach, kernel of FreeBSD ...)
#GRUB_BADRAM="0x01234567,0xfefefefe,0x89abcdef,0xefefefef"

# Uncomment to disable graphical terminal (grub-pc only)
#GRUB_TERMINAL=console

# The resolution used on graphical terminal
# note that you can use only modes which your graphic card supports via VBE
# you can see them in real GRUB with the command `vbeinfo'
#GRUB_GFXMODE=640x480

# Uncomment if you don't want GRUB to pass "root=UUID=xxx" parameter to
#GRUB_DISABLE_LINUX_UUID=true

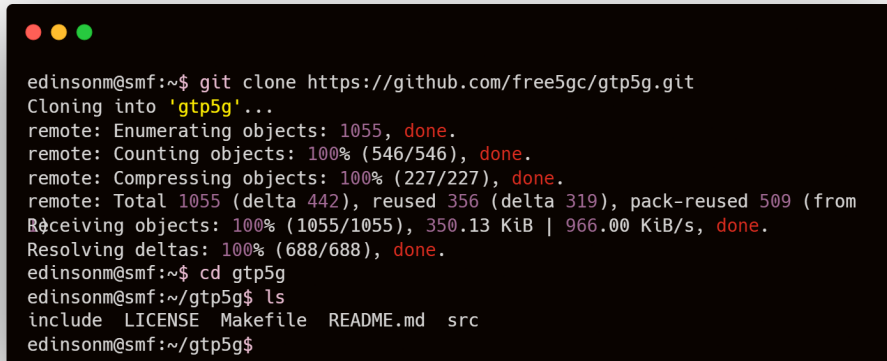
# Uncomment to disable generation of recovery mode menu entries
#GRUB_DISABLE_RECOVERY="true"

# Uncomment to get a beep at grub start
#GRUB_INIT_TUNE="480 440 1"
edinsonm@smf:~/Downloads$
edinsonm@smf:~/Downloads$ sudo update-grub
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-5.13.0-30-generic
Found initrd image: /boot/initrd.img-5.13.0-30-generic
Found linux image: /boot/vmlinuz-5.4.0-65-generic
Found initrd image: /boot/initrd.img-5.4.0-65-generic
Found memtest86+ image: /memtest86+.elf
Found memtest86+ image: /memtest86+.bin
done
edinsonm@smf:~/Downloads$

edinsonm@smf:~$ uname -r
5.4.0-65-generic
edinsonm@smf:~$
```

Figure 5.12: Edición del archivo y actualización de /etc/default/grub

Una vez reiniciado el sistema con la nueva version de kernel, se procede con la instalacion del modulo GTP. Primero se clona el repositorio oficial desde GitHub como se muestra en la figura 5.13.

A terminal window with a dark background and light-colored text. The prompt is 'edinsonm@smf:~\$'. The command 'git clone https://github.com/free5gc/gtp5g.git' is entered. The output shows the cloning process: 'Cloning into 'gtp5g'...', 'remote: Enumerating objects: 1055, done.', 'remote: Counting objects: 100% (546/546), done.', 'remote: Compressing objects: 100% (227/227), done.', 'remote: Total 1055 (delta 442), reused 356 (delta 319), pack-reused 509 (from 0)', 'Receiving objects: 100% (1055/1055), 350.13 KiB | 966.00 KiB/s, done.', 'Resolving deltas: 100% (688/688), done.'. The prompt changes to 'edinsonm@smf:~/gtp5g\$'. The command 'ls' is entered, and the output is 'include LICENSE Makefile README.md src'. The prompt returns to 'edinsonm@smf:~/gtp5g\$'.

```
edinsonm@smf:~$ git clone https://github.com/free5gc/gtp5g.git
Cloning into 'gtp5g'...
remote: Enumerating objects: 1055, done.
remote: Counting objects: 100% (546/546), done.
remote: Compressing objects: 100% (227/227), done.
remote: Total 1055 (delta 442), reused 356 (delta 319), pack-reused 509 (from 0)
Receiving objects: 100% (1055/1055), 350.13 KiB | 966.00 KiB/s, done.
Resolving deltas: 100% (688/688), done.
edinsonm@smf:~$ cd gtp5g
edinsonm@smf:~/gtp5g$ ls
include LICENSE Makefile README.md src
edinsonm@smf:~/gtp5g$
```

Figure 5.13: Clonación del repositorio del módulo GTP

Luego de clonar el repositorio, se limpia, compila e instala el modulo GTP en el kernel como se muestra en la figura 5.14.

```

edinsonm@smf:~/gtp5g$
edinsonm@smf:~/gtp5g$ make clean
make -C /lib/modules/5.4.0-65-generic/build M=/home/edinsonm/gtp5g clean
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-65-generic'
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-65-generic'
edinsonm@smf:~/gtp5g$
edinsonm@smf:~/gtp5g$ make
make -C /lib/modules/5.4.0-65-generic/build M=/home/edinsonm/gtp5g
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-65-generic'
  CC [M] /home/edinsonm/gtp5g/src/gtp5g.o
  CC [M] /home/edinsonm/gtp5g/src/log.o
  CC [M] /home/edinsonm/gtp5g/src/util.o
  CC [M] /home/edinsonm/gtp5g/src/gtpu/dev.o
  CC [M] /home/edinsonm/gtp5g/src/gtpu/encap.o
  CC [M] /home/edinsonm/gtp5g/src/gtpu/hash.o
  CC [M] /home/edinsonm/gtp5g/src/gtpu/link.o
  CC [M] /home/edinsonm/gtp5g/src/gtpu/net.o
  CC [M] /home/edinsonm/gtp5g/src/gtpu/pktinfo.o
  CC [M] /home/edinsonm/gtp5g/src/gtpu/trTCM.o
  CC [M] /home/edinsonm/gtp5g/src/genl/genl.o
  CC [M] /home/edinsonm/gtp5g/src/genl/genl_version.o
  CC [M] /home/edinsonm/gtp5g/src/genl/genl_pdr.o
  CC [M] /home/edinsonm/gtp5g/src/genl/genl_far.o
  CC [M] /home/edinsonm/gtp5g/src/genl/genl_qer.o
  CC [M] /home/edinsonm/gtp5g/src/genl/genl_urr.o
  CC [M] /home/edinsonm/gtp5g/src/genl/genl_report.o
  CC [M] /home/edinsonm/gtp5g/src/genl/genl_bar.o
  CC [M] /home/edinsonm/gtp5g/src/pfcp/api_version.o
  CC [M] /home/edinsonm/gtp5g/src/pfcp/pdr.o
  CC [M] /home/edinsonm/gtp5g/src/pfcp/far.o
  CC [M] /home/edinsonm/gtp5g/src/pfcp/qer.o
  CC [M] /home/edinsonm/gtp5g/src/pfcp/urr.o
  CC [M] /home/edinsonm/gtp5g/src/pfcp/bar.o
  CC [M] /home/edinsonm/gtp5g/src/pfcp/seid.o
  CC [M] /home/edinsonm/gtp5g/src/proc.o
  LD [M] /home/edinsonm/gtp5g/gtp5g.o
Building modules, stage 2.
MODPOST 1 modules
  CC [M] /home/edinsonm/gtp5g/gtp5g.mod.o
  LD [M] /home/edinsonm/gtp5g/gtp5g.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-65-generic'
edinsonm@smf:~/gtp5g$
edinsonm@smf:~/gtp5g$ sudo make install
cp gtp5g.ko /lib/modules/5.4.0-65-generic/kernel/drivers/net
modprobe udp_tunnel
/sbin/depmod -a
modprobe gtp5g
echo "udp_tunnel" > /etc/modules-load.d/gtp5g.conf
echo "gtp5g" >> /etc/modules-load.d/gtp5g.conf
edinsonm@smf:~/gtp5g$

```

Figure 5.14: Compilación e instalación del módulo GTP

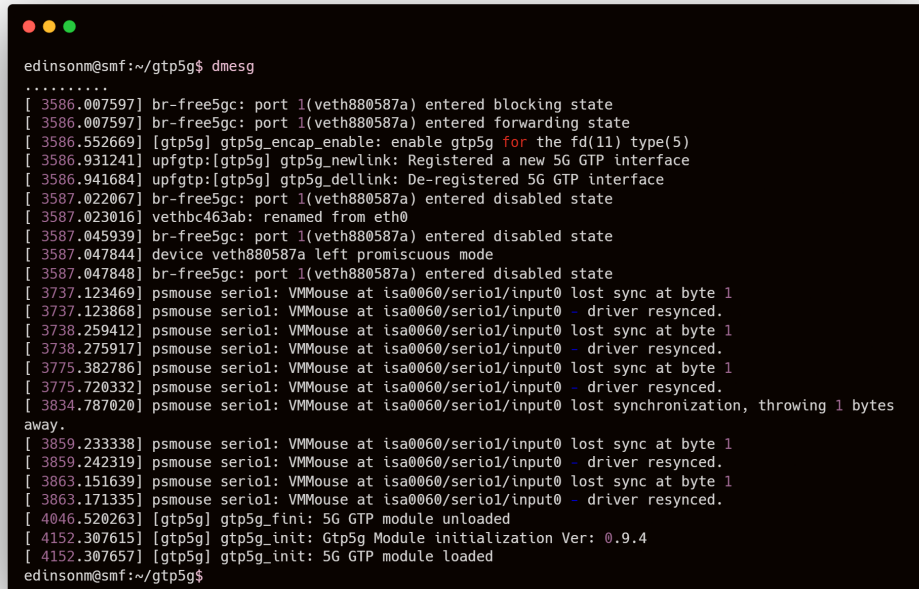
Luego de instalado el modulo GTP, se carga el modulo gtp5g en el kernel y luego se listan los modulos cargados para verificar que el modulo GTP se haya cargado correctamente, como se muestra en la figura 5.15.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The terminal shows a user named 'edinsonm' at a host 'smf' in the directory '~/gtp5g'. The user enters 'sudo modprobe gtp5g' and then 'lsmod | grep gtp5g'. The output of the second command shows two loaded modules: 'gtp5g' with size 143360 and reference count 0, and 'udp_tunnel' with size 16384 and reference count 1, which is dependent on 'gtp5g'.

```
edinsonm@smf:~/gtp5g$  
edinsonm@smf:~/gtp5g$ sudo modprobe  
gtp5g  
edinsonm@smf:~/gtp5g$ lsmod | grep gtp5g  
gtp5g                143360  0  
udp_tunnel           16384   1 gtp5g  
edinsonm@smf:~/gtp5g$
```

Figure 5.15: Carga y verificación del módulo GTP

Finalmente, se revisa en los logs del sistema que el modulo GTP se haya cargado correctamente al iniciar el sistema, como se muestra en la figura 5.16.



```

edinsonm@smf:~/gtp5g$ dmesg
.....
[ 3586.007597] br-free5gc: port 1(veth880587a) entered blocking state
[ 3586.007597] br-free5gc: port 1(veth880587a) entered forwarding state
[ 3586.552669] [gtp5g] gtp5g_encap_enable: enable gtp5g for the fd(11) type(5)
[ 3586.931241] upfgtp:[gtp5g] gtp5g_newlink: Registered a new 5G GTP interface
[ 3586.941684] upfgtp:[gtp5g] gtp5g_dellink: De-registered 5G GTP interface
[ 3587.022067] br-free5gc: port 1(veth880587a) entered disabled state
[ 3587.023016] vethbc463ab: renamed from eth0
[ 3587.045939] br-free5gc: port 1(veth880587a) entered disabled state
[ 3587.047844] device veth880587a left promiscuous mode
[ 3587.047848] br-free5gc: port 1(veth880587a) entered disabled state
[ 3737.123469] psmouse serial: VMMouse at isa0060/serial/input0 lost sync at byte 1
[ 3737.123868] psmouse serial: VMMouse at isa0060/serial/input0 - driver resynced.
[ 3738.259412] psmouse serial: VMMouse at isa0060/serial/input0 lost sync at byte 1
[ 3738.275917] psmouse serial: VMMouse at isa0060/serial/input0 - driver resynced.
[ 3775.382786] psmouse serial: VMMouse at isa0060/serial/input0 lost sync at byte 1
[ 3775.720332] psmouse serial: VMMouse at isa0060/serial/input0 - driver resynced.
[ 3834.787020] psmouse serial: VMMouse at isa0060/serial/input0 lost synchronization, throwing 1 bytes
away.
[ 3859.233338] psmouse serial: VMMouse at isa0060/serial/input0 lost sync at byte 1
[ 3859.242319] psmouse serial: VMMouse at isa0060/serial/input0 - driver resynced.
[ 3863.151639] psmouse serial: VMMouse at isa0060/serial/input0 lost sync at byte 1
[ 3863.171335] psmouse serial: VMMouse at isa0060/serial/input0 - driver resynced.
[ 4046.520263] [gtp5g] gtp5g_fini: 5G GTP module unloaded
[ 4152.307615] [gtp5g] gtp5g_init: Gtp5g Module initialization Ver: 0.9.4
[ 4152.307657] [gtp5g] gtp5g_init: 5G GTP module loaded
edinsonm@smf:~/gtp5g$

```

Figure 5.16: Verificación del módulo GTP en los logs del sistema

5.2.3.2 Obtención e instalación del código fuente SMF

Este proceso es similar al realizado para el Core-Host, y el AMF. Para fines de ahorrar espacio, no se mostrará el proceso de clonacion y construccion de imagen Docker del SMF.

5.2.3.3 Configuración de Docker Compose

Al igual que AMF, para el despliegue del contenedor SMF, se crea un archivo Docker Compose llamado *docker-compose-build-smf.yaml* con la configuración del SMF mostrado en el siguiente código 5.13.

```

1 services:
2   free5gc-smf:
3     container_name: smf
4     build:
5       context: ./nf_smf
6       args:
7         DEBUG_TOOLS: "false"
8     command: ./smf -c ./config/smfcfg.yaml -u ./config/
9             uerouting.yaml
10    expose:

```

```

10     - "8000"
11   volumes:
12     - ./config/smfcfg.yaml:/free5gc/config/smfcfg.yaml
13     - ./config/uerouting.yaml:/free5gc/config/uerouting.
14       yaml
15     - ./cert:/free5gc/cert
16   environment:
17     GIN_MODE: release
18   ports:
19     - "8000:8000"
20   networks:
21     macvlan_net:
22       ipv4_address: 172.18.0.22
23       mac_address: ca:76:58:9c:8e:ec
24       aliases:
25         - smf.free5gc.org
26   extra_hosts:
27     - "nrf.free5gc.org:172.18.0.51"
28     - "udr.free5gc.org:172.18.0.56"
29     - "pcf.free5gc.org:172.18.0.54"
30     - "ausf.free5gc.org:172.18.0.52"
31     - "udm.free5gc.org:172.18.0.55"
32     - "nssf.free5gc.org:172.18.0.53"
33     - "amf.free5gc.org:172.18.0.20"
34     - "upf.free5gc.org:172.18.0.24"
35     - "gnb.free5gc.org:172.18.10.3"
36 networks:
37   macvlan_net:
38     external: true

```

Código 5.13: Configuración del Docker Compose SMF

Al igual que en los NFs anteriores, se le especifica las direcciones IP de los demas NFs para que pueda resolver los nombre de dominio de cada NF mediante el archivo */etc/hosts* del contenedor SMF de acuerdo con el LLD mostrado en la tabla 5.1.

5.2.3.4 Registro del SMF en el NRF y conexión N4 con UPF

Una vez desplegado el contenedor SMF, se verifica que se haya registrado correctamente en el NRF y que se establezca la conexion con el UPF mediante la interfaz N4, orquestado por el NRF segun la arquitectura SBI mostrada en la figura 2.3. Como se puede apreciar en la figura 5.17, el SMF se ha registrado correctamente en el NRF y esta listo para gestionar las sesiones de usuario y la conexion N4 con el UPF esta establecida correctamente.

```

edinsonm@smf:~/free5gc-compose$
edinsonm@smf:~/free5gc-compose$ sudo docker compose -f docker-compose-build-smf.yaml up
Attaching to smf
smf | 2025-12-14T13:34:07.073075123Z [INFO][SMF][Main] SMF version:
smf | free5GC version: v4.0.1-47-gf923972
smf | build time: 2025-08-24T14:14:10Z
smf | commit hash: f7eab072
smf | commit time: 2025-08-01T03:00:56Z
smf | go version: go1.24.5 linux/amd64
smf | 2025-12-14T13:34:07.081071957Z [INFO][SMF][CFG] Read config from [./config/smfcfg.yaml]
smf | 2025-12-14T13:34:07.088413137Z [INFO][SMF][CFG] Read config from [./config/uerouting.yaml]
smf | 2025-12-14T13:34:07.089043295Z [INFO][SMF][Main] Log enable is set to [true]
smf | 2025-12-14T13:34:07.089894869Z [INFO][SMF][Main] Log level is set to [info]
smf | 2025-12-14T13:34:07.073075123Z [INFO][SMF][Main] SMF version:
smf | free5GC version: v4.0.1-47-gf923972
smf | build time: 2025-08-24T14:14:10Z
smf | commit hash: f7eab072
smf | commit time: 2025-08-01T03:00:56Z
smf | go version: go1.24.5 linux/amd64
smf | 2025-12-14T13:34:07.081071957Z [INFO][SMF][CFG] Read config from [./config/smfcfg.yaml]
smf | 2025-12-14T13:34:07.088413137Z [INFO][SMF][CFG] Read config from [./config/uerouting.yaml]
smf | 2025-12-14T13:34:07.089043295Z [INFO][SMF][Main] Log enable is set to [true]
smf | 2025-12-14T13:34:07.089894869Z [INFO][SMF][Main] Log level is set to [info]
smf | 2025-12-14T13:34:07.090006805Z [INFO][SMF][Main] Report Caller is set to [false]
smf | 2025-12-14T13:34:07.090201197Z [INFO][SMF][CTX] smfconfig Info: Version[1.0.7] Description[SMF initial local configuration]
smf | 2025-12-14T13:34:07.090006805Z [INFO][SMF][Main] Report Caller is set to [false]
smf | 2025-12-14T13:34:07.090201197Z [INFO][SMF][CTX] smfconfig Info: Version[1.0.7] Description[SMF initial local configuration]
smf | 2025-12-14T13:34:07.093862099Z [INFO][SMF][CTX] Endpoints: [upf.free5gc.org]
smf | 2025-12-14T13:34:07.094904842Z [INFO][SMF][Init] Server started
smf | 2025-12-14T13:34:07.093862099Z [INFO][SMF][CTX] Endpoints: [upf.free5gc.org]
smf | 2025-12-14T13:34:07.094904842Z [INFO][SMF][Init] Server started
smf | 2025-12-14T13:34:07.763935955Z [INFO][SMF][Main] OAuth2 setting receive from NRF: true
smf | 2025-12-14T13:34:07.763935955Z [INFO][SMF][Main] OAuth2 setting receive from NRF: true
smf | 2025-12-14T13:34:07.765511632Z [INFO][SMF][Init] SMF Registration to NRF {93201f37-e26b-4e0e-a8b8-bf5dc29133ad SMF REGISTERED
[] 0 {208 93}} [] [] {1 010203 [] false} {1 112233 [] false} [] [] [smf.free5gc.org] [] [] [] [] [] 0 0 0 <nil> areal <nil> map[]
<nil> map[] <nil> map[] <nil> map[] 0xc000098b00 map[] <nil> map[] <nil> map[] <nil> map[] <nil> map[] <nil> map[] <nil> map[] <nil>
map[] map[] map[auth2:true] <nil> false {93201f37-e26b-4e0e-a8b8-bf5dc29133adnsmf-pdusession nsmf-pdusession [{v1
https://smf.free5gc.org:8000/nsmf-pdusession/v1 2025-12-14 13:34:07.094801238 +0000 UTC}] https REGISTERED [{smf.free5gc.org 8000}]
http://smf.free5gc.org:8000 [] [] [] [] [] map[] map[] 0 0 0 <nil> <nil> [] [] [] map[] false <nil> {93201f37-e26b-4e0e-a8b8-
bf5dc29133adnsmf-event-exposure nsmf-event-exposure [{v1 https://smf.free5gc.org:8000/nsmf-pdusession/v1 2025-12-14 13:34:07.094801238
+0000 UTC}] https REGISTERED [{smf.free5gc.org 8000}] http://smf.free5gc.org:8000 [] [] [] [] [] map[] map[] 0 0 0 <nil> <nil>
[] [] [] map[] false <nil> {93201f37-e26b-4e0e-a8b8-bf5dc29133adnsmf-oam nsmf-oam [{v1 https://smf.free5gc.org:8000/nsmf-
pdusession/v1 2025-12-14 13:34:07.094801238 +0000 UTC}] https REGISTERED [{smf.free5gc.org 8000}] http://smf.free5gc.org:8000 [] []
[] [] [] map[] map[] 0 0 0 <nil> <nil> [] [] [] map[] false <nil>}} map[] false false [] <nil> <nil> [] [] false false map[] map[]
[] <nil> <nil> map[] <nil> map[] <nil> map[] <nil> map[] <nil> <nil> [] <nil> <nil>}}
smf | 2025-12-14T13:34:07.768595226Z [INFO][SMF][SBI] Start SBI server (listen on smf.free5gc.org:8000)
smf | 2025-12-14T13:34:07.765511632Z [INFO][SMF][Init] SMF Registration to NRF {93201f37-e26b-4e0e-a8b8-bf5dc29133ad SMF REGISTERED
[] 0 {208 93}} [] [] {1 010203 [] false} {1 112233 [] false} [] [] [smf.free5gc.org] [] [] [] [] [] 0 0 0 <nil> areal <nil> map[]
<nil> map[] <nil> map[] <nil> map[] 0xc000098b00 map[] <nil> map[] <nil> map[] <nil> map[] <nil> map[] <nil> map[] <nil>
map[] map[] map[auth2:true] <nil> false {93201f37-e26b-4e0e-a8b8-bf5dc29133adnsmf-pdusession nsmf-pdusession [{v1
https://smf.free5gc.org:8000/nsmf-pdusession/v1 2025-12-14 13:34:07.094801238 +0000 UTC}] https REGISTERED [{smf.free5gc.org 8000}]
http://smf.free5gc.org:8000 [] [] [] [] [] map[] map[] 0 0 0 <nil> <nil> [] [] [] map[] false <nil> {93201f37-e26b-4e0e-a8b8-
bf5dc29133adnsmf-event-exposure nsmf-event-exposure [{v1 https://smf.free5gc.org:8000/nsmf-pdusession/v1 2025-12-14 13:34:07.094801238
+0000 UTC}] https REGISTERED [{smf.free5gc.org 8000}] http://smf.free5gc.org:8000 [] [] [] [] [] map[] map[] 0 0 0 <nil> <nil>
[] [] [] map[] false <nil> {93201f37-e26b-4e0e-a8b8-bf5dc29133adnsmf-oam nsmf-oam [{v1 https://smf.free5gc.org:8000/nsmf-
pdusession/v1 2025-12-14 13:34:07.094801238 +0000 UTC}] https REGISTERED [{smf.free5gc.org 8000}] http://smf.free5gc.org:8000 [] []
[] [] [] map[] map[] 0 0 0 <nil> <nil> [] [] [] map[] false <nil>}} map[] false false [] <nil> <nil> [] [] false false map[] map[]
[] <nil> <nil> map[] <nil> map[] <nil> map[] <nil> map[] <nil> <nil> [] <nil> <nil>}}
smf | 2025-12-14T13:34:07.799697470Z [INFO][SMF][PFPCP] Pfcp running... [2025-12-14 13:34:07.807783451 +0000 UTC m=+0.964714240]
smf | 2025-12-14T13:34:07.799697470Z [INFO][SMF][PFPCP] Listen on 172.18.0.22:8805
smf | 2025-12-14T13:34:07.799697470Z [INFO][SMF][PFPCP] Listen on 172.18.0.22:8805
smf | 2025-12-14T13:34:07.807841594Z [INFO][SMF][PFPCP] Pfcp running... [2025-12-14 13:34:07.807783451 +0000 UTC m=+0.964714240]
smf | 2025-12-14T13:34:08.813301272Z [INFO][SMF][Main] Sending PFPCP Association Request to UPF[upf.free5gc.org](172.18.0.24)
smf | 2025-12-14T13:34:08.813301272Z [INFO][SMF][Main] Sending PFPCP Association Request to UPF[upf.free5gc.org](172.18.0.24)
smf | 2025-12-14T13:34:08.831001462Z [INFO][SMF][Main] Received PFPCP Association Setup Accepted Response from UPF[upf.free5gc.org]
(172.18.0.24)
smf | 2025-12-14T13:34:08.831252271Z [INFO][SMF][Main] UPF(172.18.0.24) setup association
smf | 2025-12-14T13:34:08.831001462Z [INFO][SMF][Main] Received PFPCP Association Setup Accepted Response from UPF[upf.free5gc.org]
(172.18.0.24)
smf | 2025-12-14T13:34:08.831252271Z [INFO][SMF][Main] UPF(172.18.0.24) setup association

```

Figure 5.17: Registro del SMF en el NRF

Logs logs del establecimiento de la conexion N4 entre el SMF y el UPF se muestran a detalle en la session del UPF en la figura 5.19.

5.2.4 Configuración de UPF

El proceso de obtención, compilación, instalación de modulo gtp5g y despliegue del UPF es similar al realizado para el Core-Host, AMF y SMF. Para fines de ahorrar espacio, no se mostrará el proceso de clonacion y construcción de imagen Docker del UPF.

5.2.4.1 Configuración de Docker Compose

Al igual que AMF y SMF, para el despliegue del contenedor UPF, se crea un archivo Docker Compose llamado *docker-compose-build-upf.yaml* con la configuración del UPF mostrado en el siguiente código 5.14.

```

1 version: "3.8"
2 services:
3   free5gc-upf:
4     container_name: upf
5     build:
6       context: ./nf_upf
7       args:
8         DEBUG_TOOLS: "false"
9     command: bash -c "./upf-iptables.sh && ./upf -c ./config/
10              upfcfg.yaml"
11     expose:
12       - "8000"
13       - "38412"
14     volumes:
15       - ./config/upfcfg.yaml:/free5gc/config/upfcfg.yaml
16       - ./config/upf-iptables.sh:/free5gc/upf-iptables.sh
17     cap_add:
18       - NET_ADMIN
19     ports:
20       - "38412:38412"
21       - "8000:8000"
22     networks:
23       macvlan_net:
24         ipv4_address: 172.18.0.24
25         mac_address: be:44:98:a4:e5:c4
26         aliases:
27           - upf.free5gc.org
28     extra_hosts:
29       - "nrf.free5gc.org:172.18.0.51"
30       - "udr.free5gc.org:172.18.0.56"
31       - "pcf.free5gc.org:172.18.0.54"
32       - "ausf.free5gc.org:172.18.0.52"
33       - "udm.free5gc.org:172.18.0.55"
34       - "nssf.free5gc.org:172.18.0.53"
35       - "amf.free5gc.org:172.18.0.20"

```

```
35     - "smf.free5gc.org:172.18.0.22"  
36     - "gnb.free5gc.org:172.18.10.3"  
37 networks:  
38   macvlan_net:  
39     external: true
```

Código 5.14: Configuración del Docker Compose UPF

5.2.4.2 Registro del UPF en el NRF

Una vez desplegado el contenedor UPF, en la figura ?? se aprecia que prepara los parametros para la interfaz N3 con gNB y los recursos para el UE, los cuales son *10.60.0.0/16* y *10.61.0.0/16*.

```

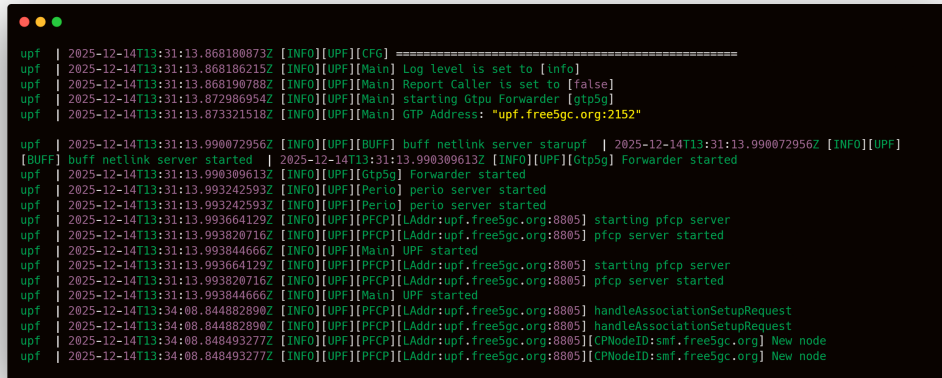
edinsonm@upf:~/free5gc-compose$ sudo docker compose -f docker-compose-build-upf.yaml up
[sudo] password for edinsonm:
WARN[0000] /home/edinsonm/free5gc-compose/docker-compose-build-upf.yaml: the attribute 'version' is obsolete, it will be ignored,
please remove it to avoid potential confusion
Attaching to upf
upf | 2025-12-14T13:31:13.843687483Z [INFO][UPF][Main] UPF version:
upf | free5GC version: v4.0.1-47-gf923972
upf | build time: 2025-08-26T20:17:34Z
upf | commit hash: 9740983c
upf | commit time: 2025-07-18T14:47:46Z
upf | go version: go1.24.5 linux/amd64
upf | 2025-12-14T13:31:13.857445343Z [INFO][UPF][CFG] Read config from [./config/upfcfg.yaml]
upf | 2025-12-14T13:31:13.843687483Z [INFO][UPF][Main] UPF version:
upf | free5GC version: v4.0.1-47-gf923972
upf | build time: 2025-08-26T20:17:34Z
upf | commit hash: 9740983c
upf | commit time: 2025-07-18T14:47:46Z
upf | go version: go1.24.5 linux/amd64
upf | 2025-12-14T13:31:13.857445343Z [INFO][UPF][CFG] Read config from [./config/upfcfg.yaml]
upf | 2025-12-14T13:31:13.868135211Z [INFO][UPF][CFG] =====
upf | 2025-12-14T13:31:13.868173594Z [INFO][UPF][CFG] (*factory.Config)(0xc0001233b0){
upf |   Version: (string) (len=5) "1.0.3",
upf |   Description: (string) (len=31) "UPF initial local configuration",
upf |   Pfcpc: (*factory.Pfcpc)(0xc000176f70){
upf |     Addr: (string) (len=15) "upf.free5gc.org",
upf |     NodeID: (string) (len=15) "upf.free5gc.org",
upf |   },
upf |   Gtpu: (*factory.Gtpu)(0xc000176f68){
upf |     Forwarder: (string) (len=5) "gtp5g",
upf |     IfList: ([]factory.IfInfo) (len=1 cap=1) {
upf |       (factory.IfInfo) {
upf |         Addr: (string) (len=15) "upf.free5gc.org",
upf |         Type: (string) (len=2) "N3",
upf |         Name: (string) "",
upf |         IfName: (string) "",
upf |         MTU: (uint32) 0
upf |       }
upf |     },
upf |     DnnList: ([]factory.DnnList) (len=2 cap=2) {
upf |       (factory.DnnList) {
upf |         Dnn: (string) (len=8) "internet",
upf |         Cidr: (string) (len=12) "10.60.0.0/16",
upf |         NatIfName: (string) ""
upf |       },
upf |       (factory.DnnList) {
upf |         Dnn: (string) (len=8) "internet",
upf |         Cidr: (string) (len=12) "10.61.0.0/16",
upf |         NatIfName: (string) ""
upf |       }
upf |     },
upf |     Logger: (*factory.Logger)(0xc000226ea0){
upf |       Enable: (bool) true,
upf |       Level: (string) (len=4) "info",
upf |       ReportCaller: (bool) false
upf |     }
upf |   },
upf | }
upf | 2025-12-14T13:31:13.868188073Z [INFO][UPF][CFG] =====
upf | 2025-12-14T13:31:13.868186215Z [INFO][UPF][Main] Log level is set to [info]
upf | 2025-12-14T13:31:13.868198788Z [INFO][UPF][Main] Report Caller is set to [false]
upf | 2025-12-14T13:31:13.872988954Z [INFO][UPF][Main] starting Gtpu Forwarder [gtp5g]
upf | 2025-12-14T13:31:13.873321518Z [INFO][UPF][Main] GTP Address: "upf.free5gupf"

```

Figure 5.18: UPF registro y preparacion de interfaz N3

5.2.4.3 Conexión N4 con SMF

En la figura 5.19 se aprecia que el UPF establece la conexión N4 (PFCP) con el SMF y queda listo para gestionar las sesiones de usuario y el tráfico de datos del UE.



```

upf | 2025-12-14T13:31:13.868108073Z [INFO][UPF][CFG] =====
upf | 2025-12-14T13:31:13.868186215Z [INFO][UPF][Main] Log level is set to [info]
upf | 2025-12-14T13:31:13.868198788Z [INFO][UPF][Main] Report Caller is set to [false]
upf | 2025-12-14T13:31:13.872988954Z [INFO][UPF][Main] starting Gtpu Forwarder [gtp5g]
upf | 2025-12-14T13:31:13.873321518Z [INFO][UPF][Main] GTP Address: "upf.free5gc.org:2152"

upf | 2025-12-14T13:31:13.990072956Z [INFO][UPF][BUFF] buff netlink server starupf | 2025-12-14T13:31:13.990072956Z [INFO][UPF]
[BUFF] buff netlink server started | 2025-12-14T13:31:13.990309613Z [INFO][UPF][Gtp5g] Forwarder started
upf | 2025-12-14T13:31:13.990309613Z [INFO][UPF][Gtp5g] Forwarder started
upf | 2025-12-14T13:31:13.993242593Z [INFO][UPF][Perio] perio server started
upf | 2025-12-14T13:31:13.993242593Z [INFO][UPF][Perio] perio server started
upf | 2025-12-14T13:31:13.993664129Z [INFO][UPF][PFCEP][LAddr:upf.free5gc.org:8805] starting pfcf server
upf | 2025-12-14T13:31:13.993820716Z [INFO][UPF][PFCEP][LAddr:upf.free5gc.org:8805] pfcf server started
upf | 2025-12-14T13:31:13.993844666Z [INFO][UPF][Main] UPF started
upf | 2025-12-14T13:31:13.993664129Z [INFO][UPF][PFCEP][LAddr:upf.free5gc.org:8805] starting pfcf server
upf | 2025-12-14T13:31:13.993820716Z [INFO][UPF][PFCEP][LAddr:upf.free5gc.org:8805] pfcf server started
upf | 2025-12-14T13:31:13.993844666Z [INFO][UPF][Main] UPF started
upf | 2025-12-14T13:34:08.844882890Z [INFO][UPF][PFCEP][LAddr:upf.free5gc.org:8805] handleAssociationSetupRequest
upf | 2025-12-14T13:34:08.844882890Z [INFO][UPF][PFCEP][LAddr:upf.free5gc.org:8805] handleAssociationSetupRequest
upf | 2025-12-14T13:34:08.848493277Z [INFO][UPF][PFCEP][LAddr:upf.free5gc.org:8805][CPNodeID:smf.free5gc.org] New node
upf | 2025-12-14T13:34:08.848493277Z [INFO][UPF][PFCEP][LAddr:upf.free5gc.org:8805][CPNodeID:smf.free5gc.org] New node

```

Figure 5.19: UPF registro y preparacion de interfaz N3

5.3 Implementación Red de Transporte con INT

El diseño de la red de transporte se encuentra en la grafica 5.20.

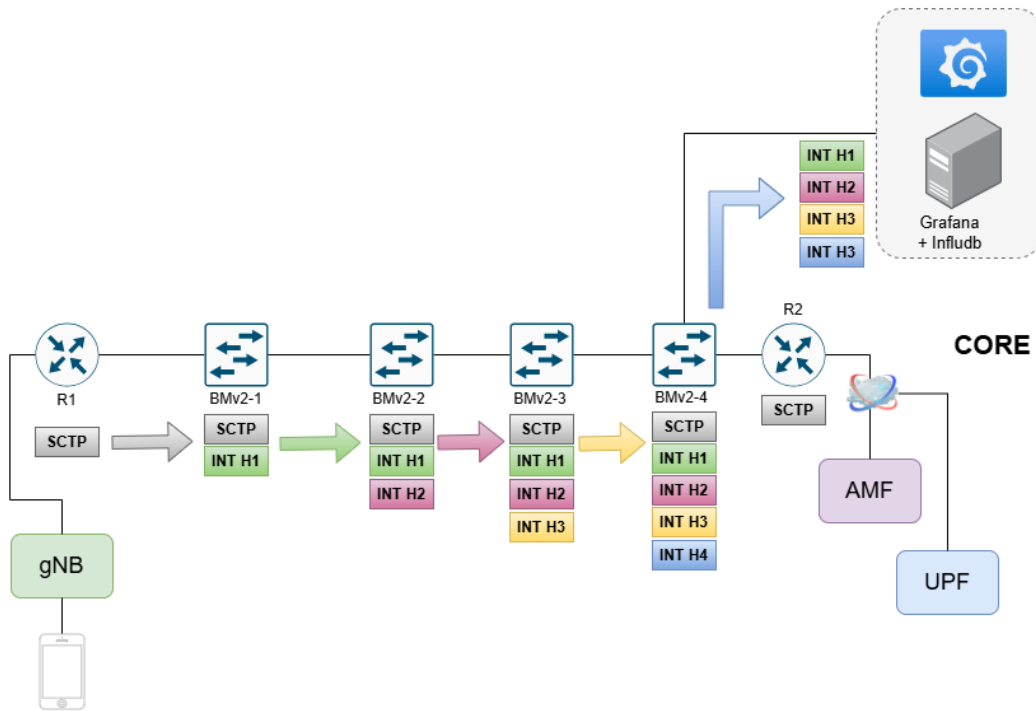


Figure 5.20: Topologia INT-MD

5.4 Despliegue de switches BMv2 con INT-MD

Los switches BMv2 se han desplegado utilizando GNS3, los cuales manejarán el tráfico SCTP y GTP-U entre el NGRAN (gNB) y el Core 5G (AMF y UPF) insertando metadatos INT-MD en los paquetes de datos. Estos metadatos reúnen los siguientes parámetros de telemetría de red:

- Switch ID
- Puerto de entrada
- Puerto de salida
- Latencia por salto
- Ocupación de cola
- Reporte de Congestión

5.4.1 Switch INT Source

El switch INT-Source (bmv2-1) fue construido para insertar metadatos INT-MD en el tráfico SCTP y GTP-U proveniente del NGRAN (gNB) hacia el Core 5G (AMF y UPF). Como el protocolo SCTP es quien transporta el protocolo NGAP en la interfaz N2, Todos los paquetes de attach por parte del usuario y el establecimiento de session con el AMF llevaran los metadatos INT-MD insertados por el switch INT-Source. De igual manera, todos los paquetes de datos del usuario transportados por GTP-U en la interfaz N3 entre el gNB y el UPF llevaran los metadatos INT-MD insertados por el switch INT-Source.

A continuación se detallan los bloques de código P4 implementados en el switch INT-Source para la inserción de metadatos INT-MD en el tráfico SCTP y GTP-U:

5.4.1.1 Definición de Constantes

```

1  #define MAX_HOP_COUNT 8
2  #define INT_INSTRUCTIONS_MASK 0x00FF
3  #define TYPE_INT 0xFA
4  #define PROTOCOL_IPV4 0x0800
5  #define PROTOCOL_SCTP 132
6  #define PROTOCOL_UDP 17
7  #define NGAP_PORT 38412
8  #define PPID_NGAP 60
9  #define GTP_U_PORT 2152

```

Código 5.15: Constantes INT

Con estas constantes se definen los parámetros necesarios para la implementación de INT, para esta prueba de concepto se ha definido un máximo de 8 saltos. Especificando los protocolos y puertos utilizados para identificar el tráfico SCTP (NGAP) y GTP-U y el tipo de encabezado INT.

5.4.1.2 Definición de Cabeceras

```

1  /***** Cabeceras comunes *****/
2  header ethernet_t {...}
3  header ipv4_t {...}
4  header udp_t {...}
5
6  /***** Cabeceras Estandar GTP-U, SCTP y NGAP *****/
7  header gtp_u_t {...}
8  header sctp_common_t {...}
9  header sctp_chunk_t {...}

```

```

10 header sctp_data_chunk_t {...}
11
12 /****** INT Shim *****/
13 header int_shim_t {
14     bit<8>  type;
15     bit<8>  reserved;
16     bit<16> length;
17     bit<16> next_proto;
18 }
19 /****** Cabecera INT *****/
20 header int_header_t {
21     bit<4>  version;
22     bit<2>  d;
23     bit<2>  q;
24     bit<5>  m;
25     bit<3>  reserved1;
26     bit<8>  hop_ml;
27     bit<16> instruction;
28     bit<8>  reserved2;
29     bit<8>  remaining_hop_cnt;
30 }
31
32 /****** Metadata por salto *****/
33 header int_md_t {
34     bit<16> padding_switch;
35     bit<16> switch_id;
36     bit<16> ingress_port_id;
37     bit<16> egress_port_id;
38     bit<32> hop_latency;
39     bit<32> queue_occupancy;
40     bit<32> ingress_timestamp;
41     bit<32> egress_timestamp;
42     bit<8>  congestion_notification;
43     bit<8>  reserved1;
44     bit<16> reserved2;
45 }
46 /****** Metadata interna del switch *****/
47 struct metadata {
48     bit<32> switch_id;
49     bit<9>  ingress_port;
50     bit<32> ingress_ts;
51     bit<32> egress_ts;
52     bit<8>  is_int_packet;
53     bit<9>  clone_port;
54     bit<32> queue_occupancy;
55     bit<8>  should_insert_int;
56     bit<16> transport_src_port;
57     bit<16> transport_dst_port;
58     bit<8>  is_ngap_traffic;

```

```

59     bit<8>  is_gtp_u_traffic;
60     bit<8>  traffic_type; // 0=other, 1=NGAP, 2=GTP-U
61 }
62 /****** Estructura de cabeceras *****/
63 struct headers {
64     ethernet_t ethernet;
65     int_shim_t int_shim;
66     int_header_t int_header;
67     int_md_t    int_md;
68     ipv4_t      ipv4;
69     udp_t       udp;
70     gtp_u_t     gtp_u;
71     sctp_common_t sctp;
72     sctp_chunk_t sctp_chunk;
73     sctp_data_chunk_t sctp_data_chunk;
74 }

```

Código 5.16: Definición de Cabeceras P4 para INT

En el bloque de código ?? se definen las cabeceras de los protocolos a monitorear (SCTP y GTP-U) y las de INT, donde se puede apreciar la reserva de espacio para un máximo de 8 saltos en la cabecera INT-MD, los parametros de telemetría a recolectar en cada salto y el tipo de encabezado INT. En la cabecera INT se define qué metadata se va a insertar en el paquete, cuántos saltos se han realizado, cuantos faltan y el modo INT; Es decir, si es Source, Transit o Sink.

La metadata por salto contiene los parámetros de telemetría que se van a recolectar en cada switch, como el ID del switch, latencia por salto, ocupación de cola, notificación de congestión, etc. Mientras que la Metadata interna del switch no se envía en el paquete, sino que se utiliza clasificar el trafico. Si el tipo de trafico es 1 (NGAP), 2 (GTP-U) o 0 para el resto. Dependiendo del tipo de trafico decide si se insertan los metadatos INT-MD y almacena temporalmente los valores de telemetría antes de insertarlos en el paquete INT.

5.4.1.3 Parser

```

1  parser MyParser(packet_in packet,
2      out headers hdr,
3      inout metadata meta,
4      inout standard_metadata_t standard_metadata) {
5
6      state start {
7          packet.extract(hdr.ethernet);
8          // Inicializacion de metadata INT
9          meta.ingress_port = standard_metadata.ingress_port;

```

```

10     meta.ingress_ts = (bit<32>)standard_metadata.
        ingress_global_timestamp;
11     meta.is_int_packet = 0;
12     meta.is_ngap_traffic = 0;
13     meta.is_gtp_u_traffic = 0;
14     meta.traffic_type = 0; // 0=other, 1=NGAP, 2=GTP-U
15     meta.should_insert_int = 0;
16     transition select(hdr.ethernet.etherType) {
17         0x0800: parse_ipv4;
18         default: accept;
19     }
20 }
21 // ... estados intermedios omitidos ...
22 state parse_udp {
23     packet.extract(hdr.udp);
24     meta.transport_src_port = hdr.udp.src_port;
25     meta.transport_dst_port = hdr.udp.dst_port;
26
27     // Deteccion de trafico GTP-U (interfaz N3)
28     transition select(hdr.udp.dst_port == GTP_U_PORT ||
29                       hdr.udp.src_port == GTP_U_PORT) {
30         true: parse_gtp_u;
31         false: accept;
32     }
33 }
34 state parse_gtp_u {
35     packet.extract(hdr.gtp_u);
36     meta.is_gtp_u_traffic = 1;
37     meta.traffic_type = 2; // Tráfico GTP-U
38     transition accept;
39 }
40 state parse_sctp_common {
41     packet.extract(hdr.sctp);
42     meta.transport_src_port = hdr.sctp.src_port;
43     meta.transport_dst_port = hdr.sctp.dst_port;
44
45     // Deteccion de trafico NGAP (interfaz N2)
46     if (hdr.sctp.dst_port == NGAP_PORT) {
47         meta.is_ngap_traffic = 1;
48         meta.traffic_type = 1; // Trafico de
            senalizacion
49     }
50     transition parse_sctp_chunk;
51 }
52
53 // ... resto de estados omitidos ...
54 }

```

Código 5.17: Parser P4 para INT

El bloque de código 5.17 reúne los estados mas relevantes del parser P4 implementado en el switch INT-Source. En el estado *start*, se inicializan las variables de metadata interna del switch, como el puerto de ingreso, timestamp de ingreso y flags para identificar si el paquete es INT, NGAP o GTP-U. Luego, en el estado *parse_udp*, se extrae la cabecera UDP y se almacenan los puertos de origen y destino en la metadata interna del switch. Si el puerto de destino o origen coincide con el puerto GTP-U (2152), se transiciona al estado *parse_gtp_u* para extraer la cabecera GTP-U y marcar el paquete como tráfico GTP-U en la metadata interna del switch. De manera similar, en el estado *parse_sctp_common*, se extrae la cabecera SCTP y se verifica si el puerto de destino coincide con el puerto NGAP (38412), marcando el paquete como tráfico NGAP en la metadata interna del switch. En caso contrario, se acepta el paquete sin marcarlo como tráfico NGAP o GTP-U.

5.4.1.4 Ingress

```

1  control MyIngress(inout headers hdr,
2                        inout metadata meta,
3                        inout standard_metadata_t
4                          standard_metadata) {
5
6      action int_source_action() {
7          // Indica que se debe insertar INT
8          meta.should_insert_int = 1;
9      }
10
11     action do_insert_int() {
12         // Almacena el Ethernet type original
13         bit<16> original_ether_type = hdr.ethernet.
14             etherType;
15
16         // Cambia el Ethernet type a INT
17         hdr.ethernet.etherType = TYPE_INT;
18
19         // Pone la cabecera INT shim
20         hdr.int_shim.setValid();
21         hdr.int_shim.type = 1;
22         hdr.int_shim.reserved = 0;
23         hdr.int_shim.length = 44; // Será actualizado por
24             switches de tránsito
25         hdr.int_shim.next_proto = original_ether_type;
26
27         // Pone la cabecera INT
28         hdr.int_header.setValid();
29         hdr.int_header.version = 1;
30         hdr.int_header.d = 0;

```

```

28     hdr.int_header.q = 0;
29     hdr.int_header.m = 1;
30     hdr.int_header.reserved1 = 0;
31     hdr.int_header.hop_ml = 1;
32     hdr.int_header.instruction = INT_INSTRUCTIONS_MASK;
33     hdr.int_header.reserved2 = 0;
34     hdr.int_header.remaining_hop_cnt = MAX_HOP_COUNT -
        1;
35
36     // Agrega bloque de metadatos
37     hdr.int_md.setValid();
38     hdr.int_md.padding_switch = 0;
39     hdr.int_md.switch_id = (bit<16>)meta.switch_id;
40     hdr.int_md.ingress_port_id = (bit<16>)meta.
        ingress_port;
41     hdr.int_md.ingress_timestamp = meta.ingress_ts;
42     hdr.int_md.egress_port_id = 0;
43     hdr.int_md.hop_latency = 0;
44     hdr.int_md.egress_timestamp = 0;
45     hdr.int_md.queue_occupancy = 0;
46     hdr.int_md.congestion_notification = 0;
47     hdr.int_md.reserved1 = 0;
48     hdr.int_md.reserved2 = 0;
49 }
50
51 action forward_action(bit<9> egress_port) {
52     standard_metadata.egress_spec = egress_port;
53 }
54
55 // Tabla 1: Verifica si la IP origen está configurada
para INT
56 table int_source_ip_table {
57     key = {
58         hdr.ipv4.srcAddr: lpm;
59     }
60     actions = {
61         int_source_action;
62         NoAction;
63     }
64     size = 1024;
65     default_action = NoAction();
66 }
67
68 // Tabla 2: Verifica el tipo de tráfico para las IP
marcadas para INT
69 table int_traffic_type_table {
70     key = {
71         meta.traffic_type: exact;
72     }

```

```

73     actions = {
74         int_source_action;
75         NoAction;
76     }
77     size = 1024;
78     default_action = NoAction();
79 }
80
81 table ipv4_lpm {
82     key = {
83         hdr.ipv4.dstAddr: lpm;
84     }
85     actions = {
86         forward_action;
87         NoAction;
88     }
89     size = 1024;
90     default_action = NoAction();
91 }
92
93 apply {
94     //Aquí es donde empieza la magia
95     meta.switch_id = 1;
96     if (hdr.ethernet.isValid() && hdr.ipv4.isValid()) {
97         // Primero: Verifica si la IP origen está
98         configurada para monitoreo INT y es válida.
99         if (hdr.ipv4.srcAddr != 0) {
100             int_source_ip_table.apply();
101         }
102
103         // Segundo: Si la IP origen coincide, verifica
104         si el tipo de tráfico es NGAP o GTP-U
105         if (meta.should_insert_int == 1) {
106             if ((hdr.ipv4.protocol == PROTOCOL_SCTP &&
107                 hdr.sctp.isValid()) ||
108                 (hdr.ipv4.protocol == PROTOCOL_UDP &&
109                 hdr.udp.isValid())) {
110                 int_traffic_type_table.apply();
111             } else {
112                 // Si no es SCTP o GTP-U, limpia el
113                 flag INT
114                 meta.should_insert_int = 0;
115             }
116
117             // Aplica el reenvío
118             ipv4_lpm.apply();
119
120             // Inserta cabeceras INT si el flag está en 1

```

```

117         if (meta.should_insert_int == 1) {
118             do_insert_int();
119         }
120     }
121 }
122 }

```

Código 5.18: Ingress P4 para INT

Como el bloque de código 5.18 Contiene comentarios en cada sección, no es necesario detallar su funcionamiento. En su lugar, es preciso destacar que la razón por la que se decidió implementar una tabla para especificar las IPs de origen que se le va a aplicar INT y otra tabla para verificar el tipo de tráfico (NGAP o GTP-U) es para optimizar el rendimiento del switch. De esta manera, se reduce la cantidad de paquetes que pasan por la lógica de inserción INT, ya que solo los paquetes que cumplen ambos criterios serán procesados para la inserción de metadatos INT-MD. Esto mejora la eficiencia del switch aportando mas flexibilidad para agregar mas IP de gnb en el futuro sin afectar el rendimiento del switch.

5.4.1.5 Egress

```

1  control MyEgress(inout headers hdr,
2                      inout metadata meta,
3                      inout standard_metadata_t
4                          standard_metadata) {
5
6      action update_int_metadata() {
7          if (hdr.int_md.isValid()) {
8              hdr.int_md.egress_port_id = (bit<16>)
9                  standard_metadata.egress_port;
10             hdr.int_md.egress_timestamp = (bit<32>)
11                 standard_metadata.egress_global_timestamp;
12             hdr.int_md.hop_latency = hdr.int_md.
13                 egress_timestamp - hdr.int_md.
14                 ingress_timestamp;
15             hdr.int_md.queue_occupancy = (bit<32>)
16                 standard_metadata.enq_qdepth;
17         }
18     }
19
20     apply {
21         if (hdr.int_shim.isValid()) {
22             update_int_metadata();
23         }
24     }
25 }

```


Código 5.19: Egress P4 para INT

El bloque de código 5.19 muestra el control Egress, donde se actualizan los campos de metadatos INT-MD con los valores correspondientes al egress port, egress timestamp, hop latency y queue occupancy antes de que el paquete salga del switch.

5.4.1.6 Deparser

```

1  control MyDeparser(packet_out packet, in headers hdr) {
2      apply {
3          // Always emit Ethernet
4          packet.emit(hdr.ethernet);
5
6          // Emit INT headers if valid
7          packet.emit(hdr.int_shim);
8          packet.emit(hdr.int_header);
9          packet.emit(hdr.int_md);
10
11         // Emit inner packet headers
12         packet.emit(hdr.ipv4);
13
14         // Emit transport layer headers - deparser only
15         emits valid headers
16         packet.emit(hdr.udp);
17         packet.emit(hdr.gtp_u);
18         packet.emit(hdr.sctp);
19         packet.emit(hdr.sctp_chunk);
20         packet.emit(hdr.sctp_data_chunk);
21     }
22 }
```

Código 5.20: Deparser P4 para INT

Finalmente, el bloque de código 5.20 muestra el deparser, el cual se encarga de reconstruir el paquete de salida, emitiendo primero la cabecera Ethernet, seguida de las cabeceras INT (si son válidas) y luego las cabeceras del paquete original (IPv4, UDP, GTP-U, SCTP, etc.). De esta manera, el paquete sale del switch con un tamaño mayor, específicamente **44 bytes** adicionales por la inserción de las cabeceras INT y los metadatos INT-MD. estos 44 bytes adicionales se desglosan de la siguiente manera:

- Cabecera INT Shim: 4 bytes
- Cabecera INT: 8 bytes

- Metadatos INT-MD: 32 bytes (para un máximo de 1 salto)
- Total: 44 bytes

5.4.2 Switch INT Transit

El INT-Transit (bmv2-2) fue construido para procesar los paquetes con metadatos INT-MD insertados por el switch INT-Source. Su función principal es actualizar los campos de metadatos INT-MD en cada salto, como el egress port, egress timestamp, hop latency y queue occupancy. Teniendo en cuenta que los aspectos generales sobre la definición de constantes, cabeceras, parser, egress y deparser son similares a los del switch INT-Source, se detallan únicamente las diferencias clave en la implementación del switch INT-Transit.

5.4.2.1 Parser

A diferencia del switch INT-Source, el INT-Transit tiene la capacidad para procesar paquetes que ya contienen metadatos INT-MD insertados.

```

1  parser MyParser(packet_in packet,
2                      out headers hdr,
3                      inout metadata meta,
4                      inout standard_metadata_t standard_metadata
5                      ) {
6
7      state start {
8          packet.extract(hdr.ethernet);
9          meta.ingress_port = standard_metadata.ingress_port;
10         meta.ingress_ts = (bit<32>)standard_metadata.
11             ingress_global_timestamp;
12         meta.md_blocks_extracted = 0;
13         meta.is_int_packet = 0;
14         transition select(hdr.ethernet.etherType) {
15             0x0800: parse_ipv4;           // IPv4 sin INT (no
16                 esperado en transit)
17             0xFA: parse_int_shim;        // Paquete con INT (
18                 esperado)
19             default: accept;
20         }
21     }
22
23     // Parsing de cabeceras INT
24     state parse_int_shim {
25         packet.extract(hdr.int_shim);
26         meta.is_int_packet = 1;          // Marcar como
27             paquete INT
28         transition parse_int_header;

```

```

24     }
25
26     state parse_int_header {
27         packet.extract(hdr.int_header);
28         // Calcular cuántos bloques de metadatos extraer
29         meta.md_blocks_extracted = hdr.int_header.hop_ml -
30                                 hdr.int_header.
31                                     remaining_hop_cnt;
32     }
33
34     state parse_int_metadata {
35         // Extraer bloques de metadatos existentes
36         // SW1 envió 1 bloque, así que extraemos int_md[0]
37         packet.extract(hdr.int_md[0]);
38         transition parse_encapsulated_protocol;
39     }
40
41     state parse_encapsulated_protocol {
42         transition select(hdr.int_shim.next_proto) {
43             0x0800: parse_encapsulated_ipv4; // IPv4
44                 encapsulado
45             default: accept;
46         }
47     }
48
49     state parse_encapsulated_ipv4 {
50         packet.extract(hdr.ipv4_inner);
51         transition select(hdr.ipv4_inner.protocol) {
52             PROTOCOL_SCTP: parse_encapsulated_sctp_common;
53             PROTOCOL_UDP: parse_encapsulated_udp;
54             default: accept;
55         }
56     }
57
58     // ... estados adicionales para procesar UDP/GTP-U o
59     // SCTP omitidos ...
60 }

```

Código 5.21: Parser INT-Transit

Al recibir el paquete, el parser del switch INT-Transit primero verifica el EtherType. Si detecta que es **0xFA** (TYPE_INT), extrae la cabecera INT Shim y marca el paquete como un paquete INT en la metadata interna del switch. Luego, extrae la cabecera INT Header y calcula cuántos bloques de metadatos ya han sido insertados por switches anteriores, almacenando este valor en la variable `md_blocks_extracted` de la metadata interna del switch. Una vez que se conocen los bloques existentes, el parser procede a extraer

esos bloques de metadatos INT-MD. Finalmente, extrae el paquete original (IPv4, SCTP/UDP, GTP-U) que está encapsulado dentro de las cabeceras INT.

5.4.2.2 Ingress

```

1  control MyIngress(inout headers hdr,
2                          inout metadata meta,
3                          inout standard_metadata_t
4                              standard_metadata) {
5
6      action forward_action(bit<9> egress_port) {
7          standard_metadata.egress_spec = egress_port;
8      }
9
10     table ipv4_lpm {
11         key = {
12             hdr.ipv4_inner.dstAddr: lpm;  // IP del paquete
13                                     encapsulado
14         }
15         actions = {
16             forward_action;
17             NoAction;
18         }
19         size = 1024;
20         default_action = NoAction();
21     }
22
23     apply {
24         meta.switch_id = 2;  // Identificador único del
25                             switch SW2
26
27         // Procesar paquetes con cabeceras INT válidas
28         if (hdr.int_shim.isValid()) {
29             // Verificar si hay espacio para añadir más
30             metadatos
31             if (hdr.int_header.remaining_hop_cnt > 0) {
32                 // SW1 usó int_md[0], SW2 añade en int_md
33                 [1]
34                 hdr.int_md[1].setValid();
35                 hdr.int_md[1].padding_switch = 0;
36                 hdr.int_md[1].switch_id = (bit<16>)meta.
37                     switch_id;
38                 hdr.int_md[1].ingress_port_id = (bit<16>)
39                     meta.ingress_port;
40                 hdr.int_md[1].ingress_timestamp = meta.
41                     ingress_ts;

```

```

35         // Campos que se actualizarán en Egress
36         hdr.int_md[1].egress_port_id = 0;
37         hdr.int_md[1].hop_latency = 0;
38         hdr.int_md[1].egress_timestamp = 0;
39         hdr.int_md[1].queue_occupancy = 0;
40         hdr.int_md[1].congestion_notification = 0;
41         hdr.int_md[1].reserved1 = 0;
42         hdr.int_md[1].reserved2 = 0;
43
44         // Actualizar contadores de la cabecera INT
45         hdr.int_header.remaining_hop_cnt =
46             hdr.int_header.remaining_hop_cnt - 1;
47         hdr.int_shim.length = hdr.int_shim.length +
48             32; // +32 bytes
49     }
50
51     // Forwarding basado en IP de destino del paquete
52     // encapsulado
53     if (hdr.ipv4_inner.isValid()) {
54         ipv4_lpm.apply();
55     }
56 }

```

Código 5.22: Ingress INT-Transit - Adición de metadatos INT a la pila existente

A este punto, el switch INT-Transit procesa todos los paquetes con cabeceras INT válidas, independientemente del protocolo encapsulado (NGAP o GTP-U). Antes de agregar un nuevo bloque, revisa si hay espacio disponible en la cabecera INT para insertar más metadatos, verificando el campo `remaining_hop_cnt`. Si este valor es mayor que cero, procede a insertar un nuevo bloque de metadatos INT-MD en la siguiente posición disponible del array `int_md[]`.

Un aspecto importante a destacar es que el switch INT-Transit añade **32 bytes adicionales** por cada salto procesado. Para fines de esta prueba de concepto, solo se cuenta con un switch INT-Transit (SW2).

5.4.2.3 Egress

El control Egress en el switch INT-Transit actualiza únicamente el bloque de metadatos que fue añadido por este switch en el Ingress. A diferencia del switch INT-Source, no actualiza todos los metadatos, sino solo el bloque correspondiente a su posición en la pila.

```

1  control MyEgress(inout headers hdr,
2                      inout metadata meta,

```

```

3          inout standard_metadata_t
              standard_metadata) {
4
5      apply {
6          if (hdr.int_shim.isValid()) {
7              if (hdr.int_md[1].isValid() &&
8                  hdr.int_md[1].egress_port_id == 0) {
9
10                 // Puerto de salida
11                 hdr.int_md[1].egress_port_id =
12                     (bit<16>)standard_metadata.egress_port;
13
14                 // Timestamp de salida
15                 hdr.int_md[1].egress_timestamp =
16                     (bit<32>)standard_metadata.
17                         egress_global_timestamp;
18
19                 // Latencia del salto (tiempo en el switch)
20                 hdr.int_md[1].hop_latency =
21                     hdr.int_md[1].egress_timestamp -
22                     hdr.int_md[1].ingress_timestamp;
23
24                 // Ocupación de la cola
25                 hdr.int_md[1].queue_occupancy =
26                     (bit<32>)standard_metadata.enq_qdepth;
27             }
28         }
29     }

```

Código 5.23: Egress INT-Transit

5.4.2.4 Deparser

El deparser del switch INT-Transit es similar al del switch INT-Source, pero con la particularidad de que debe emitir toda la pila de metadatos INT-MD que ha sido acumulada hasta ese punto.

```

1  control MyDeparser(packet_out packet, in headers hdr) {
2      apply {
3          // Emitir cabecera Ethernet
4          packet.emit(hdr.ethernet);
5
6          // Emitir cabeceras INT
7          packet.emit(hdr.int_shim);
8          packet.emit(hdr.int_header);
9
10         // Emitir TODOS los bloques de metadatos INT-MD

```

```

11      // Nota: packet.emit() solo emite cabeceras válidas
12      packet.emit(hdr.int_md[0]); // Metadatos de SW1
13      packet.emit(hdr.int_md[1]); // Metadatos de SW2 (
14      este switch)
15      packet.emit(hdr.int_md[2]); // Para switches
16      posteriores
17      packet.emit(hdr.int_md[3]);
18      packet.emit(hdr.int_md[4]);
19      packet.emit(hdr.int_md[5]);
20      packet.emit(hdr.int_md[6]);
21      packet.emit(hdr.int_md[7]); // MAX_HOP_COUNT = 8
22
23      // Emitir paquete encapsulado original
24      packet.emit(hdr.ipv4_inner);
25      packet.emit(hdr.udp);
26      packet.emit(hdr.gtp_u);
27      packet.emit(hdr.sctp);
28      packet.emit(hdr.sctp_chunk);
29      packet.emit(hdr.sctp_data_chunk);
30  }
31 }

```

Código 5.24: Deparser INT-Transit - Emisión de toda la pila INT

Es importante destacar que el método `packet.emit()` en P4 solo emite cabeceras que fueron marcadas como válidas mediante `setValid()`. Por lo tanto, aunque el deparser declara la emisión de los 8 posibles bloques de metadatos, solo se emitirán aquellos que fueron efectivamente añadidos por los switches en la ruta.

5.4.3 Switch INT Sink

El switch INT-Sink (bmv2-3) o INT-Destination, representa el último nodo en la ruta de telemetría INT, el cual se encarga de recibir los paquetes, clonarlos, enviar una copia al servidor de recolección y luego reenviar el paquete original hacia su destino final, que en este caso es el 5G Core a través de los Routers Core.

Para fines de optimización, se detallan solo los aspectos claves.

5.4.3.1 Parser

El parser del switch INT-Sink debe extraer todos los bloques de metadatos INT-MD que fueron insertados por los switches anteriores en la ruta. A diferencia del INT-Transit que solo necesita extraer los bloques anteriores, el INT-Sink debe procesar la pila completa para poder enviarla al servidor de recolección.

Como se mostró en la topología 5.20, el parser del INT-Sink debe extraer:

- **int_md[0]**: Metadatos insertados por SW1 (INT-Source)
- **int_md[1]**: Metadatos insertados por SW2 (INT-Transit)
- **int_md[2]**: Metadatos que serán completados por SW3 (INT-Sink) en Ingress/Egress

```

1  parser MyParser(packet_in packet,
2      // ... Condiciones detalladas anteriormente ...
3
4      state parse_int_metadata {
5          // Extrae todos los bloques de metadatos existentes
6          packet.extract(hdr.int_md[0]); // Metadatos de SW1
7          packet.extract(hdr.int_md[1]); // Metadatos de SW2
8          transition parse_encapsulated_protocol;
9      }
10
11     state parse_encapsulated_protocol {
12         transition select(hdr.int_shim.next_proto) {
13             0x0800: parse_encapsulated_ipv4;
14             default: accept;
15         }
16     }
17
18     // ... estados para procesar IPv4/UDP/GTP-U o SCTP
19     // encapsulados ...
20 }

```

Código 5.25: Parser INT-Sink - Extracción de metadatos completos

Se extrajeron los estados relevantes del parser en el bloque de código 5.25. Aquí, el parser extrae los bloques de metadatos `int_md[0]` y `int_md[1]` que fueron insertados por los switches anteriores. Luego, procede a extraer el paquete original encapsulado.

5.4.3.2 Ingress

A diferencia de los switches anteriores, el control Ingress del INT-Sink tiene funcionalidades adicionales para clonar el paquete y enviarlo al servidor de recolección.

```

1  control MyIngress(inout headers hdr,
2      inout metadata meta,
3      inout standard_metadata_t
4      standard_metadata) {

```



```

5  action int_sink_action() {
6      meta.clone_port = 9; // Puerto hacia el servidor
7      // Clona paquete I2E preservando la field_list 99
8      clone_preserving_field_list(CloneType.I2E, 99, 99);
9  }
10
11 action forward_action(bit<9> egress_port) {
12     standard_metadata.egress_spec = egress_port;
13 }
14
15 table int_sink_table {
16     key = {
17         hdr.int_shim.isValid(): exact; // Solo se
18         // activa para paquetes INT
19     }
20     actions = {
21         int_sink_action;
22         NoAction;
23     }
24     size = 1024;
25     default_action = NoAction();
26 }
27
28 table ipv4_lpm {
29     key = {
30         hdr.ipv4_inner.dstAddr: lpm; // IP del paquete
31         // encapsulado
32     }
33     actions = {
34         forward_action;
35         NoAction;
36     }
37     size = 1024;
38     default_action = NoAction();
39 }
40
41 apply {
42     meta.switch_id = 3; // Identificador de SW3
43
44     if (hdr.ethernet.isValid()) {
45         if (hdr.int_shim.isValid()) {
46             // Añadir metadatos de SW3 en int_md[2]
47             if (hdr.int_header.remaining_hop_cnt > 0) {
48                 hdr.int_md[2].setValid();
49                 hdr.int_md[2].padding_switch = 0;
50                 hdr.int_md[2].switch_id = (bit<16>)meta
51                     .switch_id;
52                 hdr.int_md[2].ingress_port_id = (bit

```

```

50         <16>)meta.ingress_port;
51         hdr.int_md[2].ingress_timestamp = meta.
52             ingress_ts;
53
54         // Campos que se actualizarán en Egress
55         hdr.int_md[2].egress_port_id = 0;
56         hdr.int_md[2].hop_latency = 0;
57         hdr.int_md[2].egress_timestamp = 0;
58         hdr.int_md[2].queue_occupancy = 0;
59         hdr.int_md[2].congestion_notification =
60             0;
61         hdr.int_md[2].reserved1 = 0;
62         hdr.int_md[2].reserved2 = 0;
63
64         // Actualizar contadores
65         hdr.int_header.remaining_hop_cnt =
66             hdr.int_header.remaining_hop_cnt -
67             1;
68         hdr.int_shim.length = hdr.int_shim.
69             length + 32;
70     }
71
72     // Activa clonación para enviar al servidor
73     // de recolección
74     int_sink_table.apply();
75 }
76
77 // Forwarding del paquete original hacia su destino
78 if (hdr.ipv4_inner.isValid()) {
79     ipv4_lpm.apply();
80 }
81 }

```

Código 5.26: Ingress INT-Sink - Clonación y forwarding dual

Como el Código 5.26 se encuentra bien comentado, no hay necesidad de explicarlo en detalle.

5.4.3.3 Egress

El control Egress del switch INT-Sink implementa la lógica más compleja de todos los switches INT, debido a que procesa tres tipos de paquetes:

1. **Paquete clonado** (`instance_type == 1`): Destinado al servidor de recolección, debe mantener todas las cabeceras INT intactas.

2. **Paquete original hacia el servidor** (`egress_port == 9`): También mantiene las cabeceras INT.
3. **Paquete original hacia el destino final**: Debe eliminar todas las cabeceras INT para restaurar el paquete a su formato original.

La acción `strip_int_headers()` es exclusiva del INT-Sink y se encarga de eliminar todas las cabeceras INT, restaurando el EtherType original y desencapsulando el paquete.

```

1  control MyEgress(inout headers hdr,
2                      inout metadata meta,
3                      inout standard_metadata_t
4                          standard_metadata) {
5
6      action update_int_metadata() {
7          // Asegura que int_md[2] sea válido antes de
8             actualizar
9          if (!hdr.int_md[2].isValid()) {
10             hdr.int_md[2].setValid();
11             hdr.int_md[2].padding_switch = 0;
12             hdr.int_md[2].switch_id = (bit<16>)meta.
13                 switch_id;
14             hdr.int_md[2].ingress_port_id = (bit<16>)meta.
15                 ingress_port;
16             hdr.int_md[2].ingress_timestamp = meta.
17                 ingress_ts;
18             // ... inicialización de otros campos ...
19         }
20
21         // Actualiza campos de egress
22         hdr.int_md[2].egress_port_id =
23             (bit<16>)standard_metadata.egress_port;
24         hdr.int_md[2].egress_timestamp =
25             (bit<32>)standard_metadata.
26                 egress_global_timestamp;
27         hdr.int_md[2].hop_latency =
28             hdr.int_md[2].egress_timestamp -
29             hdr.int_md[2].ingress_timestamp;
30         hdr.int_md[2].queue_occupancy =
31             (bit<32>)standard_metadata.enq_qdepth;
32     }
33
34     action strip_int_headers() {
35         // Restaura EtherType original del paquete
36             encapsulado
37         hdr.ethernet.etherType = hdr.int_shim.next_proto;
38     }
39 }

```

```
32      // Invalida cabeceras INT
33      hdr.int_shim.setInvalid();
34      hdr.int_header.setInvalid();
35
36      // Invalida todos los bloques de metadatos
37      hdr.int_md[0].setInvalid();
38      hdr.int_md[1].setInvalid();
39      hdr.int_md[2].setInvalid();
40      hdr.int_md[3].setInvalid();
41      hdr.int_md[4].setInvalid();
42      hdr.int_md[5].setInvalid();
43      hdr.int_md[6].setInvalid();
44      hdr.int_md[7].setInvalid();
45  }
46
47  table egress_control {
48      key = {
49          standard_metadata.egress_port: exact;
50      }
51      actions = {
52          strip_int_headers;
53          NoAction;
54      }
55      size = 16;
56      default_action = NoAction();
57  }
58
59  apply {
60      if (hdr.int_shim.isValid()) {
61          // actualiza metadatos de SW3 primero
62          update_int_metadata();
63
64          // Verifica tipo de paquete para su
65          // procesamiento
66          if (standard_metadata.instance_type == 1 ||
67              standard_metadata.egress_port == 9) {
68              // Paquete clonado o hacia servidor:
69              // mantiene INT intacto
70          } else {
71              // Paquete original hacia destino: elimina
72              // INT
73              egress_control.apply();
74          }
75      }
76  }
```

Código 5.27: Egress INT-Sink

La verificación `standard_metadata.instance_type == 1` identifica paquetes clonados (I2E clone), mientras que `egress_port == 9` cubre casos donde el paquete original también va al servidor. Cualquier otro puerto de salida activa la eliminación de cabeceras INT.

5.4.3.4 Deparser

El deparser del switch INT-Sink es similar al del INT-Transit, emitiendo todas las cabeceras en orden. Sin embargo, su comportamiento varía según el procesamiento en Egress:

- **Para paquetes hacia el servidor:** Emite Ethernet + INT Shim + INT Header + todos los bloques `int_md[]` + paquete encapsulado.
- **Para paquetes hacia el destino:** Emite solo Ethernet (con Ether-Type restaurado) + paquete original.

```

1 control MyDeparser(packet_out packet, in headers hdr) {
2     apply {
3         // Emite cabecera Ethernet
4         packet.emit(hdr.ethernet);
5
6         // Emite cabeceras INT (solo si siguen válidas)
7         // BMv2 automáticamente omite cabeceras inválidas
8         packet.emit(hdr.int_shim);
9         packet.emit(hdr.int_header);
10
11        // Emite todos los bloques de metadatos
12        packet.emit(hdr.int_md[0]); // SW1
13        packet.emit(hdr.int_md[1]); // SW2
14        packet.emit(hdr.int_md[2]); // SW3 (este switch)
15        packet.emit(hdr.int_md[3]);
16        packet.emit(hdr.int_md[4]);
17        packet.emit(hdr.int_md[5]);
18        packet.emit(hdr.int_md[6]);
19        packet.emit(hdr.int_md[7]);
20
21        // Emite paquete original
22        packet.emit(hdr.ipv4_inner);
23        packet.emit(hdr.udp);
24        packet.emit(hdr.gtp_u);
25        packet.emit(hdr.sctp);
26        packet.emit(hdr.sctp_chunk);
27        packet.emit(hdr.sctp_data_chunk);
28    }
29 }
```

Código 5.28: Deparser INT-Sink - Emisión condicional de cabeceras

El deparser del INT-Sink emite todas las cabeceras en el orden correcto. Si las cabeceras INT fueron invalidadas en Egress, BMv2 automáticamente las omite durante la emisión, asegurando que el paquete final hacia el destino no contenga ninguna información de telemetría.

5.4.4 Configuración de los Switches INT

A continuación, se muestran los comandos ejecutados para la configuración de las tablas de forwarding en cada uno de los switches INT (Source, Transit y Sink).

```

1  # Switch INT-Source
2  simple_switch_CLI --thrift-port 9090
3
4  table_clear MyIngress.ipv4_lpm
5  table_clear MyIngress.int_source_table
6  table_clear MyIngress.int_sink_table
7  table_add MyIngress.ipv4_lpm forward_action 172.18.10.2/32
   => 1
8  table_add MyIngress.ipv4_lpm forward_action 172.18.10.3/32
   => 1
9  table_add MyIngress.ipv4_lpm forward_action 0.0.0.0/0 => 2
   # GW SW2
10 table_add int_source_ip_table int_source_action
    172.18.10.3/32 =>
11 table_add int_traffic_type_table int_source_action 1 => #
    NGAP
12 table_add int_traffic_type_table int_source_action 2 => #
    GTP-U
13 #table_add MyIngress.int_source_table int_source_action 1
    => #NGAP
14 #table_add MyIngress.int_source_table int_source_action 2
    => #GTP-U
15
16 # Switch INT-Transit
17 simple_switch_CLI --thrift-port 9091
18 table_clear MyIngress.ipv4_lpm
19 table_add MyIngress.ipv4_lpm forward_action 172.18.0.20/32
   => 2
20 table_add MyIngress.ipv4_lpm forward_action 172.18.10.3/32
   => 1 # gNB/RAN
21 table_add MyIngress.ipv4_lpm forward_action 0.0.0.0/0 => 2
22
23 # Switch INT-Sink

```

```

24 simple_switch_CLI --thrift-port 9092
25 table_clear MyIngress.ipv4_lpm
26 table_clear MyIngress.int_sink_table
27 table_clear MyEgress.egress_control
28 table_add MyIngress.ipv4_lpm forward_action 172.18.0.20/32
    => 1 # AMF
29 table_add MyIngress.ipv4_lpm forward_action 172.18.0.24/32
    => 1 # UPF
30 table_add MyIngress.ipv4_lpm forward_action 172.18.10.3/32
    => 2 # gNB/RAN
31 table_add MyIngress.ipv4_lpm forward_action 0.0.0.0/0 => 1
32
33 # Para Clonar los INT
34 table_add MyIngress.int_sink_table int_sink_action 1 =>
35
36 # Para remover las cabeceras INT en el paquete original
37 table_add MyEgress.egress_control strip_int_headers 1 =>
38
39 # Enviar el paquete clonado al puerto 9 (servidor de
    recolección)
40 mirroring_add 99 9

```

Código 5.29: Configuración de tablas de forwarding en switches INT

5.5 Servidor de recolección y análisis

Para la recolección, análisis y visualización de los metadatos INT, se implementó un servidor (INT-Report) que captura los paquetes directamente de la interfaz de red conectada al switch INT-Sink y los almacena en un InfluxDB para su posterior visualización en Grafana. A continuación, se describen los componentes principales del servidor de recolección.

5.5.1 Instalación Grafana y InfluxDB

En la siguiente figura 5.21 Se muestran los comandos ejecutados para la instalación de Grafana e InfluxDB en el servidor de recolección INT-Report.

```
# Preparacion para instalaci3n de grafana
edinsonm@report-server:~$ sudo apt update
edinsonm@report-server:~$ sudo apt install -y software-properties-common apt-transport-https wget
edinsonm@report-server:~$ wget -q -O - https://apt.grafana.com/gpg.key | sudo gpg --dearmor | sudo tee
/usr/share/keyrings/grafana.gpg > /dev/null

edinsonm@report-server:~$ echo "deb [signed-by=/usr/share/keyrings/grafana.gpg] https://apt.grafana.com
stable main" | sudo tee /etc/apt/sources.list.d/grafana.list

edinsonm@report-server:~$ sudo apt update
edinsonm@report-server:~$

# Instalacion Grafana & Influxdb
edinsonm@report-server:~$ sudo apt install python3-pip python3-scapy influxdb influxdb-client grafana -y
edinsonm@report-server:~$

# Iniciando Servicios
edinsonm@report-server:~$ sudo systemctl enable influxdb --now
edinsonm@report-server:~$ sudo systemctl enable grafana-server --now
edinsonm@report-server:~$
```

Figure 5.21: Pantalla de inicio de Grafana

5.5.2 Creacion de la base de datos InfluxDB

En la figura 5.22 se muestran los comandos ejecutados para la creaci3n de la base de datos *int_telemetry* en InfluxDB, la cual almacenar3 los metadatos recolectados.

```
# Creaci3n de DB
edinsonm@report-server:~$ influx
>
> CREATE DATABASE int_telemetry;
> exit
edinsonm@report-server:~$
```

Figure 5.22: Creaci3n de la base de datos InfluxDB

5.5.3 Configuration de Grafana

Una vez que Grafana e InfluxDB están instalados, se procede a configurar Grafana para que utilice InfluxDB como fuente de datos. Para realizar esta configuración, se accede a la interfaz web de Grafana <http://localhost:3000>, con el usuario y contraseña por defecto (admin/admin). Una vez dentro de Grafana, en la sección de conexión de fuentes de datos se conecta a InfluxDB como se muestra en la figura 5.23.

En la figura 5.23 y 5.24 se muestra la pantalla de configuración de la fuente de datos en Grafana.

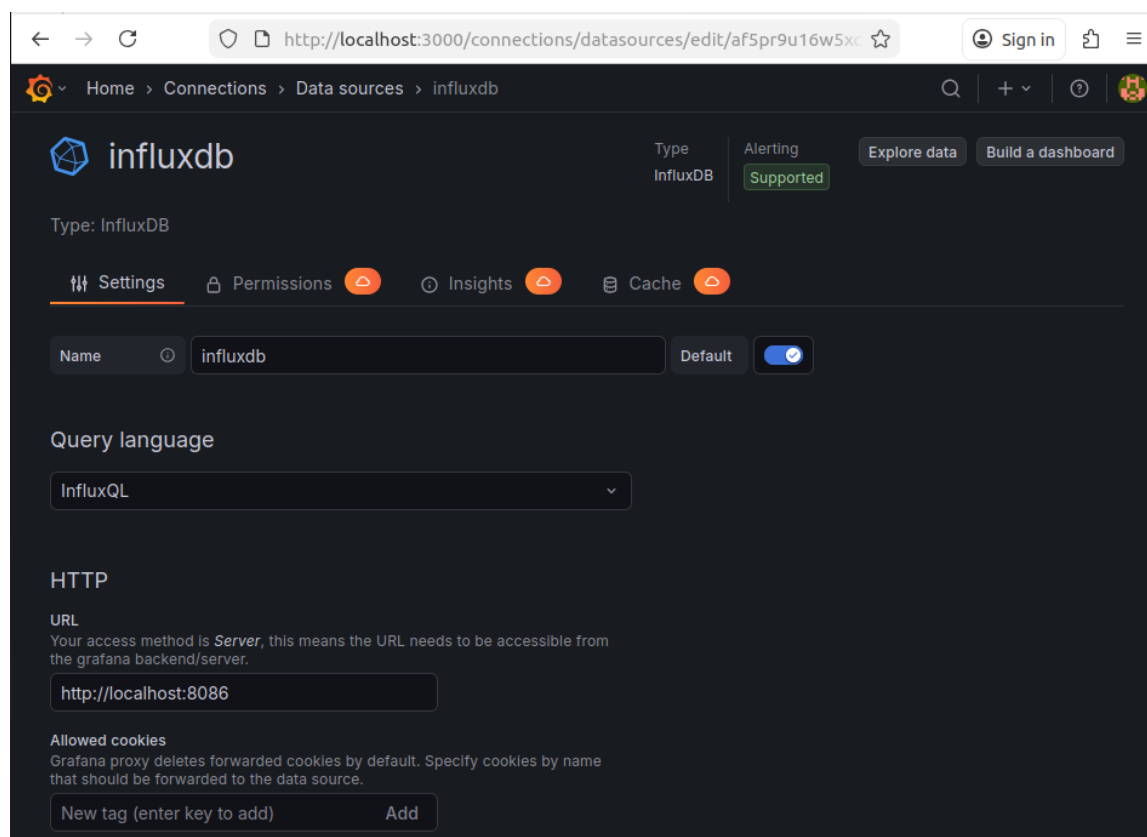


Figure 5.23: Configuración de la fuente de datos InfluxDB en Grafana

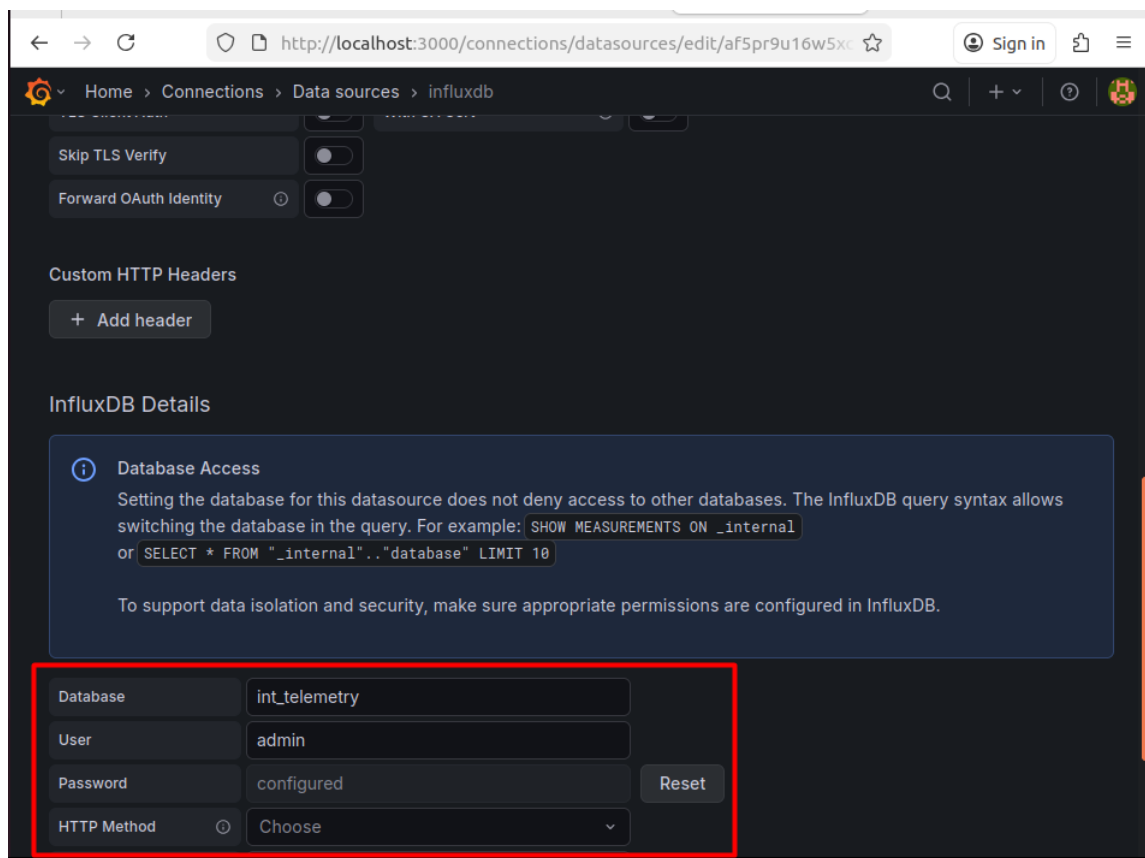


Figure 5.24: Configuración de la fuente de datos InfluxDB en Grafana - Parte 2

5.5.4 Captura y procesamiento de paquetes INT

Para la captura y procesamiento de los paquetes INT, se ha desarrollado un script en Python denominado `script-int-report.py` que utiliza la librería Scapy para capturar los paquetes directamente desde la interfaz de red conectada al switch INT-Sink. El script extrae los metadatos INT-MD de cada paquete y los almacena en la base de datos InfluxDB para su posterior análisis y visualización en Grafana.

Este script implementa un parser personalizado para el protocolo INT debido a que las herramientas estándar de análisis de paquetes (como Wireshark o tcpdump) no reconocen nativamente el EtherType 0x00FA utilizado por INT. El script opera en tiempo real, procesando cada paquete clonado que envía el switch INT-Sink al servidor de recolección.

5.5.4.1 Configuración y dependencias

El script comienza con la configuración de los parámetros esenciales para la conexión a InfluxDB y la captura de paquetes. Las principales dependencias utilizadas son:

- **Scapy**: Librería de Python para manipulación y análisis de paquetes de red a bajo nivel
- **InfluxDB Python Client**: Cliente oficial para conectarse a la base de datos de series temporales InfluxDB
- **struct**: Módulo de Python para conversión entre tipos de datos binarios y Python
- **binascii**: Para debugging y visualización de datos hexadecimales

```

1  from scapy.all import sniff
2  from influxdb import InfluxDBClient
3  import struct
4  import time
5
6  # --- Configuración ---
7  IFACE = "ens4"                # Interfaz de red conectada al
    INT-Sink
8  INFLUX_HOST = "localhost"
9  INFLUX_PORT = 8086
10 INFLUX_DB = "int_telemetry"
11
12 # --- Constantes ---
13 US_TO_MS = 1000.0            # Conversión microsegundos a
    milisegundos
14 INT_MD_LEN = 28              # Tamaño del bloque de metadatos
    INT-MD (28 bytes)
15
16 # Conectar a InfluxDB
17 client = InfluxDBClient(host=INFLUX_HOST, port=INFLUX_PORT,
18                          database=INFLUX_DB)
19
20 PARSER_VERSION = "v3-md28-endian-autodetect"
```

Código 5.30: Configuración inicial del script de captura INT

Un aspecto importante a destacar es que el tamaño del bloque de metadatos INT-MD es de **28 bytes en el wire** (224 bits), aunque el protocolo contabiliza en incrementos de 32 bytes (4-byte aligned) en el campo `int_shim.length`. Esta discrepancia se debe al padding y alineación de memoria en la implementación P4 de BMv2.

5.5.4.2 Detección automática de endianness

Uno de los desafíos más complejos en el parsing de paquetes INT es determinar el orden de bytes (endianness) correcto. Aunque el protocolo INT especifica big-endian (network byte order), en la práctica diferentes implementaciones de BMv2 pueden emitir los campos en little-endian debido a diferencias en la arquitectura del sistema.

El script implementa un sistema de **auto-detección heurística** que evalúa ambos órdenes de bytes y selecciona el más plausible mediante un sistema de puntuación:

```

1  def _score_int_md(values):
2      """Heurística para determinar el endianness más
3          plausible.
4
5          Los switch_id y ports son enteros pequeños (1..9).
6          Cuando el orden
7          de bytes es incorrecto, se convierten en valores como
8          256, 512, etc.
9      """
10     score = 0
11
12     switch_id = values["switch_id"]
13     ingress_port_id = values["ingress_port_id"]
14     egress_port_id = values["egress_port_id"]
15     queue_occupancy = values["queue_occupancy"]
16
17     # Switch IDs son enteros pequeños en este proyecto
18     if 1 <= switch_id <= 32:
19         score += 5
20     elif switch_id in (256, 512, 768, 1024):
21         score -= 5
22
23     # Puertos también son pequeños
24     for port in (ingress_port_id, egress_port_id):
25         if 0 <= port <= 64:
26             score += 2
27         elif port in (256, 512, 768, 1024, 2048):
28             score -= 2
29
30     # enq_qdepth no debe parecer un byte desplazado (0
31         xNN000000)
32     if queue_occupancy <= 200_000:
33         score += 2
34     if (queue_occupancy & 0x00FFFFFF) == 0 and
35         queue_occupancy >= 0x01000000:
36         score -= 6

```

```

33     return score
34
35 def _decode_int_md(md_data, endian):
36     """Decodificar bloque INT-MD usando endian '<' o '>'."""
37     switch_id = struct.unpack(f'{{endian}}H', md_data[2:4])
38         [0]
39     ingress_port_id = struct.unpack(f'{{endian}}H', md_data
40         [4:6])[0]
41     egress_port_id = struct.unpack(f'{{endian}}H', md_data
42         [6:8])[0]
43     hop_latency_field = struct.unpack(f'{{endian}}I', md_data
44         [8:12])[0]
45     queue_occupancy = struct.unpack(f'{{endian}}I', md_data
46         [12:16])[0]
47     ingress_timestamp = struct.unpack(f'{{endian}}I', md_data
48         [16:20])[0]
49     egress_timestamp = struct.unpack(f'{{endian}}I', md_data
50         [20:24])[0]
51     # ... extracción de otros campos ...
52
53     return {
54         "switch_id": switch_id,
55         "ingress_port_id": ingress_port_id,
56         "egress_port_id": egress_port_id,
57         "hop_latency_field": hop_latency_field,
58         "queue_occupancy": queue_occupancy,
59         "ingress_timestamp": ingress_timestamp,
60         "egress_timestamp": egress_timestamp,
61         # ...
62     }

```

Código 5.31: Sistema de auto-detección de endianness

El sistema de puntuación evalúa la plausibilidad de los valores decodificados. Por ejemplo, si al decodificar con big-endian obtenemos `switch_id = 256` pero con little-endian obtenemos `switch_id = 1`, el sistema asignará mayor puntuación a little-endian, ya que es más razonable tener un switch con ID 1 que con ID 256 en una topología pequeña.

5.5.4.3 Parsing de cabeceras INT

El parsing de paquetes INT comienza con la extracción manual de la cabecera Ethernet, seguida de la verificación del EtherType 0x00FA que identifica paquetes INT. Posteriormente, se extraen las cabeceras INT Shim e INT Header:

```

1 def parse_int_packet(pkt):

```

```
2      """Parsear paquete INT desde bytes L2.
3
4      Nota: INT usa EtherType 0x00FA que es < 0x0600, por lo
5          que
6          muchas herramientas lo clasifican como IEEE 802.3
7          length.
8          Debemos parsear los bytes directamente.
9      """
10     raw_data = bytes(pkt)
11     if len(raw_data) < 14:
12         return None
13
14     # Extraer cabecera Ethernet (14 bytes)
15     dst_mac = ':'.join(f"{b:02x}" for b in raw_data[0:6])
16     src_mac = ':'.join(f"{b:02x}" for b in raw_data[6:12])
17     type_or_len = struct.unpack('>H', raw_data[12:14])[0]
18
19     # Verificar EtherType INT
20     if type_or_len != 0x00FA:
21         return None
22
23     offset = 14
24
25     # Parsear INT Shim Header (6 bytes) - big-endian
26     shim_data = raw_data[offset:offset+6]
27     shim_type, shim_reserved, shim_length, shim_next_proto
28     = \
29     struct.unpack('>BBHH', shim_data)
30     print(f"[INT] Shim: type={shim_type}, length={
31         shim_length}, "
32         f"next_proto=0x{shim_next_proto:04x}")
33     offset += 6
34
35     # Parsear INT Header (7 bytes en teoría, pero puede
36     tener padding)
37     header_data = raw_data[offset:offset+7]
38     b0 = header_data[0]
39     b1 = header_data[1]
40     version = (b0 >> 4) & 0x0F
41     hop_ml = header_data[2]
42     instruction = struct.unpack('>H', header_data[3:5])[0]
43     remaining_hop_cnt = header_data[6]
44
45     print(f"[INT] Header: version={version}, hop_ml={hop_ml
46         }, "
47         f"instruction=0x{instruction:04x}, remaining_hops
48         ={remaining_hop_cnt}")
49
50     # Detectar padding automáticamente (7 u 8 bytes antes
```

```

    de metadatos)
44 # ... lógica de auto-detección ...
45
46 # Calcular número de bloques de metadatos desde shim.
    length
47 base_overhead = 12
48 num_blocks = (shim_length - base_overhead) // 32 if
    shim_length >= base_overhead else 0
49 num_blocks = min(num_blocks, 8) # MAX_HOP_COUNT
50
51 print(f"[INT] Extrayendo {num_blocks} bloques de
    metadatos")
52 # ...

```

Código 5.32: Parsing de cabeceras Ethernet e INT

El script implementa detección automática de padding entre la cabecera INT Header y los bloques de metadatos. Aunque la especificación indica 7 bytes (56 bits) para el INT Header, algunas implementaciones añaden 1 byte de padding para alineación a 8 bytes. El script prueba ambas configuraciones y selecciona la que produce valores más coherentes en el primer bloque de metadatos.

5.5.4.4 Extracción y validación de metadatos

La extracción de metadatos INT-MD es el componente más crítico del script. Cada bloque de 28 bytes contiene información de telemetría de un switch en la ruta. El script implementa validación robusta y recuperación de errores:

```

1 def parse_int_metadata_correct(md_data, hop_number):
2     """Parsear un bloque de metadatos INT (28 bytes).
3
4     Implementa auto-detección de endianness y recuperación
5     heurística de errores comunes.
6     """
7     if len(md_data) < INT_MD_LEN:
8         print(f"[ERROR] Bloque de metadatos muy corto: {len
9             (md_data)} bytes")
10        return None
11
12    # Auto-detectar endianness
13    be = _decode_int_md(md_data, '>') # Big-endian
14    le = _decode_int_md(md_data, '<') # Little-endian
15    be_score = _score_int_md(be)
16    le_score = _score_int_md(le)
17    chosen = le if le_score >= be_score else be
18    endian_used = 'le' if chosen is le else 'be'

```

```

19     switch_id = chosen["switch_id"]
20     ingress_port_id = chosen["ingress_port_id"]
21     egress_port_id = chosen["egress_port_id"]
22     hop_latency_field = chosen["hop_latency_field"]
23     queue_occupancy = chosen["queue_occupancy"]
24     ingress_timestamp = chosen["ingress_timestamp"]
25     egress_timestamp = chosen["egress_timestamp"]
26
27     # Calcular latencia desde timestamps si están
28     disponibles
29     ts_latency = None
30     if ingress_timestamp != 0 and egress_timestamp != 0:
31         ts_latency = (egress_timestamp - ingress_timestamp)
32         & 0xFFFFFFFF
33
34     # Recuperación heurística de errores comunes
35     # En algunos casos, los campos se desplazan debido a
36     errores
37     # de alineación: hop_latency=0, egress_ts=0, pero
38     queue_occupancy
39     # contiene un timestamp válido
40     recovered = False
41     if (hop_latency_field == 0 and egress_timestamp == 0
42         and ingress_timestamp > 1_000_000
43         and queue_occupancy > 1_000_000):
44         # Intercambiar campos desplazados
45         candidate_delta = (ingress_timestamp -
46                             queue_occupancy) & 0xFFFFFFFF
47         if candidate_delta < 1_000_000:
48             ingress_timestamp, egress_timestamp =
49                 queue_occupancy, ingress_timestamp
50             queue_occupancy = 0
51             ts_latency = (egress_timestamp -
52                           ingress_timestamp) & 0xFFFFFFFF
53             recovered = True
54
55     # Usar timestamp-derived latency si está disponible
56     hop_latency_used = ts_latency if ts_latency is not None
57     else hop_latency_field
58
59     # Convertir microsegundos a milisegundos
60     hop_latency_ms = hop_latency_used / US_TO_MS
61     ingress_timestamp_ms = ingress_timestamp / US_TO_MS
62     egress_timestamp_ms = egress_timestamp / US_TO_MS
63
64     print(f"[DEBUG] Hop {hop_number}: switch={switch_id}, "
65           f"ingress={ingress_port_id}, egress={
66               egress_port_id}, "
67           f"queue={queue_occupancy} packets, latency={

```



```

59         hop_latency_ms:.3f}ms "
60         f"[{endian_used}]" )
61     return {
62         "switch_id": switch_id,
63         "ingress_port_id": ingress_port_id,
64         "egress_port_id": egress_port_id,
65         "hop_latency": hop_latency_used,
66         "hop_latency_ms": hop_latency_ms,
67         "queue_occupancy": queue_occupancy,
68         "ingress_timestamp": ingress_timestamp,
69         "egress_timestamp": egress_timestamp,
70         "hop_number": hop_number,
71         "endian": endian_used,
72         "recovered": recovered
73     }

```

Código 5.33: Extracción y validación de metadatos INT-MD

La recuperación heurística de errores es particularmente importante cuando se detectan anomalías en los datos. Por ejemplo, si `hop_latency_field = 0` y `egress_timestamp = 0` pero `queue_occupancy` contiene un valor que parece un timestamp, el script intenta intercambiar los campos para recuperar los valores correctos.

5.5.4.5 Almacenamiento en InfluxDB

Una vez extraídos y validados los metadatos de todos los saltos, el script almacena la información en InfluxDB utilizando dos mediciones (measurements) diferentes:

1. **int_metrics**: Almacena metadatos individuales de cada salto (por-hop)
2. **int_path_metrics**: Almacena métricas agregadas de toda la ruta (end-to-end)

```

1  def handle_packet(pkt):
2      """Manejar paquetes INT entrantes"""
3      metadata_blocks = parse_int_packet(pkt)
4
5      if not metadata_blocks:
6          return
7
8      # Construir representación de la ruta
9      switch_ids = [f"SW{md['switch_id']}" for md in
                    metadata_blocks]

```

```

10 path_str = " -> ".join(switch_ids)
11 print(f" PATH: {path_str}")
12
13 # Almacenar cada bloque en InfluxDB
14 for md_block in metadata_blocks:
15     json_body = [{
16         "measurement": "int_metrics",
17         "tags": {
18             "switch_id": md_block["switch_id"],
19             "ingress_port": md_block["ingress_port_id"],
20             "egress_port": md_block["egress_port_id"],
21             "hop_number": md_block["hop_number"],
22             "parser_version": PARSER_VERSION,
23             "parser_endian": md_block["endian"],
24         },
25         "fields": {
26             "hop_latency_us": md_block["hop_latency"],
27             "hop_latency_ms": md_block["hop_latency_ms"],
28             "queue_occupancy_packets": md_block["queue_occupancy"],
29             "ingress_timestamp_us": md_block["ingress_timestamp"],
30             "egress_timestamp_us": md_block["egress_timestamp"],
31             "ingress_timestamp_ms": md_block["ingress_timestamp_ms"],
32             "egress_timestamp_ms": md_block["egress_timestamp_ms"],
33         },
34         "time": int(time.time() * 1e9) # Nanosegundos
35     }]
36     client.write_points(json_body)
37
38 # Calcular y almacenar latencia end-to-end
39 # IMPORTANTE: No restar timestamps entre switches
40     diferentes
41 # porque cada BMv2 tiene su propio dominio de tiempo.
42 # En su lugar, sumar las latencias individuales de cada
43     salto.
44
45 e2e_latency_us = sum(md["hop_latency"] for md in
46     metadata_blocks)
47 e2e_latency_ms = e2e_latency_us / US_TO_MS
48
49 path_json_body = [{
50     "measurement": "int_path_metrics",
51     "tags": {
52         "path": path_str,

```

```

49         "hop_count": len(metadata_blocks),
50     },
51     "fields": {
52         "end_to_end_latency_us": e2e_latency_us,
53         "end_to_end_latency_ms": e2e_latency_ms,
54         "total_hops": len(metadata_blocks),
55     },
56     "time": int(time.time() * 1e9)
57 }
58 client.write_points(path_json_body)
59
60 print(f" E2E latency (sum): {e2e_latency_ms:.3f}ms ({
    e2e_latency_us}us)")

```

Código 5.34: Almacenamiento de metadatos en InfluxDB

Un aspecto crucial a destacar es el cálculo de la latencia end-to-end. Dado que cada instancia de BMv2 (cada switch) tiene su propio dominio de tiempo independiente, **no es válido restar timestamps entre switches diferentes**. Por ejemplo, `SW3.ingress_timestamp - SW1.ingress_timestamp` puede resultar en valores negativos o sin sentido. Por ello, la latencia end-to-end se calcula como la **suma de las latencias individuales** de cada salto, que es la métrica correcta para medir el tiempo total que el paquete pasó en la infraestructura de red.

5.5.4.6 Ejecución y monitoreo

El script se ejecuta como un proceso en segundo plano que escucha continuamente en la interfaz de red especificada. La función principal inicializa la conexión a InfluxDB y comienza la captura de paquetes:

```

1  def main():
2      print("=== INT Telemetry Report Server ===")
3      print(f"Conversion: 1 ms = {US_TO_MS:.0f} us")
4      print("=" * 50)
5
6      # Verificar conexión a InfluxDB
7      if not test_influxdb():
8          return
9
10     print(f"Escuchando en interfaz: {IFACE}")
11     print("Esperando paquetes de telemetría INT...")
12     print("Presiona Ctrl+C para detener\n")
13
14     try:
15         # Captura continua de paquetes
16         sniff(iface=IFACE, prn=handle_packet, store=False)

```

```

17     except KeyboardInterrupt:
18         print("\n Detenido por el usuario")
19     except Exception as e:
20         print(f"Error: {e}")
21
22 if __name__ == "__main__":
23     main()

```

Código 5.35: Función principal y ejecución del script

El script genera salida de debugging detallada que permite monitorear en tiempo real el procesamiento de paquetes INT:

```

[INT] New packet: aa:bb:cc:dd:ee:ff -> 11:22:33:44:55:66, length: 198 bytes
[INT] Shim: type=1, length=108, next_proto=0x0800
[INT] Header: version=1, hop_ml=3, instruction=0x00ff, remaining_hops=0
[INT] Extrayendo 3 bloques de metadatos
[DEBUG] Hop 1: switch=1, ingress=1, egress=2, queue=0 packets,
        latency=0.245ms [1e]
[DEBUG] Hop 2: switch=2, ingress=1, egress=2, queue=0 packets,
        latency=0.187ms [1e]
[DEBUG] Hop 3: switch=3, ingress=1, egress=9, queue=0 packets,
        latency=0.156ms [1e]
PATH: SW1 -> SW2 -> SW3
E2E latency (sum): 0.588ms (588us)
Saved to DB: Switch 1, Hop 1, Latency: 0.245ms
Saved to DB: Switch 2, Hop 2, Latency: 0.187ms
Saved to DB: Switch 3, Hop 3, Latency: 0.156ms
Saved path to DB: SW1 -> SW2 -> SW3, E2E latency: 0.588ms

```

Este nivel de detalle facilita el debugging y permite verificar que el parsing se está realizando correctamente, mostrando información como el endianness detectado, la ruta completa del paquete, y las métricas individuales de cada salto.

5.6 Despliegue del NGRAN con UERANSIM

El NGRAN se ha desplegado utilizando la herramienta UERANSIM, la cual simula tanto el gNB como el UE en modo 5G Standalone (SA). UERANSIM se ha configurado para conectarse al Core 5G SA desplegado previamente, estableciendo el enlace N2 (SCTP) con el AMF y el enlace N3 (GTP-U) con el UPF.

5.6.1 Despliegue de UERANSIM

El despliegue de UERANSIM se realiza mediante Docker Compose, creando un archivo llamado *docker-compose-build-ngran.yaml* que contiene la configuración tanto del gNB como del UE. El archivo Docker Compose se muestra en el siguiente código 5.36.

```

1 version: "3.8"
2 services:
3   ueransim:
4     container_name: ueransim
5     build:
6       context: ./ueransim
7     command: ./nr-gnb -c ./config/gnbcfg.yaml
8     expose:
9       - "38412"
10    volumes:
11      - ./config/gnbcfg.yaml:/ueransim/config/gnbcfg.yaml
12      - ./config/uecfg.yaml:/ueransim/config/uecfg.yaml
13    cap_add:
14      - NET_ADMIN
15    devices:
16      - "/dev/net/tun"
17    ports:
18      - "38412:38412"
19    networks:
20      macvlan_net:
21        ipv4_address: 172.18.10.3
22        mac_address: f6:b8:eb:4d:0f:3c
23        aliases:
24          - gnb.free5gc.org
25    extra_hosts:
26      - "amf.free5gc.org:172.18.0.20"
27      - "upf.free5gc.org:172.18.0.24"
28 networks:
29   macvlan_net:
30     external: true

```

Código 5.36: Configuración del Docker Compose UERANSIM

5.6.2 Configuración del gNB

La configuración del gNB se realiza mediante el archivo *gnb.yaml* el cual se describirá mas a detalle en el capítulo de los resultados, donde se especifican los parámetros necesarios para la conexión con el Core 5G SA, como la dirección IP del AMF, el PLMN ID, y los parámetros de la interfaz N3. Un ejemplo de configuración del gNB se muestra en la figura ??.

Chapter 6

Resultados

6.1 Validación funcional

6.1.1 INT en tráfico N2

6.1.2 INT en tráfico N3

6.1.3 Recepción de metadatos en el servidor

6.2 Presentación de resultados

Chapter 7

Conclusiones y Trabajo Futuro

7.1 Conclusiones principales

7.2 Contribuciones del trabajo

7.3 Limitaciones del estudio

7.4 Líneas futuras de investigación

7.4.1 INT en 6G

7.4.2 UPF programable con P4

7.4.3 IA para análisis de telemetría

Chapter A

Apéndices

A.1 Código

A.2 Topologías de red

A.3 Configuraciones del core 5G

A.4 Capturas de tráfico

A.5 Dashboards de Grafana

A.6 Scripts y herramientas auxiliares

Bibliography

- [1] 3GPP. 3gpp ts 23.501: System architecture for the 5g system (5gs). Technical report, 2021.
- [2] Docker Inc. Install docker desktop on ubuntu, 2024.
- [3] Free5GC Community. Free5gc: Open source 5g core network, 2021.
- [4] Ulises O'Neill, Amelia Mackey, and Victor C.M. Leung. *5G Core Networks: A Comprehensive Guide to 5GC Architecture and Deployment*. O'Reilly Media, 1st edition, 2021.